

INFORMATIQUE • TLE TI

MODULE 3 : SYSTÈMES D'INFORMATION ET BASES DE DONNÉES

LEÇON N14

IMPLÉMENTATION ET ADMINISTRATION D'UNE BASE DE DONNÉES (SGBD, SQL)

DURÉE CONSEILLÉE : 22 H

DANS CETTE LEÇON

Cette leçon fait passer l'élève de la conception (MCD) à la mise en œuvre concrète d'une base de données : choix d'un SGBD, traduction du MCD en schéma relationnel, création des tables avec leurs contraintes d'intégrité, puis écriture de requêtes SQL de manipulation et d'extraction (sélection, agrégation, jointure, tri, regroupement). Elle se conclut par l'exportation, l'importation et la sauvegarde de la base, compétences directement exigibles au Bac TI.

- À la fin de cette leçon, je sais transformer un MCD en schéma relationnel en appliquant les règles de passage (entités, associations, cardinalités)
- À la fin de cette leçon, je sais définir le rôle d'un SGBD et citer ses caractéristiques et des exemples (MySQL, PostgreSQL, ...)
- À la fin de cette leçon, je sais créer et supprimer une base de données et des tables avec CREATE, ALTER et DROP
- À la fin de cette leçon, je sais choisir les types de données et poser les contraintes d'intégrité (clé primaire, clé étrangère, NOT NULL, UNIQUE, DEFAULT, CHECK)
- À la fin de cette leçon, je sais insérer, modifier et supprimer des enregistrements avec INSERT, UPDATE et DELETE
- À la fin de cette leçon, je sais écrire des requêtes SELECT avec conditions (opérateurs logiques, de comparaison, LIKE, BETWEEN, IN, NOT IN)



TÉLÉCHARGE ONBUCH

Gratuit • Google Play
Tuteur IA, annales,
concours, résultats



PASSE À ONBUCH+

Tous les cours signés, Tuteur IA
illimité, fiches PDF — onglet
OnBuch+ de l'app

CE COURS SIGNÉ CONTIENT

COURS

complet, illustré, pas à pas

MÉTHODES

et exercices résolus

EXERCICES

type examen + corrigés

FICHE BILAN

l'essentiel à retenir

**Signé • OnBuch+**

Ludovic A.

CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+

SOMMAIRE

1. Du MCD au schéma relationnel
2. SGBD : rôle, caractéristiques et exemples
3. Définition des données : tables, types et contraintes d'intégrité
4. Manipulation des données : INSERT, UPDATE, DELETE, SELECT
5. Agrégation, tri, regroupement et jointures
6. Administration : exportation, importation et sauvegarde
7. Activité d'intégration
8. Méthodes et exercices résolus
9. Exercices
10. Corrigés des exercices
11. Fiche bilan

OBJECTIFS DE LA LEÇON

- À la fin de cette leçon, je sais transformer un MCD en schéma relationnel en appliquant les règles de passage (entités, associations, cardinalités)
- À la fin de cette leçon, je sais définir le rôle d'un SGBD et citer ses caractéristiques et des exemples (MySQL, PostgreSQL, ...)
- À la fin de cette leçon, je sais créer et supprimer une base de données et des tables avec CREATE, ALTER et DROP
- À la fin de cette leçon, je sais choisir les types de données et poser les contraintes d'intégrité (clé primaire, clé étrangère, NOT NULL, UNIQUE, DEFAULT, CHECK)
- À la fin de cette leçon, je sais insérer, modifier et supprimer des enregistrements avec INSERT, UPDATE et DELETE
- À la fin de cette leçon, je sais écrire des requêtes SELECT avec conditions (opérateurs logiques, de comparaison, LIKE, BETWEEN, IN, NOT IN)
- À la fin de cette leçon, je sais utiliser les fonctions d'agrégation (COUNT, AVG, MIN, MAX, SUM) avec GROUP BY, HAVING et ORDER BY
- À la fin de cette leçon, je sais croiser plusieurs tables avec des jointures et nommer les résultats avec des alias
- À la fin de cette leçon, je sais exporter, importer et sauvegarder une base de données dans PhpMyAdmin/MySQL

PRÉREQUIS

- Notion de système d'information et de base de données (Première TI)
- Modélisation avec le MCD : entités, attributs, associations, cardinalités (leçon précédente de Terminale)
- Notions de table, champ et enregistrement vues en Première
- Utilisation élémentaire d'un environnement de travail (WAMP/XAMPP, PhpMyAdmin)

1 DU MCD AU SCHÉMA RELATIONNEL

Le lycée bilingue de Bafoussam gère encore ses 2 400 élèves dans des cahiers : notes, paiements de scolarité, absences. À chaque conseil de classe, le censeur passe trois jours à recouper les informations, et les erreurs se multiplient : un élève radié reçoit encore des bulletins, deux élèves portent le même matricule. Le proviseur mandate ton groupe TI : modéliser ces données, les implémenter dans un SGBD et permettre au secrétariat d'extraire en quelques secondes la liste des élèves de Tle TI n'ayant pas soldé la scolarité.

Problématique : **comment passer d'un cahier papier à une base de données fiable, interrogeable et sauvegardable ?**

La réponse commence par une étape décisive : transformer le **MCD** (Modèle Conceptuel de Données), conçu lors de l'analyse, en un **schéma relationnel**, c'est-à-dire en un plan précis de tables qu'un SGBD comme MySQL saura créer. C'est l'objet de cette section.

1.1 Rappel : le Modèle Conceptuel de Données (MCD)

1.2 Pourquoi un modèle conceptuel ?

Avant d'écrire la moindre ligne de SQL, il faut comprendre les données du monde réel. Le MCD est un schéma graphique, indépendant de tout logiciel, qui représente les objets manipulés par l'organisation et leurs liens. Il sert de contrat entre le client (le proviseur) et l'informaticien (toi) : tant que le MCD n'est pas validé, on ne code rien.

DÉFINITION Les briques du MCD

- Une **entité** représente un objet du monde réel, doté d'une existence propre (ex. : ELEVE, CLASSE, MATIERE).
- Un **attribut** est une propriété d'une entité (ex. : nom, dateNaissance, coefficient).
- L'**identifiant** est l'attribut (ou le groupe d'attributs) qui distingue de façon unique chaque occurrence de l'entité (ex. : matricule). On le souligne.
- Une **association** est un lien sémantique entre entités (ex. : INSCRIRE relie ELEVE et CLASSE).
- Les **cardinalités** indiquent, pour chaque patte de l'association, le nombre minimal et maximal de participations d'une occurrence de l'entité : (1,1), (0,1), (1,n), (0,n).

EXEMPLE Lire des cardinalités

Dans le MCD du lycée : un élève est inscrit dans une et une seule classe (cardinalité 1,1 côté ELEVE), et une classe accueille au moins un, au plus plusieurs élèves (cardinalité 1,n côté CLASSE). L'association INSCRIRE est donc de type (1,1) - (1,n).

1.3 Le schéma relationnel : le plan des tables

Un SGBD relationnel (MySQL, PostgreSQL...) ne connaît qu'une seule structure : la **table**. Il faut donc traduire le MCD en un ensemble de tables reliées entre elles. C'est le schéma relationnel.

DÉFINITION Schéma relationnel, clé primaire, clé étrangère

- Un **schéma relationnel** est la description des tables d'une base : pour chaque table, son nom suivi de la liste de ses colonnes, avec la clé primaire soulignée et les clés étrangères repérées par #.
- La **clé primaire** d'une table est une colonne (ou un groupe de colonnes) dont la valeur identifie de manière unique chaque ligne : jamais nulle, jamais dupliquée.
- Une **clé étrangère** est une colonne d'une table qui reprend la clé primaire d'une autre table, afin de matérialiser un lien entre les deux.

Notation : CLASSE(idClasse, libelle, niveau) et ELEVE(matricule, nom, #idClasse).

LESAVAIS-TU ? Edgar F. Codd, père du modèle relationnel

Le modèle relationnel a été inventé en 1970 par le Britannique Edgar F. Codd, chercheur chez IBM, dans son article « A Relational Model of Data for Large Shared Data Banks ». Son idée révolutionnaire : représenter toutes les données sous forme de tables mathématiques (relations) et les manipuler avec une algèbre rigoureuse. IBM n'y crut pas immédiatement ; c'est Oracle qui commercialisa le premier SGBD relationnel en 1979. Codd reçut le prix Turing en 1981. Aujourd'hui, MySQL, PostgreSQL, Oracle et SQL Server — qui font tourner les systèmes de MTN Mobile Money ou des impôts en ligne — appliquent tous son modèle.

1.4 Les trois règles de passage du MCD au relationnel

Le passage du MCD au schéma relationnel n'est pas un art : c'est un **algorithme** composé de trois règles mécaniques. Appliquées correctement, elles garantissent un schéma sans redondance ni perte d'information.

MÉTHODE Les 3 règles de transformation du MCD en schéma relationnel

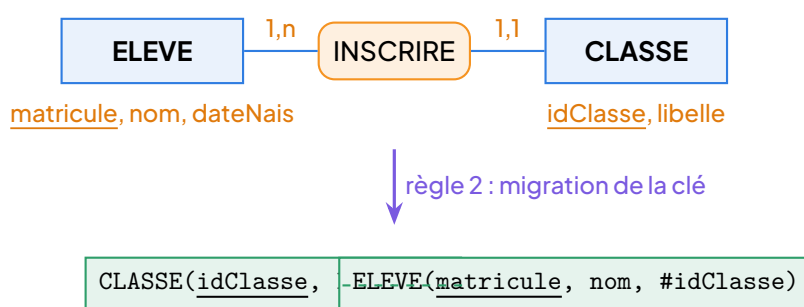
1. Règle 1 — Entité. Toute entité devient une table. Ses attributs deviennent les colonnes, et son identifiant devient la clé primaire (soulignée).
2. Règle 2 — Association $(x,1) - (x,n)$. Une association binaire dont une patte a une cardinalité maximale de 1 ne devient pas une table. On fait migrer la clé primaire du côté « n » vers le côté « 1 » : elle devient une clé étrangère dans la table du côté « 1 ». Les éventuels attributs de l'association migrent avec elle.

3. Règle 3 — Association $(x,n) - (x,n)$. Une association dont les deux pattes ont une cardinalité maximale « n » devient une table dont la clé primaire est la concaténation des clés primaires des entités reliées (ces colonnes sont à la fois clé primaire composée et clés étrangères). Ses attributs propres deviennent des colonnes de cette table.

ATTENTION Erreur fréquente : créer une table pour une association $(1,n)$

Beaucoup d'élèves créent systématiquement une table par association. C'est faux pour une association $(x,1) - (x,n)$: créer une table `INSCRIRE` avec sa propre clé primaire ajouterait une table inutile, des jointures supplémentaires et des risques d'incohérence (un élève pourrait apparaître dans deux classes !). La clé étrangère migrée impose naturellement qu'un élève n'a qu'une seule valeur de `#idClasse` : la contrainte $(1,1)$ est ainsi garantie par construction. Réserve la création d'une table aux associations (n,n) — et aux cas particuliers vus plus bas.

1.5 Illustration : l'association $(1,1) - (1,n)$ `INSCRIRE`



Transformation de l'association $(1,n)$ `INSCRIRE` : la clé primaire `idClasse` migre comme clé étrangère dans la table `ELEVE`.

Le tableau suivant résume les trois règles de passage :

Élément du MCD	Traduction relationnelle	Exemple
Entité	Table ; identifiant → clé primaire	<code>MATIERE(idMat, libelle, coef)</code>
Association $(x,1) - (x,n)$	Clé étrangère migrée du côté « n » vers le côté « 1 »	<code>ELEVE(..., #idClasse)</code>
Association $(x,n) - (x,n)$	Table ; clé primaire = concaténation des clés reliées	<code>NOTE(#matricule, #idMat, valeur)</code>

1.6 Cas particuliers : associations porteuses et associations réflexives

Association porteuse de données. Une association peut posséder ses propres attributs. Par exemple, EVALUER relie un élève à une matière et porte l'attribut `note` (la note obtenue) : cette note n'appartient ni à l'élève seul, ni à la matière seule, mais bien au couple. Selon la règle 3, ces attributs deviennent des colonnes de la table issue de l'association : `NOTE(#matricule, #idMat, note, trimestre)`. Pour une association $(1,n)$ porteuse, les attributs migrent avec la clé étrangère (règle 2).

Association unaire (réflexive). Une entité peut être en relation avec elle-même. Exemple : ELEVE — PARRAINER — ELEVE (un élève parraine au plus un nouvel élève ; un nouvel élève est parrainé par au plus un ancien). On applique les mêmes règles : ici, cardinalités $(0,1) - (0,1)$, donc on migre la clé d'un côté. La table devient `ELEVE(matricule, nom, #matriculeParrain)` : la clé étrangère référence la clé primaire de la même table. Si l'association réflexive est (n,n) (ex. : AMITIE entre élèves), elle devient une table à deux clés étrangères vers la même entité, renommées pour être distinguées : `AMITIE(#matricule1, #matricule2)`.

1.7 Exemple fil rouge : le MCD « Lycée » traduit en schéma relationnel

Reprenons la situation du lycée de Bafoussam. L'analyse a produit le MCD suivant :

- `ELEVE(matricule, nom, prenom, dateNaissance, sexe)` ;
- `CLASSE(idClasse, libelle, niveau)` ;
- `MATIERE(idMatiere, libelle, coefficient)` ;
- `INSCRIRE` : association $(1,1) - (1,n)$ entre ELEVE et CLASSE, porteuse de `anneeScolaire` ;
- `EVALUER` : association $(1,n) - (1,n)$ entre ELEVE et MATIERE, porteuse de `note` et `trimestre`.

EXEMPLE Transformation complète du MCD « Lycée » en 4 tables

Étape 1 (règle 1) — les trois entités deviennent trois tables :

- `ELEVE(matricule, nom, prenom, dateNaissance, sexe)`
- `CLASSE(idClasse, libelle, niveau)`
- `MATIERE(idMatiere, libelle, coefficient)`

Étape 2 (règle 2) — `INSCRIRE` est de type $(1,1) - (1,n)$: la clé `idClasse` migre dans `ELEVE`, avec l'attribut porté :

- `ELEVE(matricule, nom, prenom, dateNaissance, sexe, anneeScolaire, #idClasse)`

Étape 3 (règle 3) — `EVALUER` est de type (n,n) : elle devient la table `NOTE`, dont la clé primaire est la concaténation des deux clés :

- `NOTE(#matricule, #idMatiere, trimestre, note)`

Remarque : comme un élève peut avoir une note par trimestre dans la même matière, on ajoute `trimestre` à la clé primaire composée pour garantir l'unicité.

EXERCICE RÉSOLU 1 Appliquer les règles à un mini-MCD

Énoncé. Une bibliothèque scolaire est modélisée ainsi : LIVRE(isbn, titre, auteur); ADHERENT(idAdherent, nom); l'association EMPRUNTER relie ADHERENT (0,n) à LIVRE (0,n) et porte les attributs dateEmprunt et dateRetour. Donne le schéma relationnel complet.

SOLUTION

Solution détaillée.

1. Règle 1 : chaque entité devient une table.

LIVRE(isbn, titre, auteur)

ADHERENT(idAdherent, nom)

2. Règle 3 : EMPRUNTER est de type (n,n) des deux côtés, donc elle devient une table. Sa clé primaire est la concaténation des clés des entités reliées, et ses attributs propres deviennent des colonnes.

EMPRUNTER(#idAdherent, #isbn, dateEmprunt, dateRetour)

On ajoute dateEmprunt à la clé primaire composée car un même adhérent peut emprunter le même livre à plusieurs reprises : le couple (#idAdherent, #isbn) seul ne serait pas unique. C'est exactement le même raisonnement que pour le trimestre dans la table NOTE du lycée.

À RETENIR L'essentiel de la section

- Le **schéma relationnel** est la traduction du MCD en tables; il se note TABLE(clePrimaire, col2, #cleEtrangere).
- Règle 1 : entité → table, identifiant → clé primaire.
- Règle 2 : association (x,1) - (x,n) → clé étrangère migrée du côté « n » vers le côté « 1 » (avec ses attributs portés).
- Règle 3 : association (n,n) → table dont la clé primaire est la concaténation des clés reliées; les attributs portés deviennent des colonnes.
- Les associations réflexives suivent les mêmes règles, avec des clés étrangères renommées qui référencent la même table.
- La clé primaire composée doit garantir l'unicité : on y ajoute parfois un attribut discriminant (trimestre, date...).

EXERCICE 1 À toi de jouer

Le cybercafé « Connect+ » de Ngaoundéré veut gérer ses locations de postes. Le MCD

comprend : CLIENT(idClient, nom, telephone), POSTE(numPoste, marque, etat), et l'association UTILISER de type $(0,n) - (1,n)$ portant heureDebut et duree. Transforme ce MCD en schéma relationnel en justifiant chaque règle appliquée.

CORRIGÉ Exercice 1

Règle 1 : CLIENT(idClient, nom, telephone) et POSTE(numPoste, marque, etat). L'association UTILISER est de type $(0,n) - (1,n)$: les deux cardinalités maximales valent « n », donc la règle 3 s'applique (et non la règle 2 : il faut une patte à maximum 1). On obtient UTILISER(#idClient, #numPoste, heureDebut, duree), en ajoutant heureDebut à la clé primaire car un client peut utiliser le même poste plusieurs fois dans la journée.

Nous disposons désormais d'un plan de tables rigoureux. Dans la section suivante, nous étudierons le logiciel qui donnera vie à ce plan : le **SGBD**.

2 SGBD : RÔLE, CARACTÉRISTIQUES ET EXEMPLES

Dans la section précédente, nous avons appris à transformer un MCD en schéma relationnel : nous sommes donc capables, sur le papier, de décrire les tables de la bibliothèque du lycée ou de la boutique d'un commerçant de Douala. Mais un schéma relationnel n'est qu'un plan. Pour que ce plan devienne une base de données vivante, capable d'enregistrer des milliers d'emprunts, de répondre en une fraction de seconde à la question « quels élèves n'ont pas rendu leur livre ? » et de survivre à une coupure de courant, il faut un logiciel spécialisé : le **SGBD**. Cette section répond à trois questions : qu'est-ce qu'un SGBD, que fait-il exactement, et lequel choisir ?

2.1 Pourquoi un SGBD ? Le problème à résoudre

Imaginons que la bibliothèque du lycée gère ses emprunts dans des fichiers texte ou des classeurs Excel enregistrés sur l'ordinateur du surveillant. Très vite, les problèmes s'accumulent :

- **Redondance** : le nom et la classe d'un élève sont recopiés sur chaque fiche d'emprunt ; s'il change de classe, il faut corriger des dizaines de lignes, et on en oublie toujours une.
- **Incohérence** : on retrouve « 6^{ème} A », « 6e A » et « Sixième A » pour la même classe.
- **Conflits d'accès** : deux surveillants ne peuvent pas saisir en même temps dans le même fichier.
- **Perte de données** : une panne de courant pendant l'enregistrement corrompt le fichier, sans possibilité de récupération.
- **Absence de sécurité** : n'importe qui ouvrant le fichier peut modifier ou effacer les données.

C'est précisément pour résoudre ces cinq problèmes que les SGBD ont été inventés dans les années 1970.

DÉFINITION Base de données et SGBD

Une **base de données** (BD) est un ensemble structuré et organisé de données relatives à un même domaine, stockées de façon persistante et accessibles à plusieurs utilisateurs. Un **Système de Gestion de Base de Données** (SGBD) est un logiciel qui assure la description, la manipulation, l'interrogation et l'administration des données d'une ou plusieurs bases de données. Il fait l'intermédiaire entre les utilisateurs (ou les programmes) et les fichiers physiques où sont réellement stockées les données.

ATTENTION SGBD ≠ base de données

C'est la confusion la plus fréquente à l'examen. Le SGBD est le logiciel (MySQL, PostgreSQL, Oracle...); la base de données est le contenu (les tables *eleve*, *livre*, *emprunt* avec leurs enregistrements). On ne dit donc pas « j'ai installé une base de données MySQL » mais « j'ai installé le SGBD MySQL, puis j'y ai créé une base de données ». Un même SGBD peut héberger plusieurs bases de données indépendantes.

2.2 Les fonctions d'un SGBD et le langage SQL

Pour dialoguer avec le SGBD, on utilise un langage normalisé : **SQL** (Structured Query Language), standard international depuis 1986. Toutes les instructions que nous écrirons dans les sections suivantes seront en SQL. Ce langage se divise en trois familles, qui correspondent aux trois grandes fonctions du SGBD.

PROPRIÉTÉ Les trois familles du langage SQL (admise)

Toute instruction SQL appartient à l'une des trois familles suivantes :

1. le **LDD** (Langage de Définition de Données) : création et modification de la structure — instructions `CREATE`, `ALTER`, `DROP` ;
2. le **LMD** (Langage de Manipulation de Données) : gestion du contenu — instructions `INSERT`, `UPDATE`, `DELETE`, `SELECT` ;
3. le **LCD** (Langage de Contrôle des Données) : gestion des droits d'accès — instructions `GRANT` (accorder) et `REVOKE` (retirer).

Concrètement, le SGBD remplit les fonctions suivantes :

- **Définition des données (LDD)** : il permet de créer ou supprimer une base ou une table, d'ajouter une colonne, de changer un type, de déclarer des contraintes d'intégrité (clé primaire,

unicité...).

- **Manipulation et interrogation (LMD)** : il insère, modifie, supprime et surtout recherche les données. C'est le SGBD qui optimise la recherche grâce aux index : trouver un élève parmi 50 000 prend quelques millisecondes.
- **Contrôle des accès (LCD)** : il gère des comptes utilisateurs et des droits. Le surveillant peut consulter les emprunts mais pas les supprimer ; seul l'administrateur a tous les droits.

EXEMPLE Un aperçu des trois familles

```
-- LDD : definition de la structure
CREATE TABLE emprunt (
  id_emprunt INT PRIMARY KEY AUTO_INCREMENT,
  date_emprunt DATE NOT NULL
);

-- LMD : manipulation du contenu
INSERT INTO emprunt (date_emprunt) VALUES ('2025-01-15');
SELECT * FROM emprunt;

-- LCD : controle des droits
GRANT SELECT, INSERT ON bibliotheque.* TO 'surveillant'@'localhost';
REVOKE INSERT ON bibliotheque.* FROM 'surveillant'@'localhost';
```

2.3 Les caractéristiques d'un bon SGBD

Un SGBD digne de ce nom garantit six propriétés fondamentales. Retiens-les : elles reviennent souvent dans les questions de cours.

À RETENIR Les six caractéristiques d'un SGBD

1. **Indépendance données/programmes** : on peut modifier la structure des données (ajouter une colonne) sans réécrire les programmes qui les utilisent, et inversement.
2. **Non-redondance** : chaque information n'est stockée qu'une seule fois (grâce à la décomposition en tables reliées par des clés), ce qui économise l'espace et évite les contradictions.
3. **Cohérence (intégrité)** : le SGBD refuse toute opération qui violerait une contrainte — impossible d'enregistrer un emprunt pour un élève qui n'existe pas, ou deux élèves avec le même matricule.
4. **Sécurité et confidentialité** : accès protégé par comptes et mots de passe, droits différenciés par utilisateur.
5. **Accès concurrents** : plusieurs utilisateurs peuvent travailler simultanément ; le SGBD gère les verrous pour éviter que deux écritures simultanées ne se corrompent mutuel-

lement.

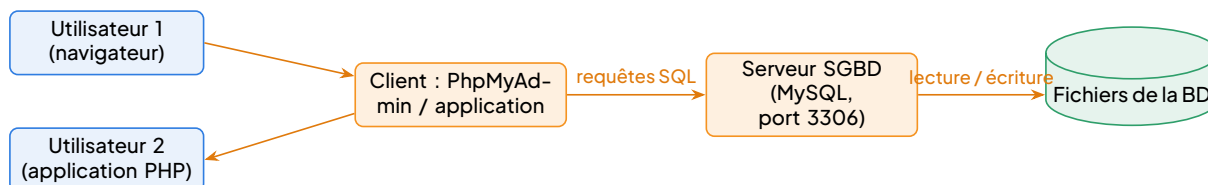
6. Reprise après panne : grâce au journal des transactions (log), le SGBD restaure un état cohérent après une coupure de courant ou un plantage.

LESAVAIS-TU ? MySQL, le SGBD du Web

MySQL, créé en 1995 par la société suédoise MySQL AB, est l'un des SGBD les plus utilisés au monde. Gratuit, rapide et fiable, il équipe la majorité des sites web hébergés en mutualisé — y compris la plupart des sites camerounais (journaux en ligne, plateformes de e-commerce, sites d'établissements scolaires). Après son rachat par Oracle, sa communauté a créé MariaDB, un dérivé 100 % libre et compatible, qui le remplace progressivement. Les grandes plateformes de Mobile Money et les applications bancaires s'appuient, elles, sur des SGBD propriétaires comme Oracle, capables de traiter des millions de transactions par jour.

2.4 Architecture client/serveur d'un SGBD

Un SGBD comme MySQL fonctionne selon le modèle **client/serveur**. Le **serveur** (le programme `mysqld`) est le seul à lire et écrire dans les fichiers physiques de la base. Les **clients** — un script PHP, l'interface PhpMyAdmin, la console `mysql` — envoient des requêtes SQL au serveur (par défaut sur le port 3306) et reçoivent les résultats. Cette séparation permet à des dizaines de clients, y compris distants, d'interroger la même base en même temps.



Architecture client/serveur : les clients envoient des requêtes SQL au serveur SGBD, seul habilité à manipuler les fichiers de la base.

2.5 Exemples de SGBD : libres ou propriétaires

SGBD	Licence	Plateforme	Usage typique
MySQL / MariaDB	Libre (GPL)	Windows, Linux, macOS	Sites web, applications PHP, hébergement mutualisé
PostgreSQL	Libre	Windows, Linux, macOS	Applications critiques, données géographiques, recherche
Oracle Database	Propriétaire (payante)	Linux, Windows	Banques, télécoms, grandes administrations
Microsoft SQL Server	Propriétaire (payante)	Windows (Linux)	Entreprises utilisant l'écosystème Microsoft
SQLite	Libre (domaine public)	Toutes, embarqué	Applications mobiles (Android), logiciels de bureau

ATTENTION Libre ne veut pas dire « moins bon »

Un SGBD libre (MySQL, PostgreSQL) est gratuit et son code source est ouvert ; un SGBD propriétaire (Oracle, SQL Server) exige l'achat d'une licence coûteuse mais fournit un support commercial. PostgreSQL rivalise avec Oracle en puissance : le choix dépend du budget, de la charge et de l'écosystème, pas d'une hiérarchie de qualité.

2.6 L'environnement de travail : WAMP/XAMPP et PhpMyAdmin

Pour pratiquer, nous utiliserons un paquetage qui installe en une fois tout le nécessaire sur ton poste : **WAMP** (Windows) ou **XAMPP** (multiplateforme). Ces logiciels regroupent : le serveur web **Apache**, le SGBD **MySQL/MariaDB**, le langage **PHP** et l'interface **PhpMyAdmin**. **PhpMyAdmin** est une application web écrite en PHP qui permet d'administrer MySQL depuis le navigateur (créer des bases et des tables, saisir des données, exécuter des requêtes SQL, exporter) sans taper de commandes système.

EXPÉRIENCE Découvrir PhpMyAdmin et exécuter sa première requête

Objectif : vérifier la distinction client/serveur et exécuter une requête SQL.

Matériel : un poste avec XAMPP (ou WAMP) installé.

Manipulation :

1. Lance le panneau de contrôle XAMPP et démarre les modules Apache et MySQL (ils doivent passer au vert).
2. Ouvre le navigateur à l'adresse `http://localhost/phpmyadmin` : PhpMyAdmin (le client) apparaît.
3. Clique sur l'onglet SQL et saisis la requête suivante, puis clique sur Exécuter :

```
SHOW DATABASES; -- liste les bases hébergées par le serveur
```

4. Dans le panneau de contrôle XAMPP, clique sur Stop devant MySQL, puis réexécute la requête dans PhpMyAdmin.

Observations : à l'étape 3, la liste des bases s'affiche ; à l'étape 4, PhpMyAdmin affiche une erreur de connexion.

Interprétation : PhpMyAdmin n'est qu'un client ; sans le serveur MySQL actif, aucune requête ne peut aboutir. Cela illustre concrètement l'architecture client/serveur.

EXERCICE RÉSOLU 2 Identifier SGBD, base de données et famille SQL

Énoncé. Le CENADI héberge sur un serveur MySQL trois applications : impôts, état civil et douanes. Pour chaque affirmation, dis si elle est correcte et justifie : (a) « Le serveur contient trois SGBD » ; (b) « CREATE TABLE relève du LMD » ; (c) « GRANT relève du LCD ».

SOLUTION

Solution.

1. (a) Faux : il y a un seul SGBD (MySQL, le logiciel serveur) qui héberge trois bases de données distinctes. On confond ici le logiciel et son contenu.
2. (b) Faux : CREATE TABLE définit la structure d'une table ; c'est une instruction du LDD (Langage de Définition de Données). Le LMD concerne le contenu (INSERT, UPDATE, DELETE, SELECT).
3. (c) Vrai : GRANT accorde des droits d'accès ; c'est une instruction du LCD (Langage de Contrôle des Données), comme REVOKE.

EXERCICE 2 À toi de jouer

1. Cite quatre caractéristiques d'un SGBD et explique en une phrase le problème que chacune résout.
2. Classe chaque instruction dans la bonne famille (LDD, LMD, LCD) : DROP TABLE, SELECT, REVOKE, ALTER TABLE, UPDATE, GRANT.

3. Dans l'architecture client/serveur de MySQL, quel composant lit physiquement les fichiers de la base ? Pourquoi les clients ne le font-ils pas directement ?

CORRIGÉ Exercice

1. Par exemple : non-redondance (évite les doubles saisies contradictoires), cohérence (bloque les données violant les contraintes), accès concurrents (permet le travail simultané sans corruption), reprise après panne (restaure un état cohérent grâce au journal). On pouvait aussi citer l'indépendance données/programmes et la sécurité.
2. LDD : DROP TABLE, ALTER TABLE ; LMD : SELECT, UPDATE ; LCD : REVOKE, GRANT.
3. C'est le serveur SGBD (mysqld) seul qui lit et écrit les fichiers. Si chaque client accédait directement aux fichiers, les écritures simultanées se corrompraient et la sécurité (droits, journalisation) serait inapplicable : le serveur centralise et contrôle tous les accès.

Nous savons désormais ce qu'est un SGBD, ce qu'il garantit et comment lui parler grâce à SQL. Il est temps de passer à la pratique : la section suivante utilisera le LDD pour créer concrètement, dans MySQL, les tables issues de notre schéma relationnel, avec leurs types et leurs contraintes d'intégrité.

3 DÉFINITION DES DONNÉES : TABLES, TYPES ET CONTRAINTES D'INTÉGRITÉ

Dans la section précédente, nous avons découvert ce qu'est un **SGBD** (MySQL, PostgreSQL, MariaDB...) et pourquoi il est indispensable pour stocker, protéger et interroger les données d'un système d'information. Mais un SGBD installé est comme un immense entrepôt vide : avant d'y ranger la moindre information, il faut construire les rayonnages. C'est précisément l'objet de cette section : apprendre à **définir** la structure d'une base de données — créer la base, créer les tables, choisir les bons types de données et poser des verrous de sécurité appelés **contraintes d'intégrité** — grâce au langage **SQL** et à l'outil **PhpMyAdmin**.

3.1 Éléments caractéristiques d'une table

Imagine le registre papier des élèves de ton lycée : chaque page est un tableau dont les colonnes sont toujours les mêmes (matricule, nom, date de naissance, classe...) et chaque ligne correspond à un élève. Une **table** dans une base de données relationnelle fonctionne exactement pareil.

DÉFINITION Table, champ, enregistrement

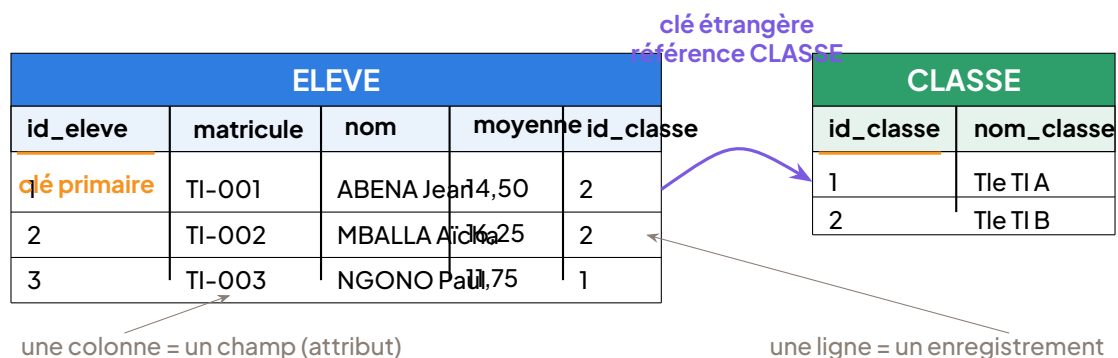
Une **table** (ou relation) est un ensemble de données organisé en deux dimensions :

- les **colonnes**, appelées aussi **champs** ou attributs, décrivent les propriétés des objets stockés (ex. nom, moyenne) ; chaque colonne possède un nom et un type de données ;
- les **lignes**, appelées aussi **enregistrements** ou tuples, représentent chacune une occurrence concrète (un élève précis, une classe précise).

Deux colonnes jouent un rôle particulier :

DÉFINITION Clé primaire et clé étrangère

- La **clé primaire** (PRIMARY KEY) est un champ (ou un groupe de champs) dont la valeur identifie de manière unique chaque enregistrement de la table. Deux lignes ne peuvent jamais avoir la même valeur de clé primaire, et cette valeur ne peut pas être vide (NOT NULL implicite).
- La **clé étrangère** (FOREIGN KEY) est un champ d'une table qui référence la clé primaire d'une autre table. C'est elle qui matérialise les liens entre tables : le champ `id_classe` de la table ELEVE pointe vers la clé primaire de la table CLASSE.



La table ELEVE annotée : champs, enregistrements, clé primaire (soulignée) et clé étrangère `id_classe` pointant vers la clé primaire de CLASSE.

ATTENTION Une clé primaire, toujours

Toute table bien conçue possède une clé primaire. Sans elle, impossible de distinguer deux élèves homonymes (deux « NGONO Paul »), de modifier ou de supprimer un enre-

gistrement précis sans risque. En pratique, on ajoute souvent un identifiant numérique auto-incrémenté (`id_eleve`) même quand un identifiant « naturel » existe déjà (matricule).

3.2 Les types de données courants en MySQL

Pourquoi déclarer un type pour chaque colonne ? Parce que le type détermine l'**espace de stockage**, les **opérations autorisées** (on peut additionner deux nombres, pas deux textes) et constitue un premier filtre contre les erreurs de saisie (une lettre dans un champ `INT` sera rejetée).

Type	Rôle	Exemple d'usage
<code>INT</code>	Nombre entier (environ -2 milliards à +2 milliards)	identifiant, effectif d'une classe
<code>DECIMAL(M,D)</code>	Nombre décimal exact, <i>M</i> chiffres au total dont <i>D</i> après la virgule	<code>DECIMAL(4,2)</code> pour une moyenne sur 20
<code>VARCHAR(n)</code>	Texte de longueur variable , au plus <i>n</i> caractères	nom, prénom, adresse
<code>CHAR(n)</code>	Texte de longueur fixe (complété par des espaces)	code pays « CM », sexe « M »/« F »
<code>DATE</code>	Date au format AAAA-MM-JJ	date de naissance
<code>TEXT</code>	Texte long (jusqu'à 65 535 caractères)	observation, appréciation du bulletin
<code>BOOLEAN</code>	Valeur vraie/fausse (stockée 1/0)	élève boursier ou non

MÉTHODE Choisir le bon type de données pour un attribut

Pour chaque attribut du schéma relationnel, pose-toi trois questions dans l'ordre :

1. Quelle est la nature de la valeur ? Nombre entier (`INT`), nombre avec virgule (`DECIMAL`), texte (`VARCHAR`), date (`DATE`), vrai/faux (`BOOLEAN`) ?
2. Quelle est la taille maximale réaliste ? Un nom camerounais dépasse rarement 50 caractères : `VARCHAR(50)` suffit, inutile de gaspiller avec `TEXT`.
3. Fera-t-on des calculs dessus ? Un matricule comme « TI-001 » contient des lettres : c'est un `VARCHAR`, jamais un entier, même s'il contient des chiffres — on ne calcule jamais la moyenne de deux matricules !

Exemples types : `matricule` → `VARCHAR(10)` ; `moyenne` → `DECIMAL(4,2)` (de 00,00 à 20,00) ; `date_naissance` → `DATE` ; `est_boursier` → `BOOLEAN`.

ATTENTION Les pièges des types

- Ne stocke jamais un numéro de téléphone en `INT` : le zéro initial de « 690123456 » disparaîtrait et les numéros internationaux contiennent « + ». Utilise `VARCHAR(20)`.
- Pour l'argent (montants Mobile Money, frais de scolarité), préfère `DECIMAL` à `FLOAT` : `FLOAT` arrondit et peut faire perdre des francs dans les calculs.
- Une date s'écrit '2007-03-15' entre apostrophes, au format année-mois-jour (ISO 8601).

3.3 Créer, sélectionner et supprimer une base de données

Avant de créer des tables, il faut un contenant : la **base de données**. Trois ordres SQL suffisent.

PROPRIÉTÉ Syntaxe : gestion d'une base de données

- Création : `CREATE DATABASE nom_base ;`
- Sélection (rendre la base « active » pour les commandes suivantes) : `USE nom_base ;`
- Suppression : `DROP DATABASE nom_base ;`

Chaque instruction SQL se termine par un point-virgule ;. SQL n'est pas sensible à la casse pour les mots-clés, mais la convention est de les écrire en MAJUSCULES.

```
-- Création de la base du lycée (jeu de caractères accentués acceptés)
CREATE DATABASE lycee CHARACTER SET utf8mb4;

-- On se place dans la base : toutes les tables créées ensuite iront dedans
USE lycee;

-- Suppression complète de la base (DANGER : tout est perdu)
-- DROP DATABASE lycee;
```

3.4 Créer une table : `CREATE TABLE` et contraintes d'intégrité

Voici le cœur de la section. Une base de données scolaire qui accepterait une moyenne de 47/20, un élève sans nom ou deux fois le même matricule serait **pire qu'inutile : dangereuse**. Les **contraintes d'intégrité** sont les gardiennes qui empêchent ces aberrations.

DÉFINITION Contrainte d'intégrité

Une **contrainte d'intégrité** est une règle déclarée au SGBD, qui rejette automatiquement toute insertion ou modification violant cette règle. Elle garantit la cohérence et la fiabi-

lité des données, quelle que soit l'application qui écrit dans la base. Les six contraintes au programme sont :

1. Clé primaire (PRIMARY KEY) : identifiant unique et non nul de chaque ligne ;
2. Valeur nulle (NOT NULL / NULL) : le champ doit-il obligatoirement être rempli ? Par défaut, un champ peut être vide (NULL) ;
3. Valeur par défaut (DEFAULT) : valeur automatiquement attribuée si l'utilisateur ne précise rien ;
4. Unicité (UNIQUE) : deux lignes ne peuvent pas avoir la même valeur pour ce champ (ex. matricule) ;
5. Valeur autorisée (CHECK) : condition que la valeur doit vérifier (ex. moyenne entre 0 et 20) ;
6. Format / domaine : imposé par le type de données lui-même (un DATE refuse « 32 janvier », un INT refuse « abc »).

On ajoute à cette liste la clé étrangère (FOREIGN KEY ... REFERENCES), qui garantit qu'une valeur référencée existe bien dans l'autre table (intégrité référentielle).

PROPRIÉTÉ Syntaxe générale de CREATE TABLE

```
CREATE TABLE nom_table (  
    colonne1 TYPE [contraintes de colonne],  
    colonne2 TYPE [contraintes de colonne],  
    ...,  
    PRIMARY KEY (colonne),  
    FOREIGN KEY (colonne) REFERENCES autre_table(colonne)  
);
```

Les contraintes peuvent être écrites au niveau de la colonne (NOT NULL, DEFAULT, UNIQUE) ou en fin de liste (PRIMARY KEY, FOREIGN KEY, CHECK).

EXEMPLE Script complet : création des tables CLASSE et ELEVE

```
-- La table référencée (CLASSE) doit être créée AVANT la table ELEVE  
CREATE TABLE CLASSE (  
    id_classe INT AUTO_INCREMENT,  
    nom_classe VARCHAR(20) NOT NULL UNIQUE, -- ex. 'Tle TIA'  
    effectif INT DEFAULT 0, -- 0 élève au départ  
    PRIMARY KEY (id_classe)  
);  
  
CREATE TABLE ELEVE (  

```

```
id_eleve      INT AUTO_INCREMENT,  
matricule     VARCHAR(10) NOT NULL UNIQUE, -- unicité du matricule  
nom           VARCHAR(50) NOT NULL,      -- nom obligatoire  
date_naissance DATE,                    -- peut être inconnue  
moyenne       DECIMAL(4,2) DEFAULT 0.00,  
est_boursier  BOOLEAN DEFAULT FALSE,  
id_classe     INT NOT NULL,              -- clé étrangère  
PRIMARY KEY (id_eleve),  
CHECK (moyenne BETWEEN 0 AND 20),       -- valeur autorisée  
FOREIGN KEY (id_classe) REFERENCES CLASSE(id_classe)  
);
```

Grâce à `AUTO_INCREMENT`, MySQL numérote lui-même les lignes (1, 2, 3...) : on n'a jamais à choisir l'identifiant à la main.

ATTENTION Erreurs fréquentes à la création

- Oublier la clé primaire : MySQL accepte la table, mais tu perds toute garantie d'unicité et les jointures futures seront impossibles à fiabiliser. Vérifie toujours la présence du `PRIMARY KEY`.
- Déclarer une clé étrangère vers une table inexistante : l'ordre de création compte ! `CLASSE` doit être créée avant `ELEVE`, sinon MySQL renvoie l'erreur `Cannot add foreign key constraint`.
- Types incompatibles : la clé étrangère `ELEVE.id_classe` doit avoir le même type que la clé primaire référencée `CLASSE.id_classe` (ici `INT` des deux côtés).
- Oublier le point-virgule final ou les virgules entre colonnes : erreur de syntaxe immédiate.

3.5 Modifier la structure d'une table : `ALTER TABLE`

En cours d'année, le proviseur décide d'ajouter le numéro de téléphone des parents dans le fichier des élèves. Faut-il tout recréer ? Non : l'ordre **`ALTER TABLE`** modifie la structure d'une table **sans perdre les données** qu'elle contient.

PROPRIÉTÉ Syntaxe : `ALTER TABLE`

- Ajouter une colonne : `ALTER TABLE t ADD col TYPE [contraintes] ;`
- Supprimer une colonne : `ALTER TABLE t DROP COLUMN col ;`
- Renommer une colonne : `ALTER TABLE t RENAME COLUMN ancien TO nouveau ;`
- Modifier le type d'une colonne : `ALTER TABLE t MODIFY col NOUVEAU_TYPE ;`

```
-- Ajout du téléphone du parent d'élève
ALTER TABLE ELEVE ADD tel_parent VARCHAR(20);

-- Le nom peut être plus long que prévu : on élargit à 80 caractères
ALTER TABLE ELEVE MODIFY nom VARCHAR(80) NOT NULL;

-- Renommage d'une colonne
ALTER TABLE ELEVE RENAME COLUMN tel_parent TO contact_parent;

-- Suppression d'une colonne devenue inutile
ALTER TABLE ELEVE DROP COLUMN est_boursier;
```

ATTENTION MODIFY et DROP COLUMN : prudence

Réduire la taille d'un type (VARCHAR(80) → VARCHAR(10)) peut tronquer les données existantes; convertir un texte en INT peut échouer ou produire des zéros. Quant à DROP COLUMN, les données de la colonne sont définitivement perdues. Avant toute modification de structure sur une base en production, fais une sauvegarde (nous verrons comment dans la section 6).

3.6 Supprimer et renommer une table

PROPRIÉTÉ DROP TABLE et RENAME TABLE

- Renommer : RENAME TABLE ancien_nom TO nouveau_nom ;
- Supprimer : DROP TABLE nom_table ; — la structure et toutes les données sont détruites.

ATTENTION DROP TABLE et DROP DATABASE sont irréversibles

Contrairement à une corbeille Windows, MySQL ne propose aucune annulation : un DROP TABLE ELEVE; exécuté par inadvertance un vendredi soir efface instantanément le fichier de 1500 élèves. De plus, on ne peut pas supprimer une table référencée par une clé étrangère (il faut d'abord supprimer ELEVE avant CLASSE). Règle d'or de l'administrateur : toujours exporter une sauvegarde de la base avant toute opération destructive. Dans PhpMyAdmin, c'est l'onglet « Exporter » — deux clics qui peuvent sauver ton année scolaire.

LESAVAIS-TU ? Le modèle relationnel, une invention française ?

C'est Edgar Frank Codd, un chercheur britannique travaillant pour IBM à San José, qui pose

en 1970 les bases mathématiques du modèle relationnel (algèbre relationnelle, clés, normalisation). Mais c'est une équipe française de l'INRIA (Institut National de Recherche en Informatique et en Automatique), autour de Michel Scholl et Françoise Fabret, qui développe dans les années 80 le SGBD relationnel MISTRAL, puis contribue activement à la standardisation SQL (norme SQL-92). Aujourd'hui encore, la recherche française (équipes de l'INRIA, LIP6, IRISA) est à la pointe sur les bases de données distribuées, le NoSQL et la gestion des données massives (Big Data) — des compétences clés pour les futurs ingénieurs camerounais formés en TI.

3.7 Réalisation guidée : créer la base « lycee » dans PhpMyAdmin

EXPÉRIENCE TP : création de la base « lycee » et de ses tables

Objectif : déployer la base `lycee` avec les tables `CLASSE` et `ELEVE` sous WampServer.

Matériel : un poste avec WampServer (ou XAMPP) démarré (icône verte), un navigateur.

1. Ouvre le navigateur et saisis l'adresse `http://localhost/phpmyadmin`. Connecte-toi avec l'utilisateur `root` (mot de passe vide par défaut sous WampServer).
2. Clique sur l'onglet SQL dans le menu supérieur : c'est la console où l'on tape directement les requêtes.
3. Saisis `CREATE DATABASE lycee CHARACTER SET utf8mb4`; puis clique sur Exécuter. La base apparaît dans la colonne de gauche.
4. Clique sur la base `lycee` à gauche pour la sélectionner (équivalent de `USE lycee`;
5. Rouvre l'onglet SQL et colle le script complet de création des tables `CLASSE` puis `ELEVE` (voir l'exemple ci-dessus), puis Exécuter.
6. Observations : dans la colonne de gauche, les deux tables apparaissent. Clique sur `ELEVE` puis sur l'onglet Structure : tu vois chaque colonne, son type, et les contraintes (clé primaire signalée par une clé jaune, index `UNIQUE` sur `matricule`).
7. Vérification des contraintes : dans l'onglet Insérer, tente d'enregistrer un élève avec une moyenne de 25, puis un élève avec `id_classe = 99` (classe inexistante).
8. Interprétation : MySQL rejette les deux insertions : la contrainte `CHECK` bloque la moyenne invalide, et la clé étrangère bloque la référence vers une classe inexistante. Les contraintes d'intégrité fonctionnent !

EXERCICE RÉSOLU 3 Modifier la structure d'une table existante

Énoncé : le lycée bilingue de Nkolbisson souhaite désormais enregistrer le genre des

élèves (« M » ou « F », obligatoire) et porter la taille du champ `nom_classe` de 20 à 30 caractères. Écris les requêtes SQL correspondantes.

SOLUTION

Solution détaillée : on utilise deux ordres `ALTER TABLE` distincts (un par modification, pour plus de clarté) :

-- 1. Ajout de la colonne `genre` : `CHAR(1)`, obligatoire, valeur contrôlée

```
ALTER TABLE ELEVE ADD genre CHAR(1) NOT NULL DEFAULT 'M';
```

```
ALTER TABLE ELEVE ADD CHECK (genre IN ('M', 'F'));
```

-- 2. Élargissement du nom de classe (aucune perte : on agrandit)

```
ALTER TABLE CLASSE MODIFY nom_classe VARCHAR(30) NOT NULL UNIQUE;
```

Remarque : le `DEFAULT 'M'` est indispensable à l'ajout, car les lignes déjà présentes recevraient sinon une valeur vide, ce qui violerait le `NOT NULL`. Agrandir un `VARCHAR` est sans risque ; le réduire pourrait tronquer des données.

À RETENIR L'essentiel de la section

- Une table = des colonnes (champs typés) et des lignes (enregistrements) ; la clé primaire identifie chaque ligne, la clé étrangère relie deux tables.
- Le type de données (`INT`, `DECIMAL`, `VARCHAR(n)`, `CHAR(n)`, `DATE`, `TEXT`, `BOOLEAN`) se choisit selon la nature, la taille et l'usage de l'attribut.
- `CREATE DATABASE / USE / DROP DATABASE` gèrent la base ; `CREATE TABLE` définit les tables avec leurs contraintes.
- Les contraintes d'intégrité (`PRIMARY KEY`, `NOT NULL`, `DEFAULT`, `UNIQUE`, `CHECK`, domaine du type, clé étrangère) protègent automatiquement la cohérence des données.
- `ALTER TABLE (ADD, DROP COLUMN, RENAME COLUMN, MODIFY)` fait évoluer la structure ; `DROP TABLE` et `RENAME TABLE` suppriment ou renomment une table.
- Toute suppression est irréversible : sauvegarde toujours avant.

EXERCICE 3 À toi de jouer

La startup doualaise « EduPay » gère les paiements de scolarité par Mobile Money. Écris le script SQL créant la base `edupay` puis la table `PAIEMENT` avec les champs : `id_paiement` (clé primaire auto-incrémentée), `montant` (décimal positif obligatoire), `date_paiement` (date du jour par défaut n'étant pas exigée, laisser sans défaut), `opérateur` (« MTN » ou « OM » uniquement), et `id_eleve` (clé étrangère vers `ELEVE`). Puis écris la requête qui ajoute en-

suite une colonne `reference_transaction` unique de 25 caractères.

CORRIGÉ Exercice : EduPay

```
CREATE DATABASE edupay CHARACTER SET utf8mb4;
USE edupay;

-- On suppose que la table ELEVE existe déjà dans cette base
CREATE TABLE PAIEMENT (
  id_paiement INT AUTO_INCREMENT,
  montant DECIMAL(10,2) NOT NULL,
  date_paiement DATE NOT NULL,
  operateur VARCHAR(4) NOT NULL,
  id_eleve INT NOT NULL,
  PRIMARY KEY (id_paiement),
  CHECK (montant > 0),
  CHECK (operateur IN ('MTN', 'OM')),
  FOREIGN KEY (id_eleve) REFERENCES ELEVE(id_eleve)
);

ALTER TABLE PAIEMENT
ADD reference_transaction VARCHAR(25) UNIQUE;
```

Points de vigilance : la table `ELEVE` doit exister avant la création de `PAIEMENT`; `DECIMAL(10,2)` permet des montants jusqu'à 99 999 999,99 FCFA; le `CHECK` sur l'opérateur empêche toute faute de frappe (« mtn », « Orange »).

La structure de notre base est désormais solide et protégée. Mais une base vide ne sert à rien : dans la prochaine section, nous apprendrons à la manipuler — insérer, modifier, supprimer et surtout interroger les données avec `INSERT`, `UPDATE`, `DELETE` et le puissant `SELECT`.

4 MANIPULATION DES DONNÉES : INSERT, UPDATE, DELETE, SELECT

Dans la section précédente, nous avons appris à définir la structure d'une base de données : créer les tables, choisir les types de colonnes et poser les contraintes d'intégrité. Mais une table vide ne sert à rien. Le lycée bilingue de Nkolndongo qui vient de créer sa table `ELEVE` veut maintenant y inscrire les élèves, corriger une erreur de saisie, radier un élève parti, et surtout interroger la base : « quels sont les élèves de Yaoundé ? », « qui a payé moins de 50 000 FCFA de scolarité ? ». C'est exactement le rôle du **LMD** (Langage de Manipulation de Données), qui regroupe quatre ordres SQL fondamentaux : `INSERT`, `UPDATE`, `DELETE` et `SELECT`.

DÉFINITION Requête

Une **requête** est un ordre écrit en SQL, envoyé au SGBD, qui demande d'effectuer une opération sur les données : les insérer, les modifier, les supprimer ou les extraire. Le SGBD analyse la requête, l'exécute et renvoie un résultat (un message de confirmation ou une table de résultats).

4.1 Notre table de travail : ÉLÈVE

Toute cette section s'appuie sur la table ELEVE ci-dessous, créée dans la section précédente et remplie de huit enregistrements. Apprends à la lire : chaque **ligne** est un **enregistrement**, chaque **colonne** est un **champ**.

id	nom	prénom	âge	ville	scolarité
1	Ngo Bell	Amina	17	Yaoundé	75 000
2	Etoundi	Jean	18	Douala	50 000
3	Nkoulou	Marie	16	Yaoundé	75 000
4	Mbarga	Paul	19	Bafoussam	40 000
5	Ndi	Chantal	17	Douala	50 000
6	Fotso	Eric	18	Bafoussam	40 000
7	Ngassa	Larissa	16	Yaoundé	NULL
8	Abena	Steve	20	Douala	50 000

Table ELEVE : 8 enregistrements servant de support à tous les exemples de la section.

4.2 Insertion d'enregistrements : INSERT INTO

Intuition. La table est une armoire à tiroirs vide : INSERT est le geste qui consiste à y ranger une fiche. Sans insertion, aucune donnée n'existe.

PROPRIÉTÉ Syntaxe de INSERT

```
-- Insertion d'une seule ligne (colonnes nommées : recommandé)
INSERT INTO nom_table (col1, col2, col3)
VALUES (val1, val2, val3);
```

```
-- Insertion de plusieurs lignes en une seule requête
INSERT INTO nom_table (col1, col2, col3)
VALUES (val1, val2, val3),
      (val4, val5, val6);
```

Les valeurs textuelles et les dates s'écrivent entre apostrophes ; les nombres sans apos-

trophes. L'ordre des valeurs doit correspondre à l'ordre des colonnes déclarées.

EXEMPLE Remplir la table ELEVE

```
-- Deux inscriptions au lycée, en une seule requête
INSERT INTO ELEVE (id, nom, prénom, âge, ville, scolarité)
VALUES (1, 'Ngo Bell', 'Amina', 17, 'Yaoundé', 75000),
       (2, 'Etoundi', 'Jean', 18, 'Douala', 50000);
```

Si la colonne `id` est en `AUTO_INCREMENT`, on l'omet simplement de la liste des colonnes : le SGBD la numérote tout seul.

ATTENTION Pièges fréquents à l'insertion

- Oublier les apostrophes autour d'une chaîne : `VALUES (Ngo Bell)` provoque une erreur de syntaxe.
- Insérer une valeur qui viole une contrainte (doublon de clé primaire, `NOT NULL` non renseigné, clé étrangère inexistante) : le SGBD refuse la ligne. C'est la preuve que les contraintes de la section précédente protègent réellement les données.

4.3 Modification : UPDATE ... SET ... WHERE

Intuition. Amina déménage de Yaoundé à Douala : il ne faut ni supprimer sa fiche ni tout réécrire, juste corriger un champ. C'est le rôle de `UPDATE`.

PROPRIÉTÉ Syntaxe de UPDATE

```
UPDATE nom_table
SET col1 = nouvelle_valeur1, col2 = nouvelle_valeur2
WHERE condition;
```

La clause `WHERE` désigne les lignes à modifier. Toutes les lignes qui satisfont la condition sont mises à jour ; les autres restent intactes.

```
-- Amina (id = 1) déménage à Douala
UPDATE ELEVE
SET ville = 'Douala'
WHERE id = 1;
```

```
-- Le lycée augmente la scolarité de 5 000 FCFA pour tous les élèves de Douala
UPDATE ELEVE
```

```
SET scolarité = scolarité + 5000  
WHERE ville = 'Douala';
```

ATTENTION Erreur fréquente : UPDATE ou DELETE sans WHERE

Si tu oublies la clause `WHERE`, la modification ou la suppression s'applique à toutes les lignes de la table, sans demande de confirmation! `UPDATE ELEVE SET ville = 'Douala';` déclare subitement que les 8 élèves habitent Douala. En entreprise, cette faute de frappe a déjà détruit des bases entières. Réflexe de survie : écris toujours le `WHERE` avant le `UPDATE`, et teste d'abord ta condition avec un `SELECT` portant le même `WHERE` pour vérifier quelles lignes seront touchées.

4.4 Suppression : DELETE FROM ... WHERE

Intuition. Steve (id = 8) quitte définitivement le lycée : on retire sa fiche de l'armoire. `DELETE` supprime des lignes, pas la table.

PROPRIÉTÉ Syntaxe de DELETE

```
DELETE FROM nom_table  
WHERE condition;
```

Seules les lignes vérifiant la condition sont supprimées. La structure de la table (colonnes, contraintes) est conservée.

```
-- Radiation de Steve  
DELETE FROM ELEVE  
WHERE id = 8;
```

À RETENIR DELETE ou DROP TABLE ?

`DELETE FROM ELEVE;` vide la table de tous ses enregistrements mais la table (vide) existe toujours. `DROP TABLE ELEVE;` détruit la table elle-même : structure et données disparaissent. Pour repartir à zéro on utilise `DELETE`; pour supprimer définitivement l'objet, `DROP`.

4.5 Interrogation : SELECT, projection et restriction

Intuition. C'est la requête la plus utilisée (plus de 80 % des requêtes en pratique). Interroger une table, c'est répondre à deux questions : quelles colonnes veux-tu voir ? quelles lignes veux-tu garder ?

DÉFINITION Projection et restriction

La **projection** est le choix des colonnes affichées dans le résultat (liste après SELECT). La **restriction** (ou sélection) est le choix des lignes conservées, exprimé par la condition du WHERE. Le résultat d'un SELECT est toujours une nouvelle table temporaire, sans modifier la table d'origine.

PROPRIÉTÉ Syntaxe générale de SELECT

```
SELECT col1, col2      -- projection : les colonnes (ou * pour toutes)
FROM nom_table
WHERE condition;      -- restriction : les lignes (facultatif)
```

-- Toutes les colonnes, toutes les lignes

```
SELECT * FROM ELEVE;
```

-- Projection : seulement nom et ville, pour tous les élèves

```
SELECT nom, ville FROM ELEVE;
```

-- Projection + restriction : nom et âge des élèves de Yaoundé

```
SELECT nom, âge
FROM ELEVE
WHERE ville = 'Yaoundé';
```

4.6 Construire la condition du WHERE

Opérateurs de comparaison et opérateurs logiques

Les opérateurs de comparaison sont : = (égal), <> (différent, parfois !=), <, <=, >, >=. On les combine avec les opérateurs logiques AND (les deux conditions vraies), OR (au moins une vraie), NOT (négation).

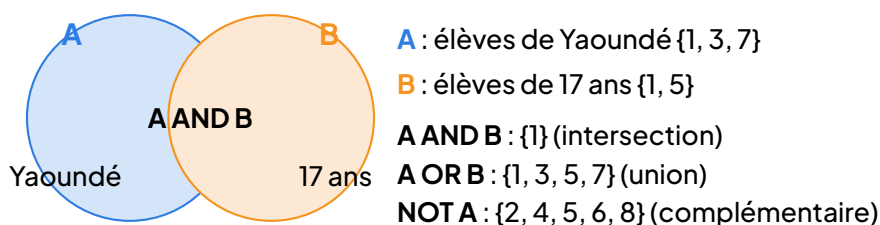


Diagramme de Venn : les opérateurs logiques AND, OR, NOT appliqués aux ensembles d'élèves.

MÉTHODE Construire une clause WHERE correcte

1. Formule la question en français : « les élèves de Douala qui ont moins de 18 ans ».
2. Découpe en conditions élémentaires : ville = 'Douala' et âge < 18.
3. Choisis le connecteur : « et » → AND ; « ou » → OR.
4. Respecte les priorités : NOT est évalué d'abord, puis AND, puis OR. Dès qu'il y a un mélange de AND et de OR, mets des parenthèses pour imposer ton intention.
5. Teste mentalement sur deux ou trois lignes de la table avant d'exécuter.

ATTENTION Priorité de AND sur OR

WHERE ville = 'Douala' OR ville = 'Yaoundé' AND âge < 18 ne signifie pas « (Douala ou Yaoundé) et moins de 18 ans » : le AND étant prioritaire, cela se lit « Douala, ou bien (Yaoundé et moins de 18 ans) ». Écris toujours : WHERE (ville = 'Douala' OR ville = 'Yaoundé') AND âge < 18.

Recherche de motifs : LIKE, intervalles : BETWEEN, listes : IN

PROPRIÉTÉ Opérateurs spécialisés du WHERE

- LIKE 'motif' : recherche sur du texte. Le joker % remplace zéro ou plusieurs caractères ; le joker _ remplace exactement un caractère.
- BETWEEN a AND b : valeur comprise entre a et b, bornes incluses (équivalent à >= a AND <= b).
- IN (v1, v2, ...) : appartenance à une liste de valeurs ; NOT IN : n'appartient pas à la liste.

```
-- Noms commençant par 'N' : Ngo Bell, Nkoulou, Ndi, Ngassa
SELECT nom FROM ELEVE WHERE nom LIKE 'N%';
```

```
-- Noms dont la 2e lettre est 'g' (un caractère quelconque puis 'g')
SELECT nom FROM ELEVE WHERE nom LIKE '_g%';
```

```
-- Scolarité entre 40 000 et 60 000 FCFA (bornes incluses)
SELECT nom, scolarité FROM ELEVE
WHERE scolarité BETWEEN 40000 AND 60000;
```

```
-- Élèves de Douala ou Bafoussam (équivalent à deux OR)
SELECT nom, ville FROM ELEVE
WHERE ville IN ('Douala', 'Bafoussam');
```

```
-- Élèves qui ne sont NI de Douala NI de Yaoundé
SELECT nom FROM ELEVE
WHERE ville NOT IN ('Douala', 'Yaoundé');
```

ATTENTION Position du joker % dans LIKE

LIKE 'N%' trouve les noms commençant par N ; LIKE '%N' ceux finissant par N ; LIKE '%N%' ceux contenant un N n'importe où. Inverser la position du joker change complètement le résultat. Autre piège : = ne tolère aucun joker ; pour un motif, c'est obligatoirement LIKE.

Valeurs nulles : IS NULL / IS NOT NULL

Larissa Ngassa n'a pas encore de scolarité enregistrée : son champ vaut NULL (absence de valeur, ce n'est ni 0 ni une chaîne vide). Or NULL n'est comparable à rien : WHERE scolarité = NULL ne renvoie jamais aucune ligne, même celles qui sont nulles !

PROPRIÉTÉ Tester la nullité

On teste une valeur nulle exclusivement avec IS NULL ou IS NOT NULL :

```
-- Élèves dont la scolarité n'est pas encore renseignée
SELECT nom FROM ELEVE WHERE scolarité IS NULL ;
```

```
-- Élèves dont la scolarité est connue
SELECT nom FROM ELEVE WHERE scolarité IS NOT NULL ;
```

4.7 Alias et DISTINCT : présenter le résultat

PROPRIÉTÉ Alias avec AS et suppression des doublons avec DISTINCT

Le mot-clé AS renomme une colonne (ou la table) dans le résultat, sans toucher à la base. Le mot-clé DISTINCT, placé juste après SELECT, élimine les lignes en double du résultat.

```
-- Alias : la colonne s'affiche sous le nom "Nom de famille"
```

```
SELECT nom AS 'Nom de famille', âge AS 'Âge (ans)'
FROM ELEVE ;
```

```
-- Villes représentées au lycée, sans doublons : Yaoundé, Douala, Bafoussam
```

```
SELECT DISTINCT ville FROM ELEVE ;
```

Sans DISTINCT, la dernière requête afficherait Yaoundé trois fois et Douala trois fois.

EXEMPLE Six requêtes progressives sur la table ÉLÈVE

-- R1 : tous les élèves (aucune restriction)
`SELECT * FROM ELEVE;`

-- R2 : les élèves de 17 ans exactement -> Ngo Bell, Ndi
`SELECT nom, prénom FROM ELEVE WHERE âge = 17;`

-- R3 : les élèves de Douala de 18 ans ou plus -> Etoundi, Abena
`SELECT nom FROM ELEVE WHERE ville = 'Douala' AND âge >= 18;`

-- R4 : les noms commençant par 'N' -> Ngo Bell, Nkoulou, Ndi, Ngassa
`SELECT nom FROM ELEVE WHERE nom LIKE 'N%';`

-- R5 : scolarité entre 45 000 et 75 000 FCFA -> ids 1, 2, 3, 5, 8
`SELECT nom, scolarité FROM ELEVE
WHERE scolarité BETWEEN 45000 AND 75000;`

-- R6 : élèves de Yaoundé ou Bafoussam sans scolarité renseignée,
-- ou mineurs de Yaoundé (parenthèses obligatoires !)
`SELECT nom, ville, âge FROM ELEVE
WHERE ville IN ('Yaoundé', 'Bafoussam')
AND (scolarité IS NULL OR âge < 17);`

EXERCICE RÉSOLU 4 De la demande du proviseur à la requête

Énoncé. Le proviseur demande : « la liste des noms et villes des élèves majeurs (18 ans ou plus) qui n'habitent ni Yaoundé ni Douala ». Écris la requête et donne son résultat sur notre table.

SOLUTION

Étape 1 — découpage. Conditions élémentaires : âge >= 18; ville différente de Yaoundé; ville différente de Douala. Connecteur : « et » partout, donc AND. La liste négative se condense avec NOT IN.

Étape 2 — requête.

```
SELECT nom, ville
FROM ELEVE
WHERE âge >= 18
AND ville NOT IN ('Yaoundé', 'Douala');
```

Étape 3 — vérification ligne par ligne. Mbarga (19 ans, Bafoussam) : retenu. Fotso (18 ans, Bafoussam) : retenu. Etoundi et Abena sont majeurs mais de Douala : exclus. Résultat : Mbarga — Bafoussam et Fotso — Bafoussam.

EXERCICE 4 À toi de jouer

Sur la table `ELEVE`, écris les requêtes suivantes :

1. Nom et âge des élèves de moins de 18 ans habitant Yaoundé.
2. Les élèves dont le prénom se termine par « e ».
3. Les villes des élèves dont la scolarité est strictement inférieure à 60 000 FCFA, sans doublons.
4. Corrige l'erreur de saisie : la scolarité de Larissa Ngassa (id = 7) est en réalité de 75 000 FCFA.

CORRIGÉ Exercice 1

```
-- 1. Mineurs de Yaoundé -> Nkoulou, Ngassa
SELECT nom, âge FROM ELEVE
WHERE âge < 18 AND ville = 'Yaoundé';

-- 2. Prénoms finissant par 'e' -> Marie, Chantal
SELECT nom, prénom FROM ELEVE
WHERE prénom LIKE '%e';

-- 3. Villes concernées, sans doublons -> Douala, Bafoussam
SELECT DISTINCT ville FROM ELEVE
WHERE scolarité < 60000;

-- 4. Correction ciblée : le WHERE sur la clé primaire garantit
-- qu'une seule ligne est modifiée
UPDATE ELEVE
SET scolarité = 75000
WHERE id = 7;
```

Remarque pour la question 3 : la condition `scolarité < 60000` exclut automatiquement Larissa, car toute comparaison avec `NULL` est fausse — d'où l'importance de la question 4 avant toute statistique.

À RETENIR L'essentiel de la section

- `INSERT INTO ... VALUES` ajoute des lignes ; `UPDATE ... SET` les modifie ; `DELETE FROM` les supprime ; `SELECT` les interroge sans jamais modifier la base.
- Sans `WHERE`, `UPDATE` et `DELETE` frappent toute la table : danger maximal.
- `SELECT` = projection (colonnes) + restriction (lignes via `WHERE`).
- Conditions : comparaisons (`=`, `<>`, `<...`), logique (`AND` prioritaire sur `OR`, parenthèses!),

LIKE avec % et _, BETWEEN (bornes incluses), IN/NOT IN, et IS NULL pour les valeurs absentes.

- AS renomme l’affichage ; DISTINCT supprime les doublons du résultat.

Maintenant que tu sais extraire des lignes précises d’une table, il reste à apprendre à en tirer des synthèses — combien d’élèves par ville ? quelle scolarité moyenne ? — et à croiser plusieurs tables entre elles : ce sera l’objet de la section suivante, consacrée à l’agrégation, au tri, au regroupement et aux jointures.

5 AGRÉGATION, TRI, REGROUPEMENT ET JOINTURES

Dans la section précédente, nous avons appris à insérer, modifier et supprimer des lignes, ainsi qu’à sélectionner des données brutes avec `SELECT ... FROM ... WHERE`. Mais une base de données ne sert pas seulement à stocker des données : elle doit permettre de **répondre à des questions décisionnelles**. Combien d’élèves par classe ? Quelle est la moyenne de la classe de Terminale A ? Quel est le montant total des factures impayées pour un client donné ? Pour répondre à ces questions, nous devons **agréger, trier, regrouper** et **croiser** les données de plusieurs tables. C’est l’objet de cette section.

5.1 Fonctions d’agrégation : résumer des ensembles de lignes

Une **fonction d’agrégation** (ou fonction de groupe) calcule une valeur unique à partir d’un ensemble de lignes (un groupe). Elle ignore généralement les valeurs NULL (sauf `COUNT(*)`).

DÉFINITION Fonctions d’agrégation standards (SQL :1999)

Les cinq fonctions d’agrégation fondamentales sont :

- `COUNT(expr)` : nombre de lignes (ou valeurs non nulles de `expr`). `COUNT(*)` compte toutes les lignes, NULL inclus.
- `SUM(expr)` : somme des valeurs (types numériques).
- `AVG(expr)` : moyenne arithmétique (types numériques).
- `MIN(expr) / MAX(expr)` : valeur minimale / maximale (numérique, date, chaîne).

Comportement avec NULL : `SUM`, `AVG`, `MIN`, `MAX` ignorent les NULL. `COUNT(colonne)` ignore les NULL ; `COUNT(*)` les compte.

EXEMPLE Agrégation sur la table NOTE (contexte : bulletin scolaire)

Soit la table `NOTE(id_eleve, id_matiere, note_sur_20)`.

```
-- 1. Nombre total de notes enregistrées
SELECT COUNT(*) AS total_notes FROM NOTE;

-- 2. Nombre de notes saisies pour les mathématiques (id_matiere = 3)
-- COUNT(note_sur_20) ignore les notes NULL (absences non justifiées)
SELECT COUNT(note_sur_20) AS notes_saisies_maths
FROM NOTE WHERE id_matiere = 3;

-- 3. Moyenne, min, max, somme des notes de l'élève n° 105
SELECT
    AVG(note_sur_20) AS moyenne,
    MIN(note_sur_20) AS note_min,
    MAX(note_sur_20) AS note_max,
    SUM(note_sur_20) AS somme_notes
FROM NOTE WHERE id_eleve = 105;
```

Résultat typique : moyenne=12,50, note_min=5, note_max=19, somme_notes=250.

ATTENTION Piège : COUNT(*) vs COUNT(colonne)

Si la colonne `note_sur_20` contient des NULL (élève absent), `COUNT(note_sur_20)` ne les comptera pas, alors que `COUNT(*)` comptera la ligne. Pour un effectif d'élèves présents à l'épreuve, utilisez `COUNT(note_sur_20)`. Pour l'effectif total d'inscrits (présents + absents), utilisez `COUNT(*)`.

5.2 Tri des résultats : ORDER BY

Par défaut, l'ordre des lignes retournées par `SELECT` est **indéterminé** (dépend du moteur de stockage, des index, etc.). Pour garantir un ordre, on utilise `ORDER BY`.

PROPRIÉTÉ Syntaxe ORDER BY

```
SELECT liste_colonnes
FROM table
[WHERE condition]
ORDER BY colonne1 [ASC|DESC], colonne2 [ASC|DESC], ... ;
```

- ASC (ascendant) : par défaut (A→Z, 0→9, date ancienne→récente).
- DESC (descendant) : ordre inverse (Z→A, 9→0, date récente→ancienne).
- Tri multi-critères : on trie d'abord par `colonne1`, puis à l'intérieur des égalités par `colonne2`, etc.
- On peut trier par un alias défini dans le `SELECT` ou par le numéro de colonne (1 pour la 1ère colonne du `SELECT`), mais l'alias est plus lisible.

EXEMPLE Tri des élèves par classe puis par moyenne décroissante

```
SELECT
    e.nom, e.prenom, c.libelle AS classe,
    AVG(n.note_sur_20) AS moyenne
FROM ELEVE e
JOIN CLASSE c ON e.id_classe = c.id_classe
JOIN NOTE n ON e.id_eleve = n.id_eleve
GROUP BY e.id_eleve, e.nom, e.prenom, c.libelle
ORDER BY c.libelle ASC, moyenne DESC;
```

Interprétation : Les élèves sont groupés par classe (ordre alphabétique), et à l'intérieur de chaque classe, classés du meilleur au plus faible.

5.3 Regroupement : GROUP BY et filtrage des groupes : HAVING

Jusqu'ici, les fonctions d'agrégation s'appliquaient sur toute la table (ou le sous-ensemble WHERE). GROUP BY permet de les appliquer **séparément pour chaque groupe** de lignes partageant une même valeur.

DÉFINITION Clause GROUP BY

```
SELECT colonne_groupe, fonction_agreg(colonne_cible), ...
FROM table
[WHERE condition_lignes]
GROUP BY colonne_groupe [, colonne_groupe2 ...];
```

Règle stricte (SQL standard) : Toute colonne apparaissant dans le SELECT (hors fonctions d'agrégation) doit figurer dans le GROUP BY. Sinon : erreur ONLY_FULL_GROUP_BY (MySQL strict, PostgreSQL, SQL Server).

PROPRIÉTÉ Différence fondamentale WHERE vs HAVING

Critère	WHERE	HAVING
Moment	Filtre avant le regroupement (lignes brutes)	Filtre après le regroupement (groupes constitués)
Colonnes autorisées	Colonnes de la table (pas d'agrégats)	Colonnes du GROUP BY et fonctions d'agrégation
Usage typique	« Notes > 10 », « Classe = 'Terminale A' »	« Moyenne > 12 », « Effectif > 30 »

MÉTHODE Écrire une requête avec regroupement et HAVING

1. Identifier la colonne de regroupement (ex. `id_classe`).
2. Écrire le `SELECT` : colonne de regroupement + fonctions d'agrégation (+ alias).
3. Ajouter `FROM` et les `JOIN` nécessaires.
4. Ajouter `WHERE` pour filtrer les lignes sources (ex. ne garder que les notes du 1er trimestre).
5. Ajouter `GROUP BY` avec la/les colonne(s) de regroupement.
6. Ajouter `HAVING` pour filtrer les groupes résultants (ex. classes ayant plus de 40 élèves).
7. Ajouter `ORDER BY` pour trier le résultat final.

EXEMPLE Effectif et moyenne par classe (uniquement classes > 30 élèves)

Contexte : Proviseur du Lycée Bilingue d'Essos veut les classes surchargées.

```
SELECT
  c.libelle AS classe,
  COUNT(e.id_eleve) AS effectif,
  ROUND(AVG(n.note_sur_20), 2) AS moyenne_classe
FROM CLASSE c
JOIN ELEVE e ON c.id_classe = e.id_classe
JOIN NOTE n ON e.id_eleve = n.id_eleve
WHERE n.id_trimestre = 1 -- Filtre lignes : seulement notes T1
GROUP BY c.id_classe, c.libelle
HAVING COUNT(e.id_eleve) > 30 -- Filtre groupes : classes > 30 élèves
ORDER BY effectif DESC;
```

ATTENTION Erreur classique : colonne « orpheline » dans le SELECT

```
-- ERREUR (mode strict) : 'nom' n'est ni dans GROUP BY ni agrégé
SELECT c.libelle, e.nom, COUNT(*)
FROM CLASSE c JOIN ELEVE e ON c.id_classe = e.id_classe
GROUP BY c.libelle;

-- CORRECT : soit on groupe par les deux, soit on agrège 'nom'
SELECT c.libelle, GROUP_CONCAT(e.nom), COUNT(*)
FROM CLASSE c JOIN ELEVE e ON c.id_classe = e.id_classe
GROUP BY c.libelle;
```

Astuce MySQL : `GROUP_CONCAT(col)` concatène les valeurs du groupe. **PostgreSQL** : `STRING_AGG(col, ',')`.

5.4 Données calculées et alias : enrichir le résultat

Le SELECT n'est pas limité aux colonnes de la table. On peut y placer des **expressions** (arithmétiques, chaînes, dates, conditions) et leur donner un nom (**alias**) avec AS (optionnel mais recommandé).

PROPRIÉTÉ Expressions dans le SELECT

- **Arithmétique** : + - * / (priorités usuelles, parenthèses). Attention : division entière en SQL Server / PostgreSQL si les deux opérandes sont entiers → `caster : col::numeric` ou `CAST(col AS DECIMAL)`.
- **Concaténation** : || (standard, PostgreSQL, Oracle) ou `CONCAT()` (MySQL, SQL Server).
- **Conditionnel** : `CASE WHEN ... THEN ... ELSE ... END` (standard).
- **Alias** : `AS nom_alias` (ou juste `nom_alias`). L'alias est utilisable dans `ORDER BY`, `HAVING`, `GROUP BY` (selon SGBD), mais pas dans `WHERE` (évalué avant le SELECT).

EXEMPLE Calcul du montant restant dû par élève (contexte : gestion scolaire)

Tables : `INSCRIPTION(id_eleve, id_annee, montant_du)`, `PAIEMENT(id_eleve, id_annee, montant_payé)`.

```
SELECT
  e.matricule,
  CONCAT(e.nom, ' ', e.prenom) AS eleve,
  i.montant_du AS total_du,
  COALESCE(SUM(p.montant_paye), 0) AS total_paye,
  (i.montant_du - COALESCE(SUM(p.montant_paye), 0)) AS reste_du,
  CASE
    WHEN (i.montant_du - COALESCE(SUM(p.montant_paye), 0)) <= 0 THEN 'Soldé'
    ELSE 'Impayé'
  END AS statut
FROM ELEVE e
JOIN INSCRIPTION i ON e.id_eleve = i.id_eleve AND i.id_annee = 2024
LEFT JOIN PAIEMENT p ON e.id_eleve = p.id_eleve AND p.id_annee = 2024
GROUP BY e.id_eleve, e.matricule, e.nom, e.prenom, i.montant_du
HAVING reste_du > 0
ORDER BY reste_du DESC;
```

Notes : `COALESCE` gère les `NULL` (aucun paiement). `LEFT JOIN` garde les élèves n'ayant rien payé. Alias `reste_du` réutilisé dans `HAVING` (MySQL) et `ORDER BY`.

5.5 Jointures : croiser les tables via les clés étrangères

Une base de données relationnelle **normalisée** éparpille l'information dans plusieurs tables pour éviter la redondance. La **jointure** (JOIN) permet de **reconstituer l'information complète** en reliant les tables via leurs clés (primaire ↔ étrangère).

DÉFINITION Jointure interne (INNER JOIN ou JOIN)

Ne retourne que les lignes pour lesquelles la condition de jointure est vraie dans les deux tables. Si un élève n'a pas de classe affectée (id_classe NULL) ou si une classe n'a pas d'élève, ils n'apparaissent pas.

```
-- Syntaxe explicite (recommandée : lisible, standard SQL-92)
SELECT e.nom, e.prenom, c.libelle AS classe
FROM ELEVE e
INNER JOIN CLASSE c ON e.id_classe = c.id_classe;

-- Syntaxe implicite (ancienne, à éviter : risque de produit cartésien oublié)
SELECT e.nom, e.prenom, c.libelle
FROM ELEVE e, CLASSE c
WHERE e.id_classe = c.id_classe;
```

ELEVE			CLASSE		
id_eleve (PK)	nom	prenom	id_classe (PK)	libelle	effectif_max
101	Mbarga	Jean	1	Terminale A	50
102	Ngono	Marie	2	Terminale C	45
103	Essomba	Paul	3	Terminale D	40
104	Atangana	Grace	NULL		

RÉSULTAT (INNER JOIN)

nom	prenom	classe
Mbarga	Jean	Terminale A
Ngono	Marie	Terminale A
Essomba	Paul	Terminale C

Atangana Grace (id_classe NULL) exclue. Terminale D (sans élève) exclue.

Illustration d'une jointure interne ELEVE ⋈ CLASSE sur la clé étrangère id_classe.

MÉTHODE Écrire une jointure en 3 étapes

1. Identifier les tables nécessaires pour répondre à la question (ex. ELEVE pour le nom, CLASSE pour le libellé, NOTE pour les notes).
2. Identifier les clés de liaison : pour chaque paire de tables, quelle colonne (FK) dans l'une correspond à quelle colonne (PK) dans l'autre? (ex. ELEVE.id_classe = CLASSE.id_classe).

3. Écrire la requête : `SELECT` les colonnes voulues (préfixées par l'alias de table si ambiguïté), `FROM` `table_principale`, puis `JOIN` `table_secondaire` `ON` condition, `WHERE` filtres métiers, `GROUP BY` / `HAVING` / `ORDER BY` si besoin.

5.6 Jointures sur plus de deux tables et alias de tables

Pour croiser N tables, on enchaîne $N - 1$ clauses `JOIN`. L'ordre logique n'importe pas pour le résultat (l'optimiseur s'en charge), mais l'ordre d'écriture aide la lecture : commencez par la table « centrale » (souvent celle qui a les FK vers les autres) ou celle du `WHERE` principal.

PROPRIÉTÉ Qualification des colonnes et alias de tables

- Qualification obligatoire : Si une colonne existe dans plusieurs tables de la requête (ex. `id_classe` dans `ELEVE` et `CLASSE`), on doit la préfixer : `e.id_classe` ou `c.id_classe`.
- Alias de table : Court (1–3 lettres), significatif (`e` pour `ELEVE`, `c` pour `CLASSE`, `n` pour `NOTE`). Utilisé pour qualifier les colonnes et raccourcir l'écriture.
- Auto-jointure : Une table jointe à elle-même (ex. hiérarchie employeur/employé). Nécessite deux alias distincts pour la même table.

EXEMPLE Jointure 4 tables : Bulletin complet (Élève, Classe, Matière, Note)

```
SELECT
  e.matricule,
  CONCAT(e.nom, ' ', e.prenom) AS eleve,
  cl.libelle AS classe,
  m.libelle AS matiere,
  n.note_sur_20,
  n.coefficient
FROM ELEVE e
JOIN CLASSE cl ON e.id_classe = cl.id_classe
JOIN NOTE n ON e.id_eleve = n.id_eleve
JOIN MATIERE m ON n.id_matiere = m.id_matiere
WHERE e.id_eleve = 101 -- Filtre sur un élève précis
      AND n.id_trimestre = 1
ORDER BY m.libelle;
```

Résultat : Une ligne par note de l'élève 101 au T1, avec le nom de la matière et sa classe.

EXERCICE RÉSOLU 5 Moyenne pondérée par élève et par matière (coefficient)

Énoncé : Calculer la moyenne pondérée de chaque élève pour chaque matière au 1er tri-

mestre. Afficher : Matricule, Nom, Prénom, Matière, Moyenne_ponderee (arrondie à 2 décimales). Trier par Élève puis Matière.

SOLUTION

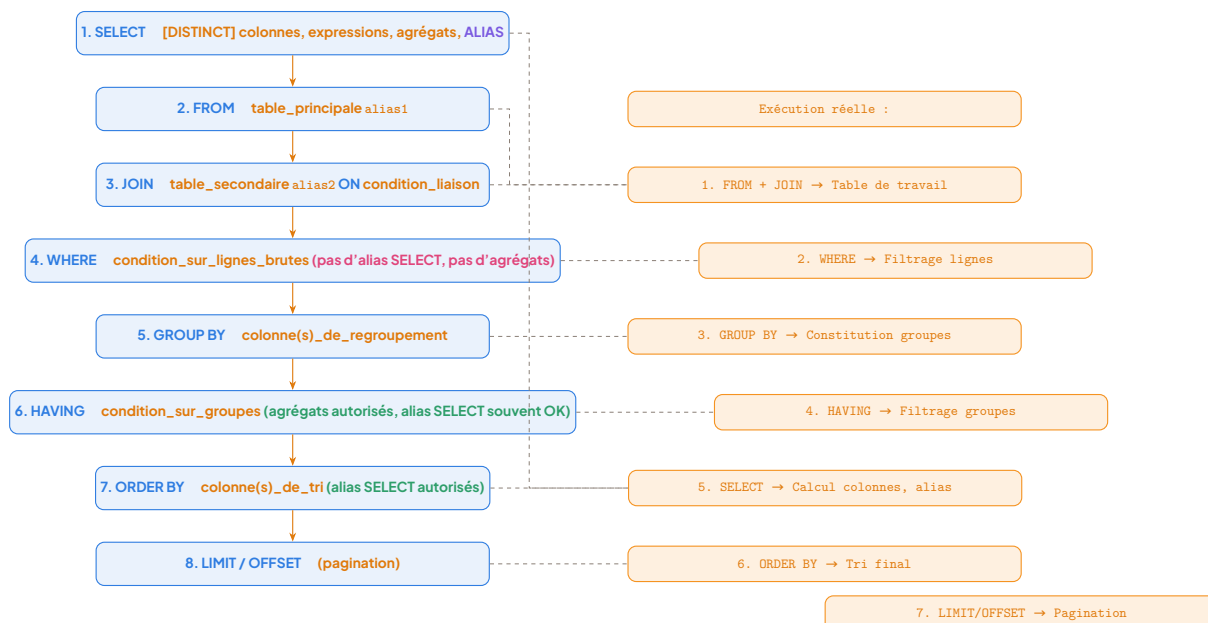
Solution détaillée : La moyenne pondérée = $\frac{\sum(note \times coef)}{\sum(coef)}$.

```
SELECT
  e.matricule,
  e.nom, e.prenom,
  m.libelle AS matiere,
  ROUND(SUM(n.note_sur_20 * n.coefficient) / SUM(n.coefficient), 2) AS moyenne_ponderee
FROM ELEVE e
JOIN NOTE n ON e.id_eleve = n.id_eleve
JOIN MATIERE m ON n.id_matiere = m.id_matiere
WHERE n.id_trimestre = 1
GROUP BY e.id_eleve, e.matricule, e.nom, e.prenom, m.id_matiere, m.libelle
ORDER BY e.nom, e.prenom, m.libelle;
```

Points clés : Utilisation d'expressions dans SUM(). GROUP BY sur toutes les colonnes non agrégées du SELECT. Qualification n.coefficient (la note porte son propre coef, souvent défini au niveau matière mais stocké en redondance contrôlée dans NOTE pour l'historique).

5.7 Synthèse : Ordre logique d'écriture et d'exécution d'une requête SELECT complète

Comprendre l'ordre **logique** d'évaluation (différent de l'ordre d'écriture) est crucial pour savoir où placer les alias, les agrégats et les filtres.



Ordre d'écriture (gauche, syntaxe SQL) vs Ordre d'exécution logique (droite, moteur). Comprendre ce décalage évite les erreurs « Unknown column 'alias' in 'where clause' ou « Invalid use of group function ».

À RETENIR

Checklist pour une requête SELECT complexe réussie

- 1. SELECT** : Liste explicite (pas de *). Alias clair pour chaque expression/agrégat.
- 2. FROM / JOIN** : Tables nécessaires, alias courts, conditions ON exactes (PK=FK). LEFT JOIN si optionnel.
- 3. WHERE** : Filtres métiers sur colonnes brutes (date, statut, id). Jamais d'agrégats ni d'alias du SELECT.
- 4. GROUP BY** : Toutes les colonnes du SELECT non agrégées. Clé primaire de l'entité principale souvent suffisante (ex. e.id_eleve) si SGBD supporte la dépendance fonctionnelle (MySQL 5.7+, PostgreSQL 9.1+), sinon lister toutes.
- 5. HAVING** : Filtres sur les groupes (agrégats, COUNT, AVG...). Alias du SELECT souvent utilisables.
- 6. ORDER BY** : Tri final. Alias du SELECT toujours utilisables. ASC/DESC explicite.
- 7. LIMIT** : Pour la pagination (ex. LIMIT 20 OFFSET 40 page 3).

EXPÉRIENCE TP Guidé : Tableau de bord « Scolarité » sur PhpMyAdmin / MySQL

Objectif : Produire trois requêtes décisionnelles sur la base `ecole_db`. Prérequis : Base `ecole_db` peuplée (tables `ELEVE`, `CLASSE`, `MATIERE`, `NOTE`, `INSCRIPTION`, `PAIEMENT`). Script SQL fourni en ressource. Matériel : Navigateur web, serveur WampServer/XAMPP, PhpMyAdmin (onglet SQL).

1. Requête 1 : « Top 5 des élèves par moyenne générale T1 »

Écrire la requête calculant la moyenne pondérée (coefficient dans `NOTE`) par élève pour `id_trimestre=1`, triée décroissante, limitée à 5.

```
SELECT e.matricule, CONCAT(e.nom, ' ', e.prenom) AS eleve,
       ROUND(SUM(n.note_sur_20 * n.coefficient)/SUM(n.coefficient),2) AS moy_gen
FROM ELEVE e
JOIN NOTE n ON e.id_eleve = n.id_eleve
WHERE n.id_trimestre = 1
GROUP BY e.id_eleve, e.matricule, e.nom, e.prenom
ORDER BY moy_gen DESC
LIMIT 5;
```

Observation : Vérifier que les élèves sans note au T1 n'apparaissent pas (INNER JOIN). Noter l'usage de l'alias `moy_gen` dans ORDER BY.

2. Requête 2 : « Classes en situation d'échec (moyenne classe < 10) »

Afficher `libelle_classe`, `effectif`, `moyenne_classe` pour les classes dont la moyenne générale (toutes matières, tous élèves, T1) est strictement inférieure à 10.

```
SELECT c.libelle AS classe, COUNT(DISTINCT e.id_eleve) AS effectif,
       ROUND(AVG(n.note_sur_20), 2) AS moyenne_classe
FROM CLASSE c
JOIN ELEVE e ON c.id_classe = e.id_classe
JOIN NOTE n ON e.id_eleve = n.id_eleve
WHERE n.id_trimestre = 1
GROUP BY c.id_classe, c.libelle
HAVING AVG(n.note_sur_20) < 10
ORDER BY moyenne_classe ASC;
```

Interprétation : `COUNT(DISTINCT e.id_eleve)` évite de compter un élève multiple fois s'il a plusieurs notes. `HAVING` filtre sur l'agrégat `AVG`.

3. Requête 3 : « Situation financière par élève (Inscription 2024) »

Lister tous les élèves inscrits en 2024 avec : `Matricule`, `Nom`, `Montant_du`, `Total_payé`, `Reste_du`, `Statut` (Soldé/Impayé). Inclure les élèves n'ayant rien payé. Trier par `Reste_du` décroissant.

```
SELECT e.matricule, CONCAT(e.nom, ' ', e.prenom) AS eleve,
       i.montant_du,
       COALESCE(SUM(p.montant_paye), 0) AS total_paye,
       (i.montant_du - COALESCE(SUM(p.montant_paye), 0)) AS reste_du,
       CASE WHEN (i.montant_du - COALESCE(SUM(p.montant_paye), 0)) <= 0 THEN 'Soldé' ELSE 'Impayé' END AS statut
```

```
FROM ELEVE e
JOIN INSCRIPTION i ON e.id_eleve = i.id_eleve AND i.id_annee = 2024
LEFT JOIN PAIEMENT p ON e.id_eleve = p.id_eleve AND p.id_annee = 2024
GROUP BY e.id_eleve, e.matricule, e.nom, e.prenom, i.montant_du
ORDER BY reste_du DESC;
```

Point clé : LEFT JOIN sur PAIEMENT + COALESCE(SUM(...), 0). Sans LEFT JOIN, les élèves sans paiement disparaîtraient. Sans COALESCE, SUM renverrait NULL et le calcul reste_du deviendrait NULL.

Bilan : Vous maîtrisez maintenant l'extraction d'indicateurs synthétiques (KPIs) à partir de données relationnelles normalisées.

LE SAVAIS-TU ?**Le Cameroun et la data : du bulletin papier au Data Lake**

Au Cameroun, la réforme du système éducatif (Loi n°98/004 du 14 avril 1998) impose la gestion informatisée des notes. Le MINEDUB déploie progressivement SIGE (Système d'Information de Gestion de l'Éducation) qui centralise les données de milliers d'établissements. Les requêtes d'agrégation que vous venez d'écrire (COUNT, AVG, GROUP BY) sont exactement celles qui tournent chaque soir sur les serveurs du CENADI (Centre National de Documentation et d'Information) pour produire les statistiques nationales : taux de réussite par région, par série, par genre, effectifs par classe pour la planification budgétaire (construction de salles, allocation de manuels). Une erreur de GROUP BY ou un INNER JOIN mal placé sur ces volumes (millions de lignes) fausserait la décision publique. La rigueur SQL n'est pas un exercice scolaire : c'est une compétence citoyenne et professionnelle.

EXERCICE 5**Entraînement : Base de données « Cybercafé Yaoundé »**

Soit le MCD simplifié : CLIENT(id_cli, nom, prenom, telephone), POSTE(id_poste, configuration, tarif_horaire), SESSION(id_cli, id_poste, debut, fin).

1. Écrire la requête qui affiche pour chaque client : son nom, le nombre total de sessions, la durée totale en heures (arrondie 2 décimales), le montant total dû (durée × tarif horaire du poste utilisé). Trier par montant décroissant.
2. Écrire la requête qui liste les postes jamais utilisés (aucune session). Indice : LEFT JOIN + WHERE ... IS NULL.
3. Écrire la requête qui donne le chiffre d'affaires journalier du mois de janvier 2024 (date, CA_jour). Utiliser DATE(debut) pour grouper par jour.

CORRIGÉ Exercice 1

1. Stats client :

```
SELECT c.nom, c.prenom,  
       COUNT(s.id_session) AS nb_sessions,  
       ROUND(SUM(TIMESTAMPDIFF(MINUTE, s.debut, s.fn))/60, 2) AS duree_heures,  
       ROUND(SUM(TIMESTAMPDIFF(MINUTE, s.debut, s.fn))/60 * p.tarif_horaire, 2) AS montant_du  
FROM CLIENT c  
JOIN SESSION s ON c.id_cli = s.id_cli  
JOIN POSTE p ON s.id_poste = p.id_poste  
GROUP BY c.id_cli, c.nom, c.prenom  
ORDER BY montant_du DESC;
```

2. Postes inutilisés :

```
SELECT p.id_poste, p.configuration  
FROM POSTE p  
LEFT JOIN SESSION s ON p.id_poste = s.id_poste  
WHERE s.id_session IS NULL;
```

3. CA Journalier Janvier 2024 :

```
SELECT DATE(s.debut) AS jour,  
       ROUND(SUM(TIMESTAMPDIFF(MINUTE, s.debut, s.fn))/60 * p.tarif_horaire, 2) AS ca_jour  
FROM SESSION s  
JOIN POSTE p ON s.id_poste = p.id_poste  
WHERE s.debut >= '2024-01-01' AND s.debut < '2024-02-01'  
GROUP BY DATE(s.debut)  
ORDER BY jour;
```

6 ADMINISTRATION : EXPORTATION, IMPORTATION ET SAUVEGARDE

Dans la section précédente, tu as appris à interroger ta base avec des requêtes puissantes : agrégations, tris, regroupements et jointures. Ta base du lycée est maintenant vivante : elle contient des élèves, des notes, des classes, et tu sais en extraire des bulletins. Mais une question de survie demeure : **que deviennent toutes ces données le jour où le serveur tombe en panne** ? Un disque dur qui lâche en pleine année scolaire, une coupure ENEO brutale pendant une écriture, un virus, une mauvaise manipulation (un `DELETE` sans `WHERE...`), et des mois de saisie s'évaporent. L'administration d'une base de données, c'est précisément l'ensemble des gestes qui garantissent que les données survivront à n'importe quel incident. C'est le dernier maillon de la chaîne — et souvent le plus négligé.

6.1 Pourquoi administrer une base de données ?

Trois situations concrètes justifient à elles seules cette section :

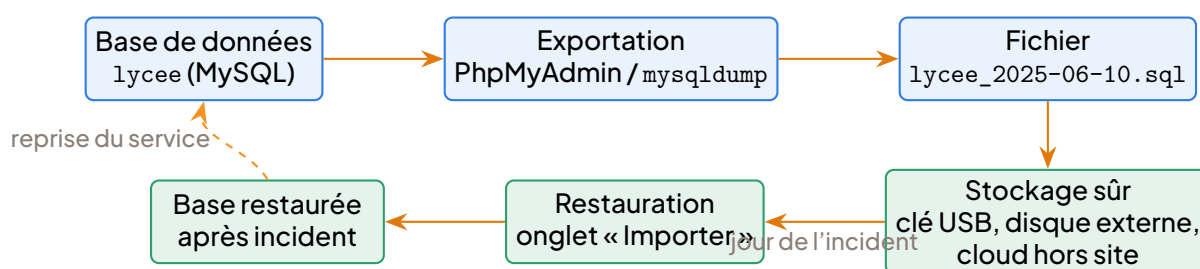
- **Sécurité et continuité de service.** Les données d'un établissement scolaire (notes, effectifs, paiements) sont critiques. Une sauvegarde régulière permet de restaurer la base en quelques minutes après un incident, au lieu de tout ressaisir.
- **Migration.** Le lycée change d'hébergeur, ou l'application développée sur le poste du club informatique doit être déployée sur le serveur du CENADI. Il faut alors transporter la base d'une machine à une autre : on exporte sur l'ancienne, on importe sur la nouvelle.
- **Archivage.** En fin d'année scolaire, on fige la base (notes définitives, effectifs) dans un fichier daté, conservé précieusement. L'année suivante démarre sur une base « propre », mais l'historique reste consultable.

DÉFINITION Sauvegarde, exportation, importation

Une **sauvegarde** (ou backup) est une copie de sécurité des données, stockée séparément du serveur, permettant de restaurer la base en cas de perte ou de corruption. L'**exportation** est l'opération qui consiste à extraire le contenu d'une base (structure et/ou données) vers un fichier (SQL, CSV...) lisible et transportable. L'**importation** est l'opération inverse : on recharge un fichier d'exportation dans un SGBD pour recréer la structure et les données.

À RETENIR Le cycle de vie d'une sauvegarde

Une sauvegarde n'a de valeur que si elle forme un cycle complet : base → exportation → fichier → stockage sûr → (un jour) restauration. Si un maillon manque — par exemple un fichier jamais testé — la sauvegarde est illusoire.



Cycle de vie d'une sauvegarde : de la base vivante au fichier daté, puis à la restauration après incident.

6.2 Exportation dans PhpMyAdmin

PhpMyAdmin, l'interface web de MySQL/MariaDB installée avec WampServer ou XAMPP, propose un onglet **Exporter** accessible à deux niveaux :

- en sélectionnant **une table** puis « Exporter » : seule cette table est extraite ;
- en sélectionnant **la base entière** (ou sans rien sélectionner, depuis la page d'accueil) : toutes les tables sont extraites.

Deux méthodes sont proposées : **Rapide** (options par défaut, suffisantes dans la plupart des cas) et **Personnalisée** (choix des tables, du format, de la compression, export de la structure seule ou des données seules). Les formats les plus utiles au programme sont :

Format	Contenu exporté	Usage typique
SQL	Structure (CREATE TABLE, clés, contraintes) et données (INSERT)	Sauvegarde complète, migration, archivage fidèle
CSV	Données seules, une ligne par enregistrement, valeurs séparées par des points-virgules	Échange avec un tableur (Excel), statistiques

ATTENTION CSV : les données sans la structure

Exporter en **CSV** ne conserve que les données brutes : ni les types de colonnes, ni les clés primaires, ni les clés étrangères, ni les contraintes d'intégrité ne sont sauvegardés. Ré-importer un CSV exige de recréer la table à la main, au bon format, colonne par colonne. Seul le format SQL conserve tout (structure + données + contraintes) et permet une restauration fidèle en une seule opération. Pour une sauvegarde de sécurité, choisis toujours SQL.

Un fichier d'exportation SQL n'est rien d'autre qu'un long script texte. En voici un extrait réalistique :

```
-- Exportation de la base 'lycee' -- 10/06/2025
CREATE DATABASE IF NOT EXISTS 'lycee';
USE 'lycee';

CREATE TABLE 'eleve' (
  'id_eleve' INT NOT NULL AUTO_INCREMENT,
  'nom' VARCHAR(50) NOT NULL,
  'date_naiss' DATE NOT NULL,
  'id_classe' INT NOT NULL,
  PRIMARY KEY ('id_eleve'),
  FOREIGN KEY ('id_classe') REFERENCES 'classe' ('id_classe')
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

INSERT INTO 'eleve' ('id_eleve', 'nom', 'date_naiss', 'id_classe') VALUES
(1, 'Ngo Bell Amina', '2007-03-14', 2),
(2, 'Etoundi Paul', '2006-11-02', 2),
(3, 'Mbarga Solange', '2007-07-25', 3);
```

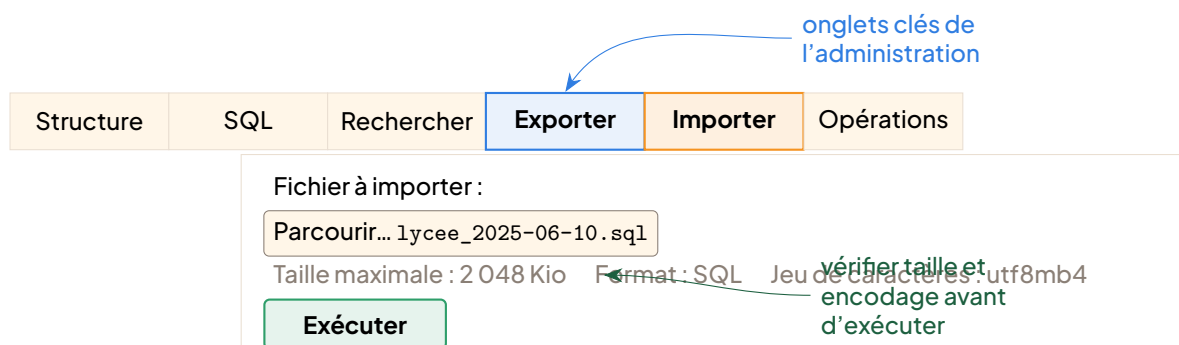
On y reconnaît tout ce que tu as appris : création de la base, création des tables avec leurs contraintes, puis insertion des enregistrements. Réimporter ce fichier revient simplement à **rejouer le script**, instruction par instruction, dans l'ordre du fichier.

6.3 Importation dans PhpMyAdmin

L'importation se fait via l'onglet **Importer** : on choisit le fichier sur son poste (bouton « Parcourir »), on vérifie le format détecté (SQL en général), puis on clique sur « Exécuter ». PhpMyAdmin rejoue le script et affiche le nombre de requêtes exécutées.

ATTENTION Erreurs fréquentes à l'importation

- Fichier trop volumineux : PhpMyAdmin limite la taille des envois (souvent 2 Mio ou 8 Mio, directives `upload_max_filesize` et `post_max_size` de PHP). Solution : compresser le fichier en `.zip` (PhpMyAdmin l'accepte) ou passer par la ligne de commande.
- Problème d'encodage : si le fichier a été enregistré en Latin-1 et importé en UTF-8, les accents deviennent des caractères bizarres (par exemple Ngo BÃ©ll au lieu de Ngo Béll). Exporte et importe toujours en UTF-8 (`utf8mb4`).
- Base déjà existante : réimporter un script contenant `CREATE TABLE` sur une base non vide provoque des erreurs « table already exists ». Avant une restauration, supprime (ou vide) l'ancienne base.



Reproduction annotée des onglets « Exporter » / « Importer » de PhpMyAdmin : le cœur de l'administration au quotidien.

MÉTHODE Exporter puis restaurer une base dans PhpMyAdmin

Exportation (sauvegarde) :

1. Ouvre PhpMyAdmin et sélectionne la base `lycee` dans la colonne de gauche.
2. Clique sur l'onglet Exporter, méthode « Rapide », format SQL.
3. Clique sur « Exécuter » : le navigateur télécharge un fichier `lycee.sql`.
4. Renomme-le immédiatement avec la date : `lycee_2025-06-10.sql`, et copie-le sur un support externe.

Restauration (après incident) :

1. Crée une base vide de même nom (ou supprime l'ancienne base corrompue puis recrée-la).
2. Sélectionne cette base, ouvre l'onglet Importer.
3. Choisis le fichier `lycee_2025-06-10.sql`, vérifie l'encodage UTF-8.
4. Clique sur « Exécuter » et contrôle le message de succès (nombre de requêtes exécutées).
5. Vérifie : ouvre quelques tables, compte les enregistrements, teste une requête de jointure.

ATTENTION Une sauvegarde non testée n'est pas une sauvegarde

Une exportation n'est utile que si l'on a vérifié qu'on peut la réimporter. Un fichier corrompu, tronqué ou mal encodé ne sert à rien le jour de l'incident. Bonne habitude professionnelle : après chaque sauvegarde importante, teste la restauration sur une base vide (par exemple `lycee_test`) et contrôle le résultat. Cinq minutes de test valent mieux qu'une année de données perdues.

EXPÉRIENCE TP guidé : sauvegarde et restauration complète de la base `lycee`

Objectif : maîtriser le cycle complet exportation → destruction simulée → restauration.

Matériel : poste avec WampServer/XAMPP, PhpMyAdmin, base `lycee` remplie (tables `eleve`, `classe`, `note`).

1. Exporte la base `lycee` au format SQL ; renomme le fichier `lycee_TP_2025-06-10.sql` et copie-le sur une clé USB.
2. Simule l'incident : dans l'onglet SQL, exécute `DROP TABLE eleve;` — la table disparaît.
3. Restaure : onglet « Importer », choisis ton fichier, exécute.
4. Vérifie : `SELECT COUNT(*) FROM eleve;` doit redonner l'effectif initial ; teste une jointure `eleve-classe`.

Observations attendues : la table restaurée contient exactement les données du jour de l'export ; les enregistrements saisis après l'export sont perdus — d'où l'importance de sauvegarder régulièrement. Interprétation : une sauvegarde ne protège que ce qui existait au moment de l'export. La fréquence des sauvegardes détermine la perte maximale acceptable.

6.4 Complément utile au Bac : la sauvegarde en ligne de commande avec mysqldump

PhpMyAdmin est pratique, mais limité (taille des fichiers, dépendance au navigateur). Les administrateurs professionnels utilisent l'outil en ligne de commande **mysqldump**, fourni avec MySQL. Cette notion est un complément utile pour le Bac : elle n'est pas exigible, mais elle valorise une copie.

PROPRIÉTÉ Syntaxe de mysqldump

```
-- Sauvegarder toute la base 'lycee' dans un fichier :  
mysqldump -u root -p lycee > lycee_2025-06-10.sql  
  
-- Sauvegarder une seule table :  
mysqldump -u root -p lycee eleve > table_eleve.sql  
  
-- Restaurer (sens inverse, avec le client mysql) :  
mysql -u root -p lycee < lycee_2025-06-10.sql
```

L'option `-u` précise l'utilisateur, `-p` demande la saisie du mot de passe, le symbole `>` redirige la sortie vers un fichier, `<` recharge un fichier.

L'avantage décisif : **mysqldump** peut être **automatisé** (tâche planifiée chaque nuit à 2 h), sans intervention humaine — impossible avec PhpMyAdmin.

6.5 Bonnes pratiques de sauvegarde

À RETENIR Les réflexes de l'administrateur

- **Régularité** : sauvegarde quotidienne pour une base active (notes saisies chaque semaine), au minimum hebdomadaire.
- **Nommage daté** : `lycee_2025-06-10.sql` plutôt que `sauvegarde_finale_v2.sql`. Le format `AAAA-MM-JJ` se trie naturellement par ordre chronologique.
- **Supports multiples** : une copie sur le poste ne protège ni du vol ni de la panne du disque.
- **Tests de restauration périodiques** sur une base d'essai.

LESAVAIS-TU ? La règle « 3-2-1 » des sauvegardes

Les professionnels de l'informatique appliquent la règle 3-2-1 : conserver 3 copies de chaque donnée importante, sur 2 supports de nature différente (par exemple disque dur du serveur + clé USB), dont 1 copie hors site (dans un autre bâtiment, ou sur un espace cloud sécurisé). Ainsi, même un incendie de la salle informatique ne détruit pas toutes les copies.

C'est cette règle qu'appliquent les hébergeurs et les administrations numérisées comme le CENADI ou `impots.cm` pour protéger les données des citoyens camerounais.

6.6 Scénario guidé : restauration après incident, de A à Z

Situation. Mardi 8 h, au secrétariat du lycée : l'application de gestion affiche « Table 'lycee.note' doesn't exist ». Une mauvaise manipulation a détruit la table des notes la veille au soir. La dernière sauvegarde date de lundi 18 h : `lycee_2025-06-09.sql`.

1. **Diagnostic** : dans PhpMyAdmin, onglet Structure — la table `note` a bien disparu. Pas de panique : la sauvegarde existe.
2. **Isolation** : on empêche toute nouvelle saisie pendant la restauration (sinon les nouvelles données seraient écrasées).
3. **Restauration** : onglet « Importer », fichier `lycee_2025-06-09.sql`, exécution. Message : « 1 248 requêtes exécutées avec succès ».
4. **Vérification** : `SELECT COUNT(*) FROM note;` redonne les 1 200 notes attendues ; un bulletin test est régénéré et comparé à un bulletin imprimé.
5. **Bilan** : seules les notes saisies mardi matin avant l'incident sont perdues — elles seront res-saisies à la main. **Leçon** : passer à une sauvegarde quotidienne automatique avec `mysqldump`.

6.7 Survol : gestion des utilisateurs et des droits (GRANT)

Dernier volet de l'administration : **qui a le droit de faire quoi** sur la base. Le compte `root` (super-utilisateur) ne doit jamais être utilisé par une application ni partagé. On crée des comptes dédiés avec des droits minimaux, grâce à l'instruction **GRANT** (complément utile, non exigible) :

```
-- Créer un utilisateur 'secretaire' avec mot de passe
CREATE USER 'secretaire'@'localhost' IDENTIFIED BY 'Mot2Passe!Fort';

-- Lui accorder uniquement lecture et insertion sur la base lycee
GRANT SELECT, INSERT ON lycee.* TO 'secretaire'@'localhost';

-- Retirer un droit si nécessaire
REVOKE INSERT ON lycee.* FROM 'secretaire'@'localhost';
```

Ainsi, même si le mot de passe du secrétariat fuit, un attaquant ne pourra ni supprimer (`DELETE`, `DROP`) ni modifier les notes. C'est le **principe du moindre privilège** : chaque utilisateur ne reçoit que les droits strictement nécessaires à sa tâche.

EXERCICE RÉSOLU 6

Choisir la bonne stratégie de sauvegarde

Énoncé. Le club informatique d'un lycée de Douala gère la base des notes (environ 3 000 enregistrements modifiés chaque semaine). Le proviseur demande : « Une sauvegarde

par trimestre sur le poste du secrétariat, ça suffit, non ? » Réponds en proposant une stratégie argumentée, avec nommage des fichiers et supports.

SOLUTION

Solution. Non, cette stratégie est insuffisante pour deux raisons : (1) avec une sauvegarde trimestrielle, un incident peut faire perdre trois mois de saisie ; (2) une copie unique sur le poste local ne survit ni à une panne de disque, ni à un vol, ni à un incendie. Stratégie proposée, conforme à la règle 3-2-1 :

- Fréquence : sauvegarde automatique quotidienne via `mysqldump` planifié chaque nuit ;
- Nommage daté : `lycee_2025-06-09.sql`, `lycee_2025-06-10.sql`, etc. ;
- 3 copies : le fichier sur le serveur + une copie sur disque externe + une copie hebdomadaire hors site (cloud ou disque conservé au domicile de l'administrateur) ;
- 2 supports différents : disque dur interne et disque externe/cloud ;
- Test mensuel : restauration sur une base `lycee_test` pour vérifier que les fichiers sont exploitables.

Perte maximale en cas d'incident : une journée de saisie, au lieu d'un trimestre.

EXERCICE 6 À toi de jouer

1. Pourquoi le format CSV est-il déconseillé pour une sauvegarde de sécurité ? Cite deux éléments perdus.
2. Écris la commande `mysqldump` qui sauvegarde la base `bibliotheque` dans un fichier daté du 15 mars 2025.
3. Un fichier d'import de 12 Mio est refusé par PhpMyAdmin. Propose deux solutions.
4. Écris l'instruction SQL accordant à l'utilisateur `prof_maths` le seul droit de lecture sur la table `note` de la base `lycee`.

CORRIGÉ Exercice — Administration

1. Le CSV ne conserve que les données brutes : il perd la structure des tables (types des colonnes) et les contraintes (clés primaires, clés étrangères, unicité). La restauration exige de tout recréer manuellement.
2. `mysqldump -u root -p bibliotheque > bibliotheque_2025-03-15.sql`
3. Compresser le fichier en `.zip` avant l'import (PhpMyAdmin l'accepte), ou importer en ligne de commande : `mysql -u root -p lycee < fichier.sql`, qui n'est pas soumis à

la limite de taille d'envoi de PHP.

4. `GRANT SELECT ON lycee.note TO 'prof_maths'@'localhost';`

À RETENIR L'essentiel de la section

- Sauvegarder = exporter la base (format SQL, qui conserve structure + données) vers un fichier daté, copié sur des supports multiples dont un hors site (règle 3-2-1).
- Restaurer = réimporter le fichier via l'onglet « Importer » de PhpMyAdmin (ou `mysql` < en ligne de commande), puis vérifier le résultat.
- Une sauvegarde n'a de valeur que si sa restauration a été testée.
- `mysqldump` automatise les sauvegardes ; `GRANT` limite les droits de chaque utilisateur au strict nécessaire.

7 ACTIVITÉ D'INTÉGRATION

8 ACTIVITÉ D'INTÉGRATION : IMPLÉMENTATION ET ADMINISTRATION D'UNE BASE DE DONNÉES SCOLARITE

8.1 Objectifs de l'activité

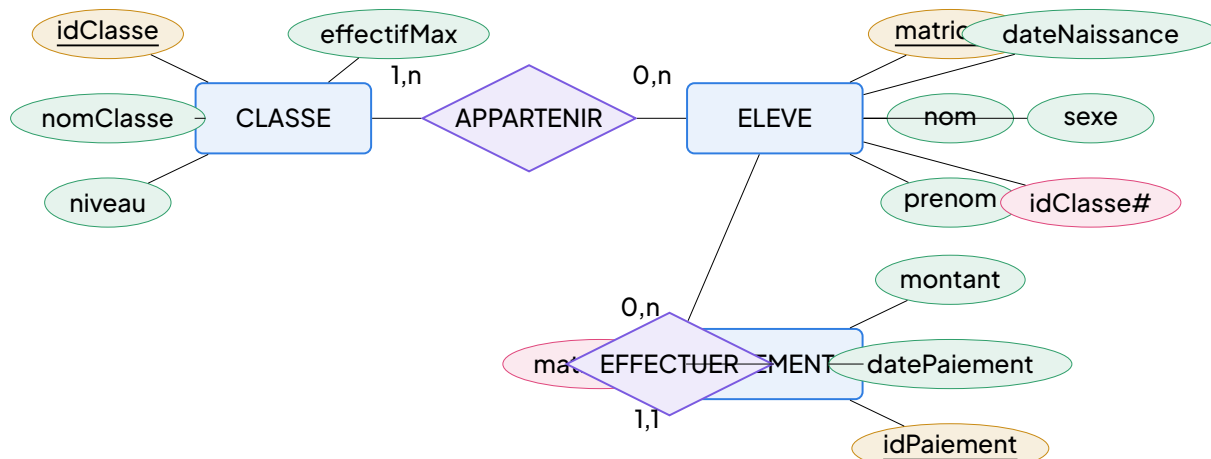
Cette activité d'intégration vise à mobiliser l'ensemble des compétences acquises au cours de la leçon « Implémentation et administration d'une base de données » dans un contexte professionnel réaliste. À l'issue de ce travail, vous devez être capable de :

- Transformer un **Modèle Conceptuel de Données (MCD)** en un **schéma relationnel** normalisé (règles de passage, identification des **clés primaires** et **clés étrangères**).
- Administrer un **SGBD** relationnel (MySQL/MariaDB via **PhpMyAdmin**) : création de la base, des tables, définition des types de données et des **contraintes d'intégrité** (PRIMARY KEY, FOREIGN KEY, NOT NULL, DEFAULT, UNIQUE, CHECK).
- Manipuler les données : insertion (INSERT), modification (UPDATE), suppression (DELETE) et interrogation complexe (SELECT avec filtres, tris, **fonctions d'agrégation**, regroupements, **jointures**).
- Assurer la continuité de service : exporter une base (dump .sql), la supprimer, puis la restaurer par importation pour valider la procédure de sauvegarde.
- Rédiger un rapport technique structuré consignant les requêtes, les captures de résultats et les justifications, simulant une remise au décideur (le proviseur).

8.2 Situation

Le Lycée de Bafoussam, établissement public de référence dans la région de l'Ouest, engage la digitalisation de son secrétariat pour remplacer la gestion papier des inscriptions et de la scolarité. Le proviseur a missionné l'équipe informatique (vous) pour concevoir et déployer la base de données `scolarite` qui centralisera les informations sur les classes, les élèves et le suivi des paiements de la scolarité.

L'analyste a produit le Modèle Conceptuel de Données (MCD) suivant, validé par le censeur et le comptable :



MCD de la base scolaire : Entités CLASSE, ELEVE, PAIEMENT et associations APPARTENIR, EFFECTUER.

Règles de gestion explicites :

- Une classe est identifiée par un idClasse (entier **auto-incrémenté**), possède un nomClasse (ex : « Tle TI »), un niveau (ex : « Terminale ») et un effectifMax.
- Un élève est identifié par son matricule (chaîne 10 caractères, ex : « 2024TI001 »), a un nom, prenom, dateNaissance, sexe ('M' ou 'F') et appartient à une seule classe (idClasse clé étrangère).
- Un paiement est identifié par idPaiement (auto-incrémenté), daté (datePaiement), d'un montant (décimal, défaut 0) et concerne un unique élève (matricule clé étrangère). Un élève peut avoir 0 ou plusieurs paiements (paiements fractionnés).

Jeu de test initial (12 élèves) : Le secrétariat fournit la liste suivante pour la rentrée 2024-2025. Les classes existent déjà (id 1 à 3).

Matricule	Nom	Prénom	Sexe	Date Naiss.	idClasse	Paiements (date, montant FCFA)
2024TI001	Nguimgo	Armand	M	2006-05-12	1	(2024-09-05, 50000), (2024-11-10, 25000)
2024TI002	Tchounga	Grace	F	2007-01-20	1	(2024-09-03, 75000)
2024TI003	Mbarga	Paul	M	2006-11-30	1	Aucun
2024TI004	Nganou	Steve	M	2007-03-15	1	(2024-09-10, 30000)
2024TI005	Fouda	Marie	F	2006-08-22	1	(2024-09-01, 75000)
20241A001	Essomba	Jean	M	2007-07-10	2	(2024-09-02, 60000)
20241A002	Nga	Henriette	F	2008-02-14	2	Aucun
20241A003	Bell	Roger	M	2007-09-05	2	(2024-09-04, 60000)
20241A004	Mvondo	Alice	F	2008-04-18	2	(2024-09-06, 30000), (2024-12-01, 30000)
20242C001	Abena	Pierre	M	2008-12-01	3	(2024-09-01, 55000)
20242C002	Ngah	Julie	F	2009-01-15	3	Aucun
20242C003	Zambo	Kevin	M	2008-06-30	3	(2024-09-03, 55000)

Données initiales : 5 élèves en Tle TI (idClasse=1), 4 en 1ère A (idClasse=2), 3 en 2nde C (idClasse=3). Noms contenant « nga » surlignés.

Le censeur a besoin urgemment des états suivants pour le conseil de discipline de fin de trimestre :

1. La liste complète des élèves de la Terminale TI (classe id=1) avec matricule, nom, prénom.
2. La liste des élèves dont le nom contient la chaîne « nga » (insensible à la casse).
3. Le total des paiements encaissés par classe (nom de la classe + somme), trié par total décroissant.
4. La liste « Nom Prénom — Montant total payé » pour **tous** les élèves (même ceux n'ayant rien payé), triée par montant décroissant.

5. La liste des élèves n'ayant **aucun** paiement enregistré (montant total = 0 ou NULL).

8.3 Consignes

Travail en binôme, durée : 3h (2h machine + 1h rédaction rapport). Chaque groupe remet un fichier PDF unique nommé `Rapport_Groupe_N.pdf` contenant :

1. **Modélisation** : Sur papier (photo/scan intégrée au PDF), proposer le schéma relationnel issu du MCD. Préciser pour chaque table : nom, attributs (nom, type, contraintes), clés primaires (PK), clés étrangères (FK) et cardinalités.
2. **Implémentation (PhpMyAdmin / SQL)** :
 - (a) Créer la base `scolarite` (jeu de caractères `utf8mb4_general_ci`).
 - (b) Créer les 3 tables dans l'ordre des dépendances avec **toutes** les contraintes : PRIMARY KEY, AUTO_INCREMENT si pertinent, NOT NULL sur les attributs obligatoires, DEFAULT 0 sur montant, FOREIGN KEY avec ON DELETE RESTRICT ON UPDATE CASCADE, CHECK (`sexe` IN ('M', 'F')).
 - (c) Insérer les 3 classes, les 12 élèves et les 11 lignes de paiement (requêtes INSERT multiples ou uniques).
3. **Interrogation (Requêtes du censeur)** : Écrire et exécuter les 5 requêtes SQL demandées. Copier-coller le code SQL exact et une capture d'écran du résultat (grille PhpMyAdmin) pour chaque requête.
4. **Administration / Sauvegarde** :
 - (a) Exporter la base `scolarite` en format SQL (structure + données) via l'onglet « Exporter » de PhpMyAdmin (méthode « Personnalisée », compression « none », format « SQL »). Enregistrer le fichier `scolarite_backup_YYYYMMDD.sql`.
 - (b) Supprimer la base `scolarite` (DROP DATABASE).
 - (c) Créer une nouvelle base vide `scolarite_restored`.
 - (d) Importer le fichier `.sql` dans `scolarite_restored` via l'onglet « Importer ».
 - (e) Vérifier l'intégrité : compter les lignes de chaque table (SELECT COUNT(*)) et exécuter la requête n°3 (total paiements par classe) pour confirmer l'identité des résultats.
5. **Bilan personnel** : Rédiger 10 lignes maximum sur les difficultés rencontrées (ex : ordre de création des tables, gestion des NULL dans les jointures, syntaxe LIKE, export/import) et les points à réviser.

8.4 Ressources à mobiliser

PROPRIÉTÉ Règles de passage MCD → Relationnel (Mérise)

- Entité forte → Table (nom entité), attributs = colonnes, identifiant = **PK**.
- Entité faible (identifiant relatif) → Table, PK composée : identifiant relatif + PK de l'entité propriétaire (clé étrangère).
- Association 1 : n → Clé étrangère migrée vers la table du côté « n » (référence la PK du côté « 1 »).
- Association n : m → Table de liaison (PK composée des deux PK étrangères).
- Attribut multi-valué → Table séparée (PK composée : PK entité + attribut).

PROPRIÉTÉ Syntaxe SQL clé (MySQL/MariaDB)

```
-- Création table avec contraintes
CREATE TABLE table_name (
  col1 TYPE CONSTRAINTS,
  col2 TYPE,
  CONSTRAINT pk_name PRIMARY KEY (col1),
  CONSTRAINT fk_name FOREIGN KEY (col_fk)
    REFERENCES parent_table(parent_pk)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT chk_name CHECK (condition)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Insertion multiple
INSERT INTO table (col1, col2) VALUES
(val1, val2), (val3, val4);

-- Jointure externe (gauche) pour garder les lignes sans correspondance
SELECT t1.col, t2.col FROM t1 LEFT JOIN t2 ON t1.pk = t2.fk;

-- Agrégation avec regroupement
SELECT groupe, SUM(montant) AS total FROM table GROUP BY groupe ORDER BY total DESC;

-- Recherche motif (insensible casse par défaut utf8mb4_general_ci)
WHERE nom LIKE '%nga%';

-- Valeur nulle
WHERE montant IS NULL OR montant = 0; -- ou COALESCE(SUM(montant), 0)=0
```

8.5 Démarche et corrigé commenté

EXERCICE RÉSOLU 7 Corrigé : Modélisation et Implémentation (Étapes 1 à 3)

Étape 1 — Schéma relationnel (sur papier) Application des règles de passage :

Table	Structure (PK, FK, Contraintes)
CLASSE	idClasse INT AUTO_INCREMENT PK, nomClasse VARCHAR(50) NOT NULL UNIQUE, niveau VARCHAR(30) NOT NULL, effectifMax INT NOT NULL
ELEVE	matricule CHAR(10) PK, nom VARCHAR(50) NOT NULL, prenom VARCHAR(50) NOT NULL, dateNaissance DATE NOT NULL, sexe CHAR(1) NOT NULL CHECK (sexe IN ('M','F')), idClasse INT NOT NULL, FK_eleve_classe FOREIGN KEY (idClasse) REFERENCES CLASSE(idClasse) ON DELETE RESTRICT ON UPDATE CASCADE
PAIEMENT	idPaiement INT AUTO_INCREMENT PK, matricule CHAR(10) NOT NULL, datePaiement DATE NOT NULL, montant DECIMAL(10,2) NOT NULL DEFAULT 0.00, FK_paiement_eleve FOREIGN KEY (matricule) REFERENCES ELEVE(matricule) ON DELETE RESTRICT ON UPDATE CASCADE

Schéma relationnel normalisé. Note : PAIEMENT n'est pas une entité faible au sens strict (identifiant propre auto-incrémenté), mais elle porte la FK vers ELEVE.

Étape 2 — Création de la base et des tables (SQL DDL) Ordre impératif : tables référencées (CLASSE) avant tables référençantes (ELEVE, puis PAIEMENT).

```
-- 1. Base de données
CREATE DATABASE IF NOT EXISTS scolarite
  CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
USE scolarite;

-- 2. Table CLASSE (aucune FK entrante)
CREATE TABLE CLASSE (
  idClasse INT AUTO_INCREMENT,
  nomClasse VARCHAR(50) NOT NULL UNIQUE,
  niveau VARCHAR(30) NOT NULL,
  effectifMax INT NOT NULL,
  CONSTRAINT pk_classe PRIMARY KEY (idClasse)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 3. Table ELEVE (FK vers CLASSE)
CREATE TABLE ELEVE (
  matricule CHAR(10),
  nom VARCHAR(50) NOT NULL,
  prenom VARCHAR(50) NOT NULL,
  dateNaissance DATE NOT NULL,
  sexe CHAR(1) NOT NULL,
  idClasse INT NOT NULL,
  CONSTRAINT pk_eleve PRIMARY KEY (matricule),
  CONSTRAINT fk_eleve_classe FOREIGN KEY (idClasse)
    REFERENCES CLASSE(idClasse)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT chk_sexe CHECK (sexe IN ('M','F'))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 4. Table PAIEMENT (FK vers ELEVE)
CREATE TABLE PAIEMENT (
  idPaiement INT AUTO_INCREMENT,
  matricule CHAR(10) NOT NULL,
```

```

datePaiement DATE NOT NULL,
montant DECIMAL(10,2) NOT NULL DEFAULT 0.00,
CONSTRAINT pk_paiement PRIMARY KEY (idPaiement),
CONSTRAINT fk_paiement_eleve FOREIGN KEY (matricule)
REFERENCES ELEVE(matricule)
ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Listing 1 – Script DDL complet : création base et tables

Étape 3 — Insertion des données (SQL DML) On insère d'abord les classes (id 1, 2, 3), puis les élèves (FK idClasse existante), enfin les paiements (FK matricule existant).

```

INSERT INTO CLASSE (nomClasse, niveau, effectifMax) VALUES
('Tle TI', 'Terminale', 60),
('1ere A', 'Premiere', 70),
('2nde C', 'Seconde', 65);
-- idClasse auto-générés : 1, 2, 3

```

Listing 2 – Insertion des classes (3 lignes)

```

INSERT INTO ELEVE (matricule, nom, prenom, dateNaissance, sexe, idClasse) VALUES
('2024TI001', 'Nguingo', 'Armand', '2006-05-12', 'M', 1),
('2024TI002', 'Tchounga', 'Grace', '2007-01-20', 'F', 1),
('2024TI003', 'Mbarga', 'Paul', '2006-11-30', 'M', 1),
('2024TI004', 'Nganou', 'Steve', '2007-03-15', 'M', 1),
('2024TI005', 'Fouda', 'Marie', '2006-08-22', 'F', 1),
('2024IA001', 'Essomba', 'Jean', '2007-07-10', 'M', 2),
('2024IA002', 'Nga', 'Henriette', '2008-02-14', 'F', 2),
('2024IA003', 'Bell', 'Roger', '2007-09-05', 'M', 2),
('2024IA004', 'Mvondo', 'Alice', '2008-04-18', 'F', 2),
('2024C001', 'Abena', 'Pierre', '2008-12-01', 'M', 3),
('2024C002', 'Ngah', 'Julie', '2009-01-15', 'F', 3),
('2024C003', 'Zambo', 'Kevin', '2008-06-30', 'M', 3);

```

Listing 3 – Insertion des 12 élèves

```

INSERT INTO PAIEMENT (matricule, datePaiement, montant) VALUES
('2024TI001', '2024-09-05', 50000.00),
('2024TI001', '2024-11-10', 25000.00),
('2024TI002', '2024-09-03', 75000.00),
('2024TI004', '2024-09-10', 30000.00),
('2024TI005', '2024-09-01', 75000.00),
('2024IA001', '2024-09-02', 60000.00),
('2024IA003', '2024-09-04', 60000.00),
('2024IA004', '2024-09-06', 30000.00),
('2024IA004', '2024-12-01', 30000.00),
('2024C001', '2024-09-01', 55000.00),
('2024C003', '2024-09-03', 55000.00);
-- Elèves sans paiement : 2024TI003, 2024IA002, 2024C002

```

Listing 4 – Insertion des 11 paiements (idPaiement auto)

ATTENTION Pièges classiques DDL

- Oublier `ENGINE=InnoDB` : MyISAM ne supporte pas les clés étrangères (FOREIGN KEY ignoré silencieusement).
- Inverser l'ordre de création : erreur `ERROR 1215 (HY000): Cannot add foreign key constraint`.
- Type de la FK différent de la PK référencée (ex : `INT` vs `BIGINT`, `CHAR(10)` vs `VARCHAR(10)`) → erreur 1215.
- `DEFAULT 0` sans `NOT NULL` : possible, mais `DEFAULT` ne s'applique qu'à l'insertion si la colonne est omise ; `NOT NULL` garantit l'intégrité.
- `CHAR(10)` pour `matricule` : les valeurs fournies font 9 caractères. MySQL complète par des espaces à droite. Préférer `VARCHAR(10)` en production pour éviter les surprises de comparaison (`'2024TI001' = '2024TI001 '` est vrai en comparaison mais pas en `LIKE`).

EXERCICE RÉSOLU 8 Corrigé : Requêtes du censeur (Étape 4)

Requête 1 : Élèves de Tle TI (`idClasse = 1`)

```
SELECT matricule, nom, prenom
FROM ELEVE
WHERE idClasse = 1
ORDER BY nom, prenom;
```

Résultat attendu : 5 lignes (Nguimgo, Tchounga, Mbarga, Nganou, Fouda).

Requête 2 : Noms contenant « nga » (insensible à la casse)

```
SELECT matricule, nom, prenom, idClasse
FROM ELEVE
WHERE nom LIKE '%nga%'
ORDER BY nom;
```

Justification : Collation `utf8mb4_general_ci` (ci = case insensitive) → « Nga », « Nganou », « Ngha », « Tchounga » matchent. Résultat : 4 lignes.

Requête 3 : Total paiements par classe (GROUP BY + SUM + Jointure + Tri) Il faut joindre `PAIEMENT` → `ELEVE` → `CLASSE` pour avoir le nom de la classe.

```
SELECT c.nomClasse, SUM(p.montant) AS total_encaisse
FROM PAIEMENT p
JOIN ELEVE e ON p.matricule = e.matricule
JOIN CLASSE c ON e.idClasse = c.idClasse
GROUP BY c.nomClasse
ORDER BY total_encaisse DESC;
```

Calculs attendus :

Tle TI : 50000 + 25000 + 75000 + 30000 + 75000 = 255 000

1ere A : 60000 + 60000 + 30000 + 30000 = 180 000

2nde C : 55000 + 55000 = 110 000

Tri décroissant : Tle TI, 1ere A, 2nde C.

Requête 4 : « Nom Prénom — Montant total payé » pour TOUS les élèves (Jointure EXTERNE) Point crucial : les élèves sans paiement (Mbarga, Nga, Ngah) doivent apparaître avec total 0. → LEFT JOIN depuis ELEVE vers PAIEMENT. Gestion du NULL avec COALESCE ou IFNULL.

```
SELECT
    CONCAT(e.nom, ' ', e.prenom) AS eleve,
    COALESCE(SUM(p.montant), 0) AS total_paye
FROM ELEVE e
LEFT JOIN PAIEMENT p ON e.matricule = p.matricule
GROUP BY e.matricule, e.nom, e.prenom
ORDER BY total_paye DESC;
```

Requête 5 : Élèves n'ayant rien payé (Total = 0) Deux approches équivalentes. La première utilise une CTE (MySQL 8.0+), la seconde NOT EXISTS (compatible toutes versions, souvent plus performant).

```
WITH Totaux AS (
    SELECT e.matricule, e.nom, e.prenom, COALESCE(SUM(p.montant), 0) AS total
    FROM ELEVE e LEFT JOIN PAIEMENT p ON e.matricule = p.matricule
    GROUP BY e.matricule, e.nom, e.prenom
)
SELECT matricule, nom, prenom, total
FROM Totaux
WHERE total = 0;
```

Listing 5 – Approche 1 : CTE + HAVING (MySQL 8.0+)

```
SELECT matricule, nom, prenom
FROM ELEVE e
WHERE NOT EXISTS (
    SELECT 1 FROM PAIEMENT p WHERE p.matricule = e.matricule
);
```

Listing 6 – Approche 2 : NOT EXISTS (toutes versions)

Résultat attendu : 3 lignes (Mbarga Paul, Nga Henriette, Ngah Julie).

ATTENTION Piège LEFT JOIN + GROUP BY

- Si on fait INNER JOIN (ou JOIN seul), les 3 élèves sans paiement disparaissent du résultat.

- `SUM(p.montant)` renvoie `NULL` pour un groupe sans ligne dans `PAIEMENT` → `COALESCE(SUM(...), 0)` ou `IFNULL(SUM(...), 0)` est indispensable pour afficher 0.
- Le `GROUP BY` doit contenir toutes les colonnes non agrégées du `SELECT` (mode `ONLY_FULL_GROUP_BY` activé par défaut dans MySQL 5.7+).

EXERCICE RÉSOLU 9 Corrigé : Administration et Sauvegarde (Étape 5)

Étape 5 — Administration : Export, Suppression, Restauration La procédure est réalisée via l'interface PhpMyAdmin (onglets Exporter, Opérations, Importer) ou en ligne de commande (`mysqldump`).

MÉTHODE Procédure PhpMyAdmin — Sauvegarde & Restauration

1. Export : Sélectionner base `scolarite` → Onglet Exporter → Méthode Personnalisée → Cocher « Structure et données », « Ajouter DROP TABLE / VIEW / PROCEDURE / FUNCTION / EVENT / TRIGGER », Compression « Aucune », Format « SQL » → Exécuter → Enregistrer `scolarite_backup_20250115.sql`.
2. Suppression : Onglet Opérations sur la base → « Supprimer la base de données (DROP) » → Confirmer. Ou SQL : `DROP DATABASE scolarite;`
3. Création base cible : Accueil PhpMyAdmin → « Nouvelle base » → Nom `scolarite_restored` → Interclassement `utf8mb4_general_ci` → Créer.
4. Import : Sélectionner `scolarite_restored` → Onglet Importer → Choisir fichier `.sql` → Exécuter.
5. Vérification : Exécuter `SELECT COUNT(*) FROM CLASSE;` (attendu 3), `SELECT COUNT(*) FROM ELEVE;` (attendu 12), `SELECT COUNT(*) FROM PAIEMENT;` (attendu 11). Relancer Requête 3 : résultats identiques.

EXEMPLE Extrait du fichier dump `scolarite_backup.sql` (vérification manuelle)

```
-- phpMyAdmin SQL Dump
-- ... en-têtes ...
CREATE DATABASE /*!32312 IF NOT EXISTS*/ 'scolarite' /*!40100 DEFAULT CHARACTER SET utf8mb4
  COLLATE utf8mb4_general_ci */;
USE 'scolarite';

-- Structure de la table CLASSE
```

```
DROP TABLE IF EXISTS 'CLASSE';
CREATE TABLE 'CLASSE' (...) ENGINE=InnoDB ...;

-- Déchargement des données de la table CLASSE
INSERT INTO 'CLASSE' ('idClasse', 'nomClasse', 'niveau', 'effectifMax') VALUES
(1, 'Tle TI', 'Terminale', 60),
(2, 'Iere A', 'Premiere', 70),
(3, '2nde C', 'Seconde', 65);
-- ... suites pour ELEVE et PAIEMENT ...
```

Le fichier .sql est un script texte reproductible : il contient les CREATE DATABASE, CREATE TABLE, INSERT. L'importation le rejoue à l'identique.

8.6 Bilan de l'activité

Cette activité a permis de valider l'ensemble de la chaîne de valeur « Données → Modèle → Implémentation → Exploitation → Sauvegarde » dans un SGBD relationnel réel (MySQL/MariaDB via PhpMyAdmin).

À RETENIR Compétences validées

- Modélisation : Passage rigoureux **MCD** → **MLD** (schéma relationnel) avec identification correcte des **PK**, **FK**, types, contraintes (NOT NULL, DEFAULT, CHECK, UNIQUE) et règles de référence (ON DELETE RESTRICT, ON UPDATE CASCADE).
- DDL (Data Definition Language) : Maîtrise de CREATE DATABASE, CREATE TABLE avec moteur InnoDB (obligatoire pour l'intégrité référentielle), ordre de création respectant les dépendances.
- DML (Data Manipulation Language) : INSERT multiple, gestion des valeurs auto-incrémentées (omises dans la liste des colonnes).
- DQL (Data Query Language) avancé :
 - Filtrage : opérateurs de comparaison, LIKE avec jokers %, IN, BETWEEN.
 - Jointures : INNER JOIN (correspondances strictes) vs LEFT JOIN (conservation lignes table gauche — crucial pour « élèves sans paiement »).
 - Agrégation : SUM(), COUNT(), GROUP BY (avec colonnes non agrégées listées), HAVING (filtre sur agrégat), ORDER BY (tri sur alias ou expression).
 - Gestion des NULL : COALESCE()/IFNULL() pour remplacer NULL par 0 dans les sommes.
 - Alias (AS) pour lisibilité des colonnes résultat.
- Administration (DBA junior) : Export (dump logique .sql : structure + données), suppression (DROP DATABASE), restauration par importation (rejouer le script), vérification d'intégrité post-restauration (comptages, requêtes métiers).

ATTENTION Erreurs fréquentes observées en TP — À ne plus reproduire

1. Oubli du `LEFT JOIN` pour la requête « tous les élèves même sans paiement » → perte de lignes (jointure interne implicite).
2. `GROUP BY` incomplet (mode `ONLY_FULL_GROUP_BY`) → erreur `ERROR 1055 (42000)`. Solution : lister toutes les colonnes non agrégées du `SELECT` dans le `GROUP BY` (ou utiliser `ANY_VALUE()` si sémantiquement justifié).
3. Confusion `WHERE` vs `HAVING` : `WHERE` filtre les lignes avant regroupement (pas de fonction d'agrégation autorisée) ; `HAVING` filtre les groupes après agrégation.
4. Ordre de suppression/insertion : On ne peut pas insérer un élève si sa classe n'existe pas (FK) ; on ne peut pas supprimer une classe si des élèves y sont rattachés (`ON DELETE RESTRICT`).
5. Export sans « Structure » : ne génère que les `INSERT` → échec à l'import dans une base vide (tables inexistantes).
6. Encodage : Ne pas choisir `utf8mb4` → problèmes d'accents, de caractères spéciaux (noms camerounais : « Ngà », « Été ») et tri incorrect.

LESAVAIS-TU ? Contexte camerounais : La donnée scolaire, un enjeu national

Au Cameroun, le Ministère des Enseignements Secondaires (MINESEC) déploie progressivement la plateforme SIGE (Système d'Information de Gestion de l'Éducation) pour centraliser les effectifs, les notes et la scolarité. La maîtrise du SQL et de l'administration de bases de données (MySQL/PostgreSQL) est une compétence stratégique pour les futurs techniciens supérieurs (BTS, DUT, Licence Pro) qui seront amenés à :

- Développer des modules d'interfaçage avec l'API nationale (échange JSON/XML via HTTPS).
- Assurer la sauvegarde quotidienne (scripts `mysqldump` automatisés par `cron` sur serveur Linux) et la restauration testée mensuellement (PRA — Plan de Reprise d'Activité).
- Sécuriser l'accès : comptes utilisateurs à privilèges minimaux (`GRANT SELECT, INSERT ON scolarite.* TO 'secretaire'@'localhost' IDENTIFIED BY '...'`), audit des connexions, protection contre injections SQL (requêtes paramétrées en PHP/PDO).

Cette activité est une simulation fidèle de ce que vous réaliserez en stage ou premier emploi dans une administration, une ONG ou une startup EdTech (ex : Kiro'o Games, Scholar, DiaspoPay pour les paiements Mobile Money MTN/Orange intégrés aux frais de scolarité).

9 MÉTHODES ET EXERCICES RÉSOLUS

10 IMPLÉMENTATION ET ADMINISTRATION D'UNE BASE DE DONNÉES (SGBD, SQL)

10.1 Du MCD au schéma relationnel

DÉFINITION Modèle Conceptuel de Données (MCD) et Modèle Relationnel

Le MCD (souvent représenté par le formalisme Entité-Association Merise) décrit les données indépendamment de tout SGBD : entités (objets du monde réel), propriétés (attributs), identifiants (clés candidates) et associations (liens sémantiques avec cardinalités). Le schéma relationnel est la traduction logique : un ensemble de relations (tables) composées de colonnes (attributs) et de lignes (occurrences), liées par des clés primaires et clés étrangères.

PROPRIÉTÉ Règles de passage MCD → Relationnel (Merise)

1. Entité → Table : chaque entité devient une relation. Son identifiant devient la **clé primaire** (soulignée ou PRIMARY KEY). Ses propriétés deviennent des colonnes.
2. Association binaire (1,n) – (1,1) : migration de la clé primaire de l'entité côté 1 (ou 0,1) vers la table de l'entité côté n comme **clé étrangère** (notée # ou FOREIGN KEY).
3. Association binaire (n,m) : l'association devient une table de liaison (ou table de jointure). Sa clé primaire est composée des deux clés primaires migrées (qui sont aussi clés étrangères). Les propriétés propres de l'association deviennent des colonnes de cette table.
4. Association unaire (réflexive) : même règle que binaire, la clé étrangère référence la même table (ex. EMPLOYE(#id_superviseur)).
5. Héritage / Entités faibles : selon la méthode (fusion, table par entité, table unique), l'identifiant de l'entité parente migre comme clé primaire (et étrangère) dans la table fille.

MÉTHODE Passer du MCD au schéma relationnel

1. Lister toutes les entités et créer les tables correspondantes avec leur clé primaire.
2. Analyser chaque association : noter ses cardinalités (min, max des deux côtés).
3. Appliquer la règle de migration selon le type de cardinalités (voir Propriété ci-dessus).
4. Pour chaque table, lister toutes les colonnes : clés primaires, clés étrangères (préfixées #), attributs simples.
5. Vérifier : toute clé étrangère référence une clé primaire existante ; pas de perte d'attribut ; pas de redondance évitable.

Notation normalisée : `NOM_TABLE(cle_primaire, attribut1, attribut2, #cle_etrangere_1, #cle_etrangere_2)`.

EXERCICE RÉSOLU 10 MCD d'une bibliothèque municipale de Bafoussam

La bibliothèque municipale de Bafoussam gère ses prêts avec le MCD suivant : une entité ADHERENT (id_adh, nom, quartier, telephone), une entité LIVRE (id_livre, titre, auteur, annee), et une association EMPRUNTER (date_emprunt, date_retour) de cardinalités (1,n) côté ADHERENT et (1,n) côté LIVRE (un adhérent peut emprunter plusieurs livres, un livre peut être emprunté plusieurs fois par différents adhérents à des dates différentes). Transforme ce MCD en schéma relationnel.

SOLUTION

Étape 1 — Entités → Tables. Les identifiants deviennent clés primaires.

```
ADHERENT(id_adh, nom, quartier, telephone)
LIVRE(id_livre, titre, auteur, annee)
```

Étape 2 — Analyse de l'association EMPRUNTER. Cardinalités (1,n) – (1,n) $\equiv$ type (n,m). Un adhérent emprunte n livres ; un livre est emprunté par m adhérents. L'association devient une table à part entière. Étape 3 — Clé primaire de la table de liaison. Elle est composée des deux identifiants migrants : (id_adh, id_livre). Pour distinguer deux emprunts du même livre par le même adhérent à des dates différentes, on ajoute date_emprunt à la clé primaire : (id_adh, id_livre, date_emprunt). Les propriétés de l'association (date_retour) deviennent des colonnes. Étape 4 — Schéma relationnel final :

```
ADHERENT(\underline{id_adh}, nom, quartier, telephone)
LIVRE(\underline{id_livre}, titre, auteur, annee)
EMPRUNTER(\underline{id_adh}, \underline{id_livre}, \underline{date_emprunt}, date_retour)
```

Vérification : chaque table a une clé primaire ; #id_adh référence ADHERENT(id_adh) et #id_livre référence LIVRE(id_livre) ; aucune propriété n'est perdue. Conclusion : le MCD se traduit en trois relations : ADHERENT, LIVRE et EMPRUNTER, cette dernière ma-

térialisant l'association (n,m) avec une clé primaire composée de trois attributs.

10.2 SGBD : rôle, caractéristiques et exemples

DÉFINITION Système de Gestion de Base de Données (SGBD)

Un SGBD (DBMS en anglais) est un logiciel système qui permet de créer, manipuler, sécuriser, partager et maintenir des bases de données. Il assure l'interface entre les utilisateurs/applications et les fichiers physiques de données.

PROPRIÉTÉ Fonctions essentielles d'un SGBD

- Définition des données (DDL) : création/suppression/modification de la structure (schéma).
- Manipulation des données (DML) : insertion, lecture, modification, suppression des occurrences.
- Contrôle d'accès et sécurité : gestion des utilisateurs, droits (GRANT/REVOKE), chiffrement.
- Intégrité des données : application des contraintes (clés, domaines, unicité, référentielle).
- Gestion de la concurrence : verrouillage (locks), transactions (ACID : Atomicité, Cohérence, Isolation, Durabilité).
- Sauvegarde et restauration : dump, logs, réplication, récupération après panne.
- Optimisation : indexation, plan d'exécution, cache, partitionnement.

À RETENIR Principaux SGBD du marché (2024–2025)

SGBD	Type / Licence	Usage typique au Cameroun & contexte
MySQL / MariaDB	Relationnel, Open Source (GPL)	Standard de fait pour le web PHP (WordPress, Presta-Shop), PhpMyAdmin, Wamp-Server/XAMPP. Très utilisé dans les PME, cybercafés, startups (ex. Kiro'o Games, GiftedMom).
PostgreSQL	Relationnel, Open Source (Licence BSD)	Réputé pour sa rigueur SQL standard, types avancés (JSONB, GIS), extensibilité. Choisi pour des projets critiques (banque, santé, CENADI, ANTIC).
SQLite	Embarqué, fichier unique, Domaine public	Applications mobiles (Android), apps desktop légères, prototypes, Termux sur Android.
Microsoft SQL Server	Relationnel, Propriétaire (Express gratuit)	Environnements .NET / C#, administrations utilisant l'écosystème Microsoft.
Oracle Database	Relationnel, Propriétaire	Grands comptes, banques (BICEC, Afriland), télécoms (MTN, Orange), État.
MongoDB	NoSQL (Document), SSPL	Données semi-structurées, logs, IoT, startups FinTech (API Mobile Money).

LESAVAIS-TU ?

Le Cameroun numérique et les bases de données

L'ANTIC (Agence Nationale des Technologies de l'Information et de la Communication) héberge les données sensibles de l'État (annuaire national, certificats numériques). La CENADI (Centre National de Traitement de l'Information) gère les infrastructures serveurs. Les API MTN Mobile Money (MoMo API) et Orange Money (OM API) reposent sur des clusters de bases de données distribuées (souvent PostgreSQL/Cassandra) pour garantir la disponibilité des transactions financières (USSD *126#, *150#). Le projet e-Gouv.cm et le portail impots.cm (DGI) utilisent des SGBD robustes pour la déclaration en ligne. En TP, nous utilisons MySQL/MariaDB via PhpMyAdmin (inclus dans WampServer ou XAMPP) car c'est l'environnement le plus accessible dans les lycées et cybercafés camerounais.

10.3 Définition des données : tables, types et contraintes d'intégrité

DÉFINITION Structure d'une table relationnelle

Une table (relation) est un ensemble de lignes (enregistrements, tuples) et de colonnes (champs, attributs).

- Clé primaire (Primary Key – PK) : identifiant unique, non nul, immuable (ex. `id_client`). Une seule par table.
- Clé étrangère (Foreign Key – FK) : colonne(s) référençant la clé primaire d'une autre table (table parente). Garantit l'intégrité référentielle.
- Domaine (Type de données) : ensemble de valeurs autorisées pour une colonne (entier, chaîne, date, etc.).
- Contraintes d'intégrité : règles déclaratives que le SGBD applique automatiquement.

PROPRIÉTÉ Types de données MySQL/MariaDB courants (Choix pédagogique)

Catégorie	Type SQL	Usage / Précision
Entier	TINYINT, SMALLINT, INT, BIGINT	INT (4 octets, $\pm 2 \times 10^9$) suffit pour la plupart des ID. UNSIGNED pour n'accepter que le positif.
Décimal exact	DECIMAL(m, d) ou NUMERIC(m, d)	Obligatoire pour l'argent (FCFA). m =nb total chiffres, d =décimales. Ex : DECIMAL(10,2) → 99 999 999,99 FCFA.
Flottant approx.	FLOAT, DOUBLE	Sciences, mesures. Interdit pour la monnaie (erreurs d'arrondi binaire).
Chaîne courte	VARCHAR(n)	Longueur variable $\leq n$ (ex. VARCHAR(50) pour un nom). Efficace.
Chaîne fixe	CHAR(n)	Longueur fixe (ex. CHAR(10) pour code ISO, CHAR(32) pour hash MD5).
Texte long	TEXT, LONGTEXT	Description, article, JSON brut. Pas de DEFAULT possible avant MySQL 8.0.13.
Date/Heure	DATE, TIME, DATETIME, TIMESTAMP	DATE (YYYY-MM-DD), DATETIME (YYYY-MM-DD HH :MM :SS). TIMESTAMP = UTC, zone auto.
Booléen	BOOLEAN / TINYINT(1)	Stocké comme 0 (faux) / 1 (vrai).
Énumération	ENUM('v1', 'v2')	Liste fermée (ex. statut ENUM('actif', 'bloqué', 'supprime')).

PROPRIÉTÉ Contraintes d'intégrité (Syntaxe MySQL)

- PRIMARY KEY (col) : unique + not null. Une seule par table.
- NOT NULL : valeur obligatoire.
- UNIQUE : valeurs distinctes (autorise NULL sauf si NOT NULL combiné).
- DEFAULT valeur : valeur insérée si omise (ex. DEFAULT 'Bonanjo', DEFAULT CURRENT_TIMESTAMP).
- AUTO_INCREMENT : entier incrémenté automatiquement (sur clé primaire INT ou BIGINT).
- CHECK (condition) : MySQL 8.0.16+ l'applique. Ex : CHECK (solde >= 0), CHECK (age BETWEEN 18 AND 100).

- `FOREIGN KEY (col) REFERENCES table_parent(col_parent) [ON DELETE CASCADE|SET NULL] [ON UPDATE CASCADE]` : **intégrité référentielle**. Actions en cascade optionnelles.

MÉTHODE Écrire un script SQL de création (`CREATE DATABASE / CREATE TABLE`)

1. Créer et sélectionner la base : `CREATE DATABASE nom_base CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci; puis USE nom_base;` (PhpMyAdmin : panneau gauche → Nouvelle base).
2. Ordre des tables : créer d'abord les tables parentes (sans clé étrangère), puis les tables enfants.
3. Pour chaque colonne : choisir le type le plus restrictif possible (ex. `TINYINT UNSIGNED` pour un âge, `VARCHAR(30)` pour un quartier).
4. Déclarer les contraintes : au niveau colonne (inline) ou au niveau table (après la liste des colonnes, recommandé pour clés composées et FK nommées).
5. Exécuter et vérifier : onglet SQL de PhpMyAdmin → Exécuter → `DESCRIBE nom_table;` ou `SHOW CREATE TABLE nom_table;`.
6. Modifier la structure : `ALTER TABLE nom_table avec ADD COLUMN, DROP COLUMN, CHANGE COLUMN ancien_nom nouveau_nom type, MODIFY COLUMN nom nouveau_type, ADD CONSTRAINT ... FOREIGN KEY ....`

EXERCICE RÉSOLU 11 Base de données d'un cybercafé à Douala (Bonanjo)

Un cybercafé veut créer la base `cybercafe` contenant :

- Table `CLIENT` : identifiant auto-incrémenté, nom obligatoire, téléphone unique, quartier (défaut « Bonanjo »), solde en FCFA positif ou nul.
- Table `SESSION` : identifiant auto-incrémenté, date, durée en minutes (positive), montant (défaut 0), référence au client.

Écris le script SQL complet de création.

SOLUTION

Raisonnement : base → table parente `CLIENT` → table enfant `SESSION`. On utilise `utf8mb4` pour gérer les accents et émojis. On nomme les contraintes FK pour une maintenance aisée.

```
-- 1. Creation de la base avec encodage UTF-8 complet
CREATE DATABASE cybercafe
CHARACTER SET utf8mb4
```

```
COLLATE utf8mb4_general_ci;
USE cybercafe;

-- 2. Table parente : CLIENT (aucune cle etrangere)
CREATE TABLE CLIENT (
  id_client INT UNSIGNED AUTO_INCREMENT,
  nom VARCHAR(50) NOT NULL,
  telephone VARCHAR(20) NOT NULL,
  quartier VARCHAR(30) DEFAULT 'Bonanjo',
  solde DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  date_inscription DATETIME DEFAULT CURRENT_TIMESTAMP,
  -- Contraintes de table (nommees pour diagnostic clair)
  CONSTRAINT pk_client PRIMARY KEY (id_client),
  CONSTRAINT uq_client_tel UNIQUE (telephone),
  CONSTRAINT ck_client_solde CHECK (solde >= 0)
) ENGINE=InnoDB; -- Moteur transactionnel, gere les FK

-- 3. Table enfant : SESSION (reference CLIENT)
CREATE TABLE SESSION (
  id_session INT UNSIGNED AUTO_INCREMENT,
  date_session DATE NOT NULL,
  duree SMALLINT UNSIGNED NOT NULL, -- minutes, max 65535
  montant DECIMAL(10,2) NOT NULL DEFAULT 0.00,
  id_client INT UNSIGNED NOT NULL,
  -- Contraintes
  CONSTRAINT pk_session PRIMARY KEY (id_session),
  CONSTRAINT fk_session_client
    FOREIGN KEY (id_client) REFERENCES CLIENT(id_client)
    ON DELETE RESTRICT -- Empeche suppression client s'il a des sessions
    ON UPDATE CASCADE -- Propagation si id_client change (rare)
) ENGINE=InnoDB;
```

Justification des choix techniques :

- UNSIGNED sur les ID et duree : double la capacité positive, interdit le négatif.
- DECIMAL(10,2) pour solde et montant : exactitude monétaire (FCFA).
- ENGINE=InnoDB : seul moteur supportant les clés étrangères et transactions (ACID).
- ON DELETE RESTRICT : sécurité métier (on ne supprime pas un client ayant un historique).
- DEFAULT CURRENT_TIMESTAMP : horodatage automatique de l'inscription.

Vérification : DESCRIBE CLIENT; et DESCRIBE SESSION; confirment la structure. SHOW CREATE TABLE SESSION\G affiche la définition complète avec les noms de contraintes.

10.4 Manipulation des données : INSERT, UPDATE, DELETE

MÉTHODE Requêtes DML : INSERT, UPDATE, DELETE

1. INSERT (ajout) :

```
INSERT INTO table (col1, col2, ...) VALUES (val1, val2, ...), (val3, val4, ...);
```

- Lister explicitement les colonnes (sécurité si ordre change).
- Chaînes et dates entre apostrophes simples '... '.
- Omettre les colonnes `AUTO_INCREMENT` et celles avec `DEFAULT`.
- Insertion multiple : plusieurs tuples séparés par des virgules (plus performant).

2. UPDATE (modification) :

```
UPDATE table SET col1 = expr1, col2 = expr2 WHERE condition;
```

- Clause `WHERE` obligatoire : sans elle, toutes les lignes sont modifiées.
- L'expression peut utiliser l'ancienne valeur : `SET solde = solde + 500`.
- Réflexe pro : tester d'abord avec `SELECT * FROM table WHERE condition;`.

3. DELETE (suppression) :

```
DELETE FROM table WHERE condition;
```

- Même vigilance sur `WHERE`.
- Intégrité référentielle : impossible de supprimer une ligne parente référencée par une enfant (erreur 1451). Solutions : supprimer les enfants d'abord, ou `ON DELETE CASCADE` (défini à la création), ou `SET NULL` (si FK nullable).
- `TRUNCATE TABLE table;` : vide la table entièrement, réinitialise `AUTO_INCREMENT`, plus rapide, non transactionnel (pas de `ROLLBACK`), ne déclenche pas les triggers.

EXERCICE RÉSOLU 12 Gestion des clients du cybercafé

Dans la base `cybercafe` : (a) insère la cliente « Amina Fotso », téléphone « 699112233 », quartier « Bali », solde 2000 FCFA ; (b) crédite son solde de 1500 FCFA ; (c) supprime le client d'identifiant 7, sachant qu'il n'a aucune session enregistrée.

SOLUTION

(a) Insertion : on omet `id_client` (auto) et `date_inscription` (défaut). On respecte l'ordre des colonnes listées.

```
INSERT INTO CLIENT (nom, telephone, quartier, solde)
VALUES ('Amina Fotso', '699112233', 'Bali', 2000.00);
```

```
-- 1. Verification (reflexe securite)
SELECT id_client, nom, solde FROM CLIENT WHERE telephone = '699112233';
-- 2. Mise a jour : solde = ancien + 1500
UPDATE CLIENT
SET solde = solde + 1500.00
WHERE telephone = '699112233';
-- 3. Controle post-execution
SELECT solde FROM CLIENT WHERE telephone = '699112233'; -- Doit afficher 3500.00
```

```
DELETE FROM CLIENT WHERE id_client = 7;
```

10.5 Interrogation des données : SELECT, opérateurs, alias et tri

MÉTHODE Construire une requête SELECT complète

1. Syntaxe canonique :

```
SELECT [DISTINCT] liste_colonnes_ou_expressions
FROM table_principale
[JOIN table_secondaire ON condition_liaison]
[WHERE condition_lignes]
[GROUP BY liste_colonnes_regroupement]
[HAVING condition_groupes]
[ORDER BY col1 [ASC|DESC], col2 ...]
[LIMIT nombre [OFFSET decalage]];
```

2. Opérateurs de comparaison : =, <> (standard) ou !=, <, <=, >, >=.

3. Opérateurs logiques : AND (priorité haute), OR (priorité basse), NOT. Utiliser des parenthèses pour forcer l'ordre : WHERE (A OR B) AND C.

4. Opérateurs spéciaux :

- LIKE : % (0 à n caractères), _ (exactement 1 caractère). Ex : nom LIKE 'N%' (commence par N), telephone LIKE '%55' (finit par 55).
- BETWEEN a AND b : intervalle inclusif (équivalent col >= a AND col <= b).
- IN (v1, v2, ...) / NOT IN : appartenance à une liste discrète.
- IS NULL / IS NOT NULL : test de nullité (jamais = NULL).

5. Alias (AS) : renomme une colonne ou une expression dans le résultat. Obligatoire pour

les colonnes calculées ($\text{solde} * 1.05$ AS solde_majore) et dans les jointures pour préfixer ($C.\text{nom}$).

6. Tri (ORDER BY) : ASC (défaut, croissant), DESC (décroissant). Peut trier sur alias ou position (ORDER BY 2 DESC).

EXERCICE RÉSOLU 13 Recherches dans la table CLIENT du cybercafé

Écris les requêtes SQL qui : (a) affichent les clients des quartiers « Bali » ou « Akwa » dont le solde > 1000 ; (b) affichent les clients dont le nom commence par « N » ; (c) affichent pour chaque client le solde majoré de 5 % sous le nom solde_bonus , trié par solde décroissant ; (d) affichent les clients dont le solde est entre 500 et 5000 FCFA.

SOLUTION

(a) Priorité AND > OR → parenthèses obligatoires sur le OR. Équivalent IN plus lisible.

```
SELECT * FROM CLIENT
WHERE (quartier = 'Bali' OR quartier = 'Akwa')
AND solde > 1000;
-- Ou plus propre :
SELECT * FROM CLIENT
WHERE quartier IN ('Bali', 'Akwa') AND solde > 1000;
```

(b) Motif de début de chaîne.

```
SELECT * FROM CLIENT WHERE nom LIKE 'N%';
```

(c) Expression calculée + alias + tri sur colonne d'origine (ou alias).

```
SELECT nom, solde, solde * 1.05 AS solde_bonus
FROM CLIENT
ORDER BY solde DESC;
```

Exemple : solde 3500 → $3500 \times 1.05 = 3675.00$ FCFA. (d) Intervalle inclusif.

```
SELECT * FROM CLIENT WHERE solde BETWEEN 500 AND 5000;
```

Conclusion : ces requêtes couvrent les opérateurs logiques, les motifs, les données calculées/alias/tri, et les intervalles.

10.6 Agrégation, regroupement et jointures

MÉTHODE Fonctions d'agrégation, GROUP BY, HAVING et Jointures

1. Fonctions d'agrégation (retournent une valeur unique par groupe) : COUNT(*) (nb

lignes), COUNT(col) (nb non-null), SUM(col), AVG(col), MIN(col), MAX(col).

2. Regroupement (GROUP BY) : divise le jeu de résultats en groupes partageant la même valeur(s) de colonne(s). Le SELECT ne peut contenir que : colonnes du GROUP BY, fonctions d'agrégation, constantes.
3. Filtrage des groupes (HAVING) : condition après regroupement, peut contenir des agrégats. WHERE = avant regroupement (lignes), HAVING = après (groupes).
4. Jointure interne (INNER JOIN ou JOIN) : ne garde que les lignes ayant une correspondance dans les deux tables.

```
SELECT C.nom, S.date_session
FROM CLIENT C
JOIN SESSION S ON C.id_client = S.id_client;
```

5. Jointures externes : LEFT JOIN : toutes les lignes de la table de gauche + correspondances (NULL si absent). RIGHT JOIN : symétrique.
6. Auto-jointure : table jointe à elle-même (alias différents obligatoires).
7. Ordre logique d'exécution : FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.

EXERCICE RÉSOLU 14 Statistiques de gestion du cybercafé

Tables : CLIENT(id_client, nom, quartier, solde) et SESSION(id_session, date_session, duree, montant, #id_client). Requêtes : (a) nb total sessions; (b) recette totale et durée moyenne; (c) nb clients par quartier (quartiers ≥ 3 clients); (d) nom client + total dépensé (tri décroissant).

SOLUTION

(a) Agrégat global simple.

```
SELECT COUNT(*) AS nb_sessions FROM SESSION;
```

(b) Deux agrégats simultanés.

```
SELECT SUM(montant) AS recette_totale,
       AVG(duree) AS duree_moyenne_min
FROM SESSION;
```

(c) Regroupement + filtre sur agrégat (→ HAVING).

```
SELECT quartier, COUNT(*) AS nb_clients
FROM CLIENT
GROUP BY quartier
HAVING COUNT(*) >= 3;
```

(d) Jointure + regroupement par client + agrégat somme.

```
SELECT C.nom, SUM(S.montant) AS total_depense
FROM CLIENT C
JOIN SESSION S ON C.id_client = S.id_client
GROUP BY C.id_client, C.nom -- C.nom fonctionnellement depend de C.id_client (PK)
ORDER BY total_depense DESC;
```

Justification : la jointure associe chaque session à son client ; GROUP BY forme un groupe par client ; SUM additionne les montants du groupe ; alias C, S pour lisibilité. Conclusion : indicateurs clés (fréquentation, CA, géographie, top clients) pour le gérant.

10.7 Administration : exportation, importation et sauvegarde

MÉTHODE Sauvegarder et restaurer une base MySQL/MariaDB

1. Exportation (Dump) via PhpMyAdmin : Base → Onglet Exporter → Méthode « Rapide » (toutes tables) ou « Personnalisée » (choix tables, options : DROP TABLE, AUTO_INCREMENT, CREATE DATABASE) → Format SQL → Exécuter. Fichier .sql téléchargé.

2. Exportation ligne de commande (production, automatisation) :

```
mysqldump -u root -p --single-transaction --routines --triggers cybercafe > cybercafe_$(date +%F).sql
```

--single-transaction : cohérence sans verrouiller (InnoDB). --routines --triggers : inclut procédures/fonctions/triggers.

3. Importation via PhpMyAdmin : Créer/Sélectionner base cible → Onglet Importer → Choisir fichier .sql → Exécuter. Attention à max_execution_time et upload_max_filesize (php.ini) pour gros fichiers.

4. Importation ligne de commande :

```
mysql -u root -p cybercafe < cybercafe_2024-05-10.sql
```

5. Bonnes pratiques professionnelles :

- Sauvegardes quotidiennes (incrémentielles via logs binaires) + hebdomadaires (complètes).
- Stockage externe (disque USB, NAS, cloud Wasabi, AWS S3, Azure Blob).
- Test de restauration mensuel sur serveur de recette (procédure Disaster Recovery Plan).
- Chiffrement des dumps sensibles (gpg -c fichier.sql).

EXPÉRIENCE TP guidé : Migration complète d'un cybercafé (Bonanjo → Nouveau poste)

Objectif : Transférer la base `cybercafe` d'un poste source (WampServer) vers un poste cible neuf, vérifier l'intégrité. Matériel/Logiciels : 2 postes Windows 10/11, WampServer 3.3+ (Apache, MySQL 8.0, PHP 8.2), clé USB 8 Go, navigateur. Manipulation :

1. Poste source (Export) : Lancer WampServer (icône verte). Ouvrir `http://localhost/phpmyadmin`. Clic sur base `cybercafe` → Exporter → « Personnalisée » : cocher « Structure et données », « Ajouter DROP TABLE », « AUTO_INCREMENT », « Créer la base ». Format SQL. Exécuter. Enregistrer `cybercafe_2024-06-15.sql` sur clé USB.
2. Poste cible (Install) : Installer WampServer (même version). Vérifier icône verte. Ouvrir PhpMyAdmin.
3. Poste cible (Import) : Onglet Bases de données → Créer `cybercafe` (interclassement `utf8mb4_general_ci`). Sélectionner `cybercafe` → Importer → Choisir fichier sur clé USB → Exécuter. Attendre « Importation terminée avec succès ».
4. Vérification croisée :

```
-- Sur les DEUX postes, comparer :  
SHOW TABLES;  
SELECT COUNT(*) AS nb_clients FROM CLIENT;  
SELECT COUNT(*) AS nb_sessions FROM SESSION;  
SELECT SUM(montant) AS ca_total FROM SESSION;  
SELECT quartier, COUNT(*) FROM CLIENT GROUP BY quartier;
```

5. Test métier : Simuler une insertion (nouveau client + session) sur le poste cible. Vérifier `AUTO_INCREMENT` continu.

Observations attendues : Identiques compteurs, même structure (`DESCRIBE`), même comportement applicatif. Interprétation : Le fichier `.sql` est une représentation textuelle portable du schéma + données. L'importation rejoue les ordres DDL/DDI. C'est la base de la sauvegarde/restauration et de la réplication logique.

ATTENTION Les 5 erreurs fatales au Baccalauréat (et en production)

1. WHERE manquant dans UPDATE/DELETE. → Massacre de données. Toujours écrire le WHERE et le tester par un SELECT préalable.
2. Confusion WHERE / HAVING. WHERE filtre lignes (pas d'agrégat). HAVING filtre groupes (agrégats autorisés). WHERE `COUNT(*) > 3` = Erreur syntaxe/sémantique.
3. Mauvaise translation association (n,m). Créer une seule FK dans une des tables au lieu d'une table de liaison à clé composée. Perte de la sémantique « plusieurs-à-plusieurs ».

4. Oubli apostrophes / priorité opérateurs. WHERE nom = Bali (erreur colonne incon-
nue) vs WHERE nom = 'Bali'. WHERE ville='Yaounde' OR ville='Douala' AND solde>0
≠ WHERE (ville='Yaounde' OR ville='Douala') AND solde>0.
5. Jointure sans condition (Produit cartésien). FROM CLIENT, SESSION sans ON/WHERE →
N × M lignes. Toujours JOIN ... ON PK = FK et préfixer colonnes ambiguës (C.nom,
S.montant).

EXERCICE 7 Entraînement Bac : Gestion d'un stock de pièces informatiques

On souhaite gérer le stock d'une boutique de pièces informatiques à Yaoundé (quartier Mvog-Ada). MCD fourni :

- Entité FOURNISSEUR (id_four, nom_four, ville, telephone).
- Entité PIECE (ref_piece, designation, prix_unitaire, stock_actuel, stock_alerte).
- Association FOURNIR (delai_livraison_jours) entre FOURNISSEUR (1,n) et PIECE (1,n).
Une pièce a un fournisseur principal ; un fournisseur fournit plusieurs pièces.

Travail demandé :

1. Proposer le schéma relationnel normalisé (notation TABLE(PK, col, #FK)).
2. Écrire le script SQL complet de création de la base stock_pieces et des tables avec contraintes adaptées (types, CHECK, DEFAULT, FK).
3. Insérer 2 fournisseurs (ex. « TechPlus Douala », « InfoKit Yaoundé ») et 3 pièces (ex. « SSD 1To », « RAM 16Go DDR4 », « Alim 600W ») avec leurs liaisons.
4. Écrire la requête qui affiche la désignation, le prix, le stock et le nom du fournisseur pour les pièces dont le stock est inférieur ou égal au stock d'alerte (rupture imminente).
5. Écrire la requête qui donne, pour chaque fournisseur, le nombre de pièces fournies et le prix moyen de ces pièces, en ne gardant que les fournisseurs proposant au moins 2 pièces.

CORRIGÉ Exercice : Gestion d'un stock de pièces informatiques

1. Schéma relationnel

Association FOURNIR (1,n) – (1,n) → migration clé id_four dans table PIECE (côté n).

FOURNISSEUR(\underline{id_four}, nom_four, ville, telephone)

PIECE(\underline{ref_piece}, designation, prix_unitaire, stock_actuel, stock_alerte, #id_four)

2. Script SQL de création

```
CREATE DATABASE stock_pieces CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
USE stock_pieces;
```

```
CREATE TABLE FOURNISSEUR (
  id_four INT UNSIGNED AUTO_INCREMENT,
  nom_four VARCHAR(60) NOT NULL,
  ville VARCHAR(40) NOT NULL,
  telephone VARCHAR(20),
  CONSTRAINT pk_fournisseur PRIMARY KEY (id_four)
) ENGINE=InnoDB;
```

```
CREATE TABLE PIECE (
  ref_piece VARCHAR(20) NOT NULL, -- Ex : 'SSD-1TO-SAM', 'RAM-16G-DDR4'
  designation VARCHAR(100) NOT NULL,
  prix_unitaire DECIMAL(10,2) NOT NULL CHECK (prix_unitaire > 0),
  stock_actuel INT UNSIGNED NOT NULL DEFAULT 0,
  stock_alerte INT UNSIGNED NOT NULL DEFAULT 5,
  id_four INT UNSIGNED NOT NULL,
  CONSTRAINT pk_piece PRIMARY KEY (ref_piece),
  CONSTRAINT fk_piece_fournisseur
    FOREIGN KEY (id_four) REFERENCES FOURNISSEUR(id_four)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT ck_stock_alerte CHECK (stock_alerte >= 0)
) ENGINE=InnoDB;
```

3. Insertion de données

```
-- Fournisseurs
INSERT INTO FOURNISSEUR (nom_four, ville, telephone) VALUES
('TechPlus Douala', 'Douala', '690112233'),
('InfoKit Yaounde', 'Yaounde', '677445566');

-- Pieces (id_four recupere : 1 pour TechPlus, 2 pour InfoKit)
INSERT INTO PIECE (ref_piece, designation, prix_unitaire, stock_actuel, stock_alerte, id_four) VALUES
('SSD-1TO-SAM', 'SSD 1To Samsung 870 EVO', 45000.00, 3, 5, 1),
('RAM-16G-DDR4', 'RAM 16Go DDR4 3200MHz Corsair', 38000.00, 2, 3, 2),
('ALIM-600W-MS', 'Alimentation 600W MSI MAG A650BN', 32000.00, 10, 4, 1);
```

4. Pièces en alerte de stock (rupture imminente)

Jointure + condition sur colonnes de la même table.

```
SELECT P.designation, P.prix_unitaire, P.stock_actuel, P.stock_alerte, F.nom_four
FROM PIECE P
JOIN FOURNISSEUR F ON P.id_four = F.id_four
WHERE P.stock_actuel <= P.stock_alerte;
```

Résultat attendu : SSD 1To (stock $3 \leq 5$) et RAM 16Go (stock $2 \leq 3$). 5. Statistiques par fournisseur (min 2 pièces)

```
SELECT F.nom_four, COUNT(P.ref_piece) AS nb_pieces, AVG(P.prix_unitaire) AS prix_moyen
FROM FOURNISSEUR F
JOIN PIECE P ON F.id_four = P.id_four
GROUP BY F.id_four, F.nom_four
HAVING COUNT(P.ref_piece) >= 2;
```

Résultat attendu : « TechPlus Douala » : 2 pièces (SSD, Alim), prix moyen (45000 +

$32000)/2 = 38500$. « InfoKit Yaounde » : 1 pièce → exclu par `HAVING`.

11 EXERCICES

11.1 Je vérifie mes connaissances

EXERCICE 8 QCM : SGBD et tables

Pour chaque question, une seule réponse est correcte. Entoure la bonne lettre.

1. Un SGBD est :
 - (a) un langage de programmation ;
 - (b) un logiciel qui permet de créer, administrer et exploiter des bases de données ;
 - (c) un composant matériel de l'ordinateur ;
 - (d) un protocole réseau.
2. Dans une table, une clé primaire :
 - (a) peut prendre la valeur `NULL` ;
 - (b) peut être dupliquée sur plusieurs lignes ;
 - (c) identifie de manière unique chaque enregistrement ;
 - (d) est toujours de type `VARCHAR`.
3. La requête qui supprime définitivement la table `ELEVE` et toutes ses données est :
 - (a) `DELETE TABLE ELEVE ;`
 - (b) `DROP TABLE ELEVE ;`
 - (c) `REMOVE ELEVE ;`
 - (d) `TRUNCATE DATABASE ELEVE ;`
4. La fonction d'agrégation qui calcule la moyenne des valeurs d'une colonne est :
 - (a) `SUM()` ;
 - (b) `COUNT()` ;
 - (c) `AVG()` ;
 - (d) `MAX()`.

EXERCICE 9 Vrai ou faux — justifier

Réponds par vrai ou faux, puis justifie ta réponse en une ou deux phrases.

1. Une clé étrangère d'une table fait référence à la clé primaire d'une autre table.
2. La contrainte `DEFAULT` interdit qu'une colonne contienne la valeur `NULL`.
3. L'opérateur `LIKE 'A%'` sélectionne les chaînes qui se terminent par la lettre A.
4. MySQL et PostgreSQL sont des exemples de SGBD relationnels.
5. La clause `HAVING` s'utilise avec `GROUP BY` pour filtrer les groupes, alors que `WHERE` filtre les lignes avant regroupement.
6. `UPDATE` permet d'ajouter une nouvelle colonne à une table existante.

EXERCICE 10 Définitions et vocabulaire

1. Définis les termes suivants : **SGBD**, **clé primaire**, **clé étrangère**, **contrainte d'intégrité**.
2. Donne deux exemples de SGBD autres que MySQL.
3. Cite les trois grandes familles de requêtes SQL vues en classe et donne pour chacune un exemple de mot-clé.
4. Explique la différence entre un **champ** (colonne) et un **enregistrement** (ligne) dans une table.

EXERCICE 11 Lecture de requêtes

Soit la table `NOTE(id_note, matricule, matiere, valeur, trimestre)`. Pour chacune des requêtes suivantes, décris en français le résultat obtenu :

1. `SELECT * FROM NOTE WHERE valeur >= 10 AND trimestre = 1 ;`
2. `SELECT COUNT(*) FROM NOTE WHERE matiere = 'Informatique' ;`
3. `SELECT matricule, AVG(valeur) AS moyenne FROM NOTE GROUP BY matricule ;`
4. `SELECT * FROM NOTE WHERE matiere IN ('Maths', 'Physique') ORDER BY valeur DESC ;`

11.2 Je m'entraîne

EXERCICE 12 Du MCD au schéma relationnel

On considère le MCD suivant, modélisant la gestion des commandes d'une boutique en ligne de Douala :

- CLIENT (id_client, nom, telephone, ville);
 - PRODUIT (id_produit, designation, prix);
 - COMMANDER : un client passe une ou plusieurs commandes; une commande porte sur un ou plusieurs produits avec une quantité et une date_commande.
1. Transforme ce MCD en schéma relationnel : donne la liste des relations (tables) avec leurs attributs, en soulignant les clés primaires et en indiquant les clés étrangères par le symbole #.
 2. Justifie la création de la table issue de l'association COMMANDER.

EXERCICE 13 Création de tables avec contraintes

En t'appuyant sur le schéma relationnel obtenu à l'exercice précédent :

1. Écris la requête SQL de création de la base de données `boutique_douala`, puis l'instruction qui permet de la sélectionner pour y travailler (USE).
2. Écris les requêtes CREATE TABLE des tables CLIENT et PRODUIT en imposant les contraintes suivantes : le nom du client est obligatoire, la ville vaut 'Douala' par défaut, le prix d'un produit doit être strictement positif.
3. Écris la requête qui ajoute à la table CLIENT une colonne `email` de type chaîne de 100 caractères, devant être unique.

EXERCICE 14 Requêtes de manipulation

Soit la table PRODUIT(id_produit, designation, prix, quantite, categorie) contenant les données suivantes :

id_produit	designation	prix (FCFA)	quantite	categorie
1	Savon liquide 1 L	2500	40	Hygiène
2	Riz parfumé 5 kg	4500	25	Céréale
3	Jus de bissap 1 L	1000	60	Boisson
4	Huile raffinée 5 L	9000	15	Alimentaire
5	Eau minérale 1,5 L	500	100	Boisson

Écris les requêtes SQL permettant de :

1. insérer le produit suivant : (6, 'Maïs grain 10 kg', 6000, 30, 'Céréale');
2. afficher les produits dont le prix est compris entre 1000 et 5000 FCFA (utiliser BETWEEN);

3. afficher les produits des catégories « Boisson » ou « Céréale » (utiliser `IN`);
4. augmenter de 10 % le prix de tous les produits de la catégorie « Hygiène »;
5. supprimer les produits dont la quantité en stock est nulle;
6. afficher les désignations qui commencent par la lettre « R ».

EXERCICE 15 Agrégation, tri et alias

Toujours à partir de la table `PRODUIT` de l'exercice précédent, écris les requêtes SQL qui :

1. affichent le nombre total de produits enregistrés;
2. affichent le prix le plus élevé, le prix le plus bas et le prix moyen, avec les alias `prix_max`, `prix_min` et `prix_moyen`;
3. affichent la valeur totale du stock par catégorie (valeur = prix × quantité), avec l'alias `valeur_stock`, triée par valeur décroissante;
4. affichent uniquement les catégories dont la valeur totale du stock dépasse 100 000 FCFA (utiliser `HAVING`).

11.3 Je me prépare au Bac

EXERCICE 16 Type Bac — Gestion d'une bibliothèque scolaire

Le lycée bilingue de Nkongsamba souhaite informatiser sa bibliothèque. Le documentaliste a conçu le MCD suivant :

- ADHERENT (id_adherent, nom, classe, telephone);
 - LIVRE (id_livre, titre, auteur, annee_edition);
 - EMPRUNT : un adhérent effectue un ou plusieurs emprunts; chaque emprunt concerne un livre et possède les attributs `date_emprunt` et `date_retour_prevue`.
1. (2 pts) Transforme ce MCD en schéma relationnel : donne les relations avec leurs attributs, leurs clés primaires (soulignées) et leurs clés étrangères (marquées #).
 2. (3 pts) Écris la requête de création de la base `biblio_nkongsamba`, puis les requêtes `CREATE TABLE` des trois tables en respectant les contraintes suivantes : le nom de l'adhérent et le titre du livre sont obligatoires; l'année d'édition doit être comprise entre 1950 et 2025; les clés étrangères de la table `EMPRUNT` doivent référencer les tables parentes.
 3. (1,5 pt) Écris la requête qui affiche la liste des adhérents (nom, classe) ayant emprunté le livre de titre 'Les Misérables' (utiliser une jointure).

4. (2 pts) Écris la requête qui affiche, pour chaque adhérent, son nom et le nombre de livres empruntés (alias `nb_emprunts`), triée par nombre décroissant.
5. (1,5 pt) Écris la requête qui affiche uniquement les adhérents ayant emprunté plus de 3 livres (utiliser `GROUP BY` et `HAVING`).

EXERCICE 17 Type Bac — Correction d'un script et contraintes d'intégrité

L'entreprise camerounaise AgroMarket SARL, spécialisée dans la vente de produits agricoles en ligne, fait appel à toi comme technicien TI. Le stagiaire a rédigé le script SQL suivant, qui contient quatre erreurs :

```
CREATE DATABASE agromarket

CREATE TABLE CATEGORIE (
    id_cat INT,
    libelle VARCHAR(50)
);

CREATE TABLE ARTICLE (
    id_art INT,
    designation VARCHAR(80),
    prix VARCHAR(20),
    id_cat INT,
    FOREIGN KEY (id_cat) REFERENCES CATEGORIE(id_cat)
);
```

1. (2 pts) Définis les notions de **clé primaire** et de **clé étrangère**, puis cite trois autres contraintes d'intégrité étudiées en classe.
2. (4 pts) Relève les quatre erreurs du script (clé primaire absente, point-virgule manquant, type de données inadapté pour le prix, clé étrangère référençant une colonne qui n'est pas clé primaire) et réécris le script corrigé et complet, en ajoutant l'instruction `USE`, la contrainte `NOT NULL` sur la désignation et une valeur par défaut 0 sur le prix.
3. (2 pts) On exécute ensuite la requête : `SELECT libelle, COUNT(id_art) AS nb_articles FROM CATEGORIE C JOIN ARTICLE A ON C.id_cat = A.id_cat GROUP BY libelle ;`. Explique ligne par ligne le rôle de cette requête et précise le rôle des alias `C`, `A` et `nb_articles`.
4. (2 pts) Le directeur veut exporter la base chaque soir vers un disque dur externe. Décris la procédure d'exportation de la base `agromarket` au format `.sql` à l'aide de Php-MyAdmin, puis cite deux avantages de cette sauvegarde régulière.

EXERCICE 18 Type Bac — Incident et restauration au lycée de Bafoussam

La veille des conseils de classe, le serveur du lycée de Bafoussam tombe en panne : le disque dur est hors service. Heureusement, l'administrateur réseau effectue chaque soir à 22 h une sauvegarde complète de la base `notes_lycee` (tables `ELEVE`, `MATIERE`, `NOTE`) vers un fichier `notes_lycee.sql` stocké sur un serveur distant. Un nouveau serveur MySQL vient d'être installé.

La table `NOTE` a la structure suivante : `NOTE(id_note, id_eleve#, id_matiere#, valeur, sequence)`.

1. (2 pts) Décris, étape par étape, la procédure de restauration de la base à partir du fichier `notes_lycee.sql` en utilisant PhpMyAdmin (création de la base vide, importation, vérification).
2. (1,5 pt) Justifie le choix d'une sauvegarde quotidienne et externalisée (sur un serveur distant) plutôt qu'une sauvegarde hebdomadaire sur le même disque. Cite deux risques évités.
3. (2 pts) Une fois la base restaurée, le proviseur demande la liste des élèves de la classe de « Terminale TI » dont la moyenne en informatique est supérieure ou égale à 12. On donne : `ELEVE(id_eleve, nom, classe)` et `MATIERE(id_matiere, intitule)`. Écris la requête complète (jointures, condition sur la matière et la classe, regroupement, `HAVING` sur la moyenne, alias `moyenne_info`, tri alphabétique des noms).
4. (1,5 pt) Le proviseur souhaite aussi connaître les élèves dont le nom contient la syllabe « nga » (par exemple Nganou, Tchoupo-Ngassa). Écris la requête utilisant l'opérateur `LIKE` adapté.
5. (1 pt) Pour éviter qu'un tel incident ne se reproduise, propose deux mesures d'administration complémentaires (matérielles ou logicielles) à mettre en place au lycée.

12 CORRIGÉS DES EXERCICES

CORRIGÉ Exercice 1 — QCM : SGBD et tables

1. Réponse b. Un **SGBD** (Système de Gestion de Bases de Données) est un logiciel qui permet de créer, administrer et exploiter des bases de données (exemples : MySQL, PostgreSQL, Oracle). Ce n'est ni un langage (le langage utilisé est le SQL), ni un composant matériel, ni un protocole réseau.
2. Réponse c. La **clé primaire** identifie de manière unique chaque enregistrement d'une table. Elle ne peut donc jamais être NULL ni dupliquée, et elle peut être de tout type (souvent INT).
3. Réponse b. `DROP TABLE ELEVE` ; supprime la structure de la table et toutes ses données, de façon définitive. `DELETE` supprime des lignes mais conserve la table ; `REMOVE` et `TRUNCATE DATABASE` n'existent pas en SQL standard.
4. Réponse c. `AVG()` (average, moyenne) calcule la moyenne arithmétique des valeurs d'une colonne. `SUM()` calcule la somme, `COUNT()` compte les lignes, `MAX()` donne la plus grande valeur.

CORRIGÉ Exercice 2 — Vrai ou faux — justifier

1. Vrai. Par définition, une **clé étrangère** (FOREIGN KEY) est une colonne d'une table qui fait référence à la clé primaire d'une autre table (ou de la même table) ; elle garantit l'intégrité référentielle entre les tables.
2. Faux. La contrainte `DEFAULT` attribue une valeur par défaut à une colonne lorsque l'utilisateur n'en fournit pas à l'insertion. C'est la contrainte `NOT NULL` qui interdit la valeur NULL.
3. Faux. Le joker % remplace zéro, un ou plusieurs caractères. `LIKE 'A%'` sélectionne les chaînes qui commencent par A. Pour celles qui se terminent par A, il faudrait écrire `LIKE '%A'`.
4. Vrai. MySQL et PostgreSQL sont deux **SGBDR** (SGBD relationnels) libres et très répandus : les données y sont organisées en tables reliées par des clés.
5. Vrai. `WHERE` filtre les lignes avant le regroupement, tandis que `HAVING` filtre les groupes formés par `GROUP BY`, après calcul des agrégats. `HAVING` ne s'emploie donc qu'avec `GROUP BY`.
6. Faux. `UPDATE` modifie les données (les valeurs des lignes). Pour ajouter une colonne, on utilise `ALTER TABLE ... ADD COLUMN`, qui appartient au langage de définition des don-

nées (LDD).

CORRIGÉ Exercice 3 — Définitions et vocabulaire

1. Définitions.

- **SGBD** : logiciel qui permet de créer, décrire, manipuler, administrer et sécuriser des bases de données, en offrant un langage d'interrogation (SQL) et des mécanismes de contrôle d'accès.
- **Clé primaire** : attribut (ou ensemble d'attributs) dont la valeur identifie de manière unique chaque enregistrement d'une table ; elle est non nulle et non dupliquée.
- **Clé étrangère** : attribut d'une table qui fait référence à la clé primaire d'une autre table, afin de matérialiser un lien entre les deux tables.
- **Contrainte d'intégrité** : règle imposée aux données d'une table (clé primaire, NOT NULL, UNIQUE, DEFAULT, CHECK, clé étrangère) pour garantir la validité et la cohérence de la base.

2. Deux exemples de SGBD autres que MySQL : PostgreSQL et Oracle Database (on accepte aussi SQLite, Microsoft SQL Server, MariaDB, Access).

3. Les trois grandes familles de requêtes SQL :

- le LDD (Langage de Définition des Données) : CREATE, ALTER, DROP ;
- le LMD (Langage de Manipulation des Données) : INSERT, UPDATE, DELETE ;
- le LCD (Langage de Contrôle des Données) : GRANT, REVOKE (l'interrogation SELECT est souvent classée dans le LMD ou citée à part comme DQL).

4. Un **champ** (colonne) décrit une propriété commune à tous les enregistrements (ex. la colonne `nom`) : il possède un nom et un type de données. Un **enregistrement** (ligne) est une occurrence concrète de la table : un ensemble de valeurs, une par champ (ex. la ligne (1, 'Mbarga', 'Tle TI')). Autrement dit, les colonnes définissent la structure, les lignes contiennent les données.

CORRIGÉ Exercice 4 — Lecture de requêtes

1. Affiche toutes les informations (toutes les colonnes) des notes supérieures ou égales à 10 obtenues au premier trimestre. Les deux conditions doivent être vraies simultanément (AND).
2. Compte et affiche le nombre total de notes enregistrées en Informatique (toutes lignes confondues, grâce à COUNT(*)).

3. Affiche, pour chaque élève (regroupement par `matricule`), son `matricule` et la moyenne de toutes ses notes, la colonne calculée étant renommée `moyenne` grâce à l'alias `AS`.
4. Affiche toutes les notes obtenues en Maths ou en Physique (l'opérateur `IN` teste l'appartenance à une liste de valeurs), triées de la plus grande valeur à la plus petite (`ORDER BY valeur DESC`).

CORRIGÉ Exercice 5 — Du MCD au schéma relationnel

1. Schéma relationnel obtenu :

- CLIENT (id_client, nom, telephone, ville);
- PRODUIT (id_produit, designation, prix);
- COMMANDER (#id_client, #id_produit, date_commande, quantite).

La clé primaire de `COMMANDER` est le couple (`id_client`, `id_produit`); chacun de ses deux attributs est en même temps clé étrangère vers la table parente correspondante.

2. Justification. L'association `COMMANDER` relie plusieurs clients à plusieurs produits : c'est une association de type plusieurs-à-plusieurs (cardinalités 1,n de part et d'autre). Or le modèle relationnel interdit les listes de valeurs dans une case : on ne peut pas stocker plusieurs produits dans une ligne de `CLIENT`, ni plusieurs clients dans une ligne de `PRODUIT`. La règle de transformation impose donc de convertir toute association plusieurs-à-plusieurs en une table supplémentaire, dont la clé primaire est formée des clés primaires des deux entités reliées, et qui porte les attributs propres à l'association (`date_commande`, `quantite`).

Point de vigilance : ne pas oublier de reporter les attributs de l'association (`quantite`, `date_commande`) dans la nouvelle table, et de souligner les deux attributs de la clé primaire composée.

CORRIGÉ Exercice 6 — Création de tables avec contraintes

1. Création et sélection de la base :

```
CREATE DATABASE boutique_douala;  
USE boutique_douala;
```

2. Création des tables `CLIENT` et `PRODUIT` avec les contraintes demandées :

```
CREATE TABLE CLIENT (  
    id_client INT PRIMARY KEY AUTO_INCREMENT,
```

```
nom VARCHAR(50) NOT NULL,           -- nom obligatoire
telephone VARCHAR(20),
ville VARCHAR(50) DEFAULT 'Douala'  -- valeur par défaut
);

CREATE TABLE PRODUIT (
  id_produit INT PRIMARY KEY AUTO_INCREMENT,
  designation VARCHAR(80),
  prix DECIMAL(10,2) CHECK (prix > 0) -- prix strictement positif
);
```

La contrainte `CHECK (prix > 0)` traduit la règle « valeur autorisée » : toute insertion d'un prix nul ou négatif sera rejetée par le SGBD (supporté depuis MySQL 8.0.16).

3. Ajout de la colonne `email` unique :

```
ALTER TABLE CLIENT ADD COLUMN email VARCHAR(100) UNIQUE;
```

La contrainte `UNIQUE` garantit que deux clients ne pourront jamais partager la même adresse électronique.

Point de vigilance : `ALTER TABLE` (et non `UPDATE`) est l'instruction du LDD qui modifie la structure d'une table existante.

CORRIGÉ Exercice 7 — Requêtes de manipulation

1. Insertion du nouveau produit (on précise les colonnes, l'identifiant 6 est fourni pour l'exercice) :

```
INSERT INTO PRODUIT (id_produit, designation, prix, quantite, categorie)
VALUES (6, 'Maïs grain 10 kg', 6000, 30, 'Cereale');
```

2. Produits dont le prix est compris entre 1 000 et 5 000 FCFA (bornes incluses) :

```
SELECT * FROM PRODUIT WHERE prix BETWEEN 1000 AND 5000;
```

Résultat attendu sur le tableau : Savon liquide (2 500), Riz parfumé (4 500) et Jus de bissap (1 000).

3. Produits des catégories « Boisson » ou « Céréale » :

```
SELECT * FROM PRODUIT WHERE categorie IN ('Boisson', 'Cereale');
```

Résultat attendu : Riz parfumé, Jus de bissap, Eau minérale (puis Maïs grain après insertion).

4. Augmentation de 10 % du prix des produits d'hygiène (multiplier par $1 + \frac{10}{100} = 1,1$) :

```
UPDATE PRODUIT SET prix = prix * 1.1 WHERE categorie = 'Hygiene';
```

Le prix du Savon liquide passe ainsi de 2 500 à $2500 \times 1,1 = 2750$ FCFA.

5. Suppression des produits en rupture de stock :

```
DELETE FROM PRODUIT WHERE quantite = 0;
```

Sur le tableau actuel, aucun produit n'est supprimé car toutes les quantités sont strictement positives.

6. Désignations commençant par « R » :

```
SELECT designation FROM PRODUIT WHERE designation LIKE 'R%';
```

Résultat attendu : « Riz parfumé 5 kg ». Le joker % placé après le R signifie « n'importe quelle suite de caractères ».

Point de vigilance : ne jamais oublier la clause WHERE dans un UPDATE ou un DELETE : sans elle, toutes les lignes de la table seraient modifiées ou supprimées.

CORRIGÉ Exercice 8 — Agrégation, tri et alias

1. Nombre total de produits :

```
SELECT COUNT(*) AS nb_produits FROM PRODUIT;
```

Résultat sur le tableau initial : 5 (puis 6 après l'insertion de l'exercice 7).

2. Prix extrêmes et prix moyen avec alias :

```
SELECT MAX(prix) AS prix_max,  
       MIN(prix) AS prix_min,  
       AVG(prix) AS prix_moyen  
FROM PRODUIT;
```

Vérification sur le tableau initial : prix_max = 9000, prix_min = 500, et

$$\text{prix_moyen} = \frac{2500 + 4500 + 1000 + 9000 + 500}{5} = \frac{17500}{5} = 3500 \text{ FCFA.}$$

3. Valeur totale du stock par catégorie (donnée calculée, regroupement, tri décroissant) :

```
SELECT categorie,  
       SUM(prix * quantite) AS valeur_stock  
FROM PRODUIT  
GROUP BY categorie  
ORDER BY valeur_stock DESC;
```

Vérification sur le tableau initial :

Alimentaire : $9000 \times 15 = 135\,000$,

Céréale : $4500 \times 25 = 112\,500$,

Hygiène : $2500 \times 40 = 100\,000$,

Boisson : $(1000 \times 60) + (500 \times 100) = 60\,000 + 50\,000 = 110\,000$.

L'ordre d'affichage est donc : Alimentaire (135 000), Céréale (112 500), Boisson (110 000), Hygiène (100 000).

4. Catégories dont la valeur de stock dépasse 100 000 FCFA : on filtre les groupes avec HAVING :

```
SELECT categorie,
       SUM(prix * quantite) AS valeur_stock
FROM PRODUIT
GROUP BY categorie
HAVING SUM(prix * quantite) > 100000;
```

Résultat attendu : Alimentaire (135 000), Céréale (112 500) et Boisson (110 000). La catégorie Hygiène (exactement 100 000) est exclue car la condition est strictement supérieure.

Point de vigilance : on ne peut pas écrire `WHERE SUM(...) > 100000` : WHERE s'applique aux lignes avant le regroupement et ne connaît pas encore les agrégats ; c'est précisément le rôle de HAVING.

CORRIGÉ Exercice 9 — Type Bac : Gestion d'une bibliothèque scolaire

Barème indicatif : $2 + 3 + 1,5 + 2 + 1,5 = 10$ points.

1. Schéma relationnel (2 pts). L'association EMPRUNT étant plusieurs-à-plusieurs (un adhérent fait plusieurs emprunts, un livre est emprunté plusieurs fois), elle devient une table :

- ADHERENT (id_adherent, nom, classe, telephone);
- LIVRE (id_livre, titre, auteur, annee_edition);
- EMPRUNT (id_emprunt, #id_adherent, #id_livre, date_emprunt, date_retour_prevue).

On introduit un identifiant propre `id_emprunt` car un même adhérent peut emprunter le même livre à des dates différentes : le couple (`id_adherent`, `id_livre`) ne suffirait pas à distinguer deux emprunts successifs. Point de vigilance : souligner les clés primaires et marquer les deux clés étrangères par #.

2. Création de la base et des tables (3 pts).

```
CREATE DATABASE biblio_nkongsamba;
USE biblio_nkongsamba;

CREATE TABLE ADHERENT (
    id_adherent INT PRIMARY KEY AUTO_INCREMENT,
    nom VARCHAR(50) NOT NULL,    -- nom obligatoire
    classe VARCHAR(20),
    telephone VARCHAR(20)
);

CREATE TABLE LIVRE (
    id_livre INT PRIMARY KEY AUTO_INCREMENT,
    titre VARCHAR(100) NOT NULL, -- titre obligatoire
    auteur VARCHAR(60),
    annee_edition INT CHECK (annee_edition BETWEEN 1950 AND 2025)
);

CREATE TABLE EMPRUNT (
    id_emprunt INT PRIMARY KEY AUTO_INCREMENT,
    id_adherent INT NOT NULL,
    id_livre INT NOT NULL,
    date_emprunt DATE,
    date_retour_prevue DATE,
    FOREIGN KEY (id_adherent) REFERENCES ADHERENT(id_adherent),
    FOREIGN KEY (id_livre) REFERENCES LIVRE(id_livre)
);
```

Points de vigilance : créer les tables parentes avant la table EMPRUNT (sinon les clés étrangères ne peuvent pas être déclarées); chaque FOREIGN KEY doit référencer la clé primaire de la table parente.

3. Adhérents ayant emprunté « Les Misérables » (1,5 pt). La condition porte sur le titre (table LIVRE) et le résultat sur les adhérents : il faut donc deux jointures enchaînées.

```
SELECT AD.nom, AD.classe
FROM ADHERENT AD
JOIN EMPRUNT E ON AD.id_adherent = E.id_adherent
JOIN LIVRE L ON E.id_livre = L.id_livre
WHERE L.titre = 'Les Misérables';
```

4. Nombre de livres empruntés par adhérent (2 pts).

```
SELECT AD.nom, COUNT(E.id_emprunt) AS nb_emprunts
FROM ADHERENT AD
JOIN EMPRUNT E ON AD.id_adherent = E.id_adherent
GROUP BY AD.id_adherent, AD.nom
ORDER BY nb_emprunts DESC;
```

Point de vigilance : le regroupement se fait sur l'identifiant (et le nom) de l'adhérent; l'alias nb_emprunts peut être utilisé dans ORDER BY.

5. Adhérents ayant emprunté plus de 3 livres (1,5 pt).

```
SELECT AD.nom, COUNT(E.id_emprunt) AS nb_emprunts
FROM ADHERENT AD
```

```
JOIN EMPRUNTE ON AD.id_adherent = E.id_adherent  
GROUP BY AD.id_adherent, AD.nom  
HAVING COUNT(E.id_emprunt) > 3;
```

La condition sur l'agrégat (> 3) se place obligatoirement dans HAVING, jamais dans WHERE.

CORRIGÉ Exercice 10 — Type Bac : Correction d'un script et contraintes d'intégrité

Barème indicatif : 2 + 4 + 2 + 2 = 10 points.

1. Définitions (2 pts). Une **clé primaire** est un attribut (ou groupe d'attributs) qui identifie de manière unique chaque enregistrement d'une table ; elle est non nulle et non dupliquée. Une **clé étrangère** est un attribut d'une table qui fait référence à la clé primaire d'une autre table, assurant l'intégrité référentielle. Trois autres contraintes d'intégrité : NOT NULL (valeur obligatoire), UNIQUE (unicité), DEFAULT (valeur par défaut) ; on accepte aussi CHECK (valeur autorisée / format).
2. Les quatre erreurs (4 pts).
 - (a) Point-virgule manquant à la fin de CREATE DATABASE agromarket : chaque instruction SQL doit se terminer par ; ;
 - (b) Clé primaire absente : ni CATEGORIE ni ARTICLE ne déclarent de PRIMARY KEY ;
 - (c) Type inadapté pour le prix : prix VARCHAR(20) stocke le prix comme du texte, ce qui empêche les calculs (sommes, moyennes) et les comparaisons numériques ; il faut un type numérique (DECIMAL ou INT) ;
 - (d) Clé étrangère invalide : FOREIGN KEY (id_cat) REFERENCES CATEGORIE(id_cat) ne peut fonctionner que si CATEGORIE.id_cat est clé primaire (ou unique), ce qui n'est pas le cas dans le script du stagiaire.

Script corrigé et complet :

```
CREATE DATABASE agromarket ;  
USE agromarket ;  
  
CREATE TABLE CATEGORIE (  
    id_cat INT PRIMARY KEY AUTO_INCREMENT,  
    libelle VARCHAR(50)  
);  
  
CREATE TABLE ARTICLE (  
    id_art INT PRIMARY KEY AUTO_INCREMENT,  
    designation VARCHAR(80) NOT NULL,  
    prix DECIMAL(10,2) DEFAULT 0,  
    id_cat INT,  
    FOREIGN KEY (id_cat) REFERENCES CATEGORIE(id_cat)  
);
```

3. Explication de la requête (2 pts).

- `SELECT libelle, COUNT(id_art) AS nb_articles` : on affiche le libellé de chaque catégorie et le nombre d'articles qui lui sont rattachés; l'alias `nb_articles` renomme la colonne calculée pour la lisibilité du résultat;
- `FROM CATEGORIE C JOIN ARTICLE A ON C.id_cat = A.id_cat` : on réalise une jointure entre les deux tables en faisant correspondre la clé étrangère `A.id_cat` avec la clé primaire `C.id_cat`; les alias `C` et `A` sont des noms raccourcis qui évitent de réécrire les noms complets des tables et lèvent toute ambiguïté sur la colonne `id_cat`, présente dans les deux tables;
- `GROUP BY libelle` : les lignes sont regroupées par catégorie afin que `COUNT` soit calculé pour chaque groupe et non sur toute la table.

Au final, la requête affiche le nombre d'articles par catégorie.

4. Exportation via PhpMyAdmin (2 pts). Procédure :

- ouvrir PhpMyAdmin dans le navigateur et se connecter;
- cliquer sur la base `agromarket` dans le panneau de gauche;
- cliquer sur l'onglet Exporter, choisir la méthode Rapide (ou Personnalisée) et le format SQL;
- cliquer sur Exécuter : le fichier `agromarket.sql` est téléchargé;
- copier ce fichier sur le disque dur externe.

Deux avantages de la sauvegarde régulière : (1) pouvoir restaurer rapidement les données en cas de panne, de suppression accidentelle ou d'attaque (rançongiciel); (2) limiter les pertes : avec une sauvegarde quotidienne, on perd au maximum une journée de données. On accepte aussi : archivage, migration vers un autre serveur, conservation de l'historique.

CORRIGÉ Exercice 11 — Type Bac : Incident et restauration au lycée de Bafoussam

Barème indicatif : $2 + 1,5 + 2 + 1,5 + 1 = 8$ points.

1. Procédure de restauration (2 pts).

- installer et démarrer MySQL sur le nouveau serveur, ouvrir PhpMyAdmin et se connecter;
- créer une base vide : onglet Bases de données, saisir le nom `notes_lycee`, cliquer sur Créer;
- sélectionner cette base dans le panneau de gauche, puis cliquer sur l'onglet Importer;

- (d) cliquer sur Choisir un fichier, sélectionner `notes_lycee.sql` (format SQL), puis cliquer sur Exécuter : les tables `ELEVE`, `MATIERE`, `NOTE` sont recréées et remplies ;
- (e) vérifier : contrôler la présence des trois tables, le nombre d'enregistrements (`SELECT COUNT(*)`) et exécuter quelques requêtes de test pour valider l'intégrité des données.
2. Justification de la sauvegarde quotidienne et externalisée (1,5 pt). Une sauvegarde quotidienne limite la perte de données à une journée au maximum (ici, les notes saisies la veille), alors qu'une sauvegarde hebdomadaire pourrait faire perdre jusqu'à une semaine de saisies. Une sauvegarde externalisée (serveur distant) survit à la destruction du serveur local. Deux risques évités : (1) la perte totale des données en cas de panne matérielle du disque (cas présent) ; (2) la destruction simultanée des données et de leur sauvegarde en cas d'incendie, de vol ou de dégât des eaux dans la salle serveur.
3. Requête du proviseur (2 pts). Il faut joindre les trois tables, filtrer sur la classe et la matière, regrouper par élève, puis filtrer les groupes sur la moyenne :

```
SELECT E.nom, AVG(N.valeur) AS moyenne_info
FROM ELEVE E
JOIN NOTE N ON E.id_eleve = N.id_eleve
JOIN MATIERE M ON N.id_matiere = M.id_matiere
WHERE E.classe = 'Terminale TI'
      AND M.intitule = 'Informatique'
GROUP BY E.id_eleve, E.nom
HAVING AVG(N.valeur) >= 12
ORDER BY E.nom ASC;
```

Points de vigilance : la condition sur la matière et la classe (lignes) va dans `WHERE` ; la condition sur la moyenne (agrégat) va dans `HAVING` ; le tri alphabétique s'obtient avec `ORDER BY E.nom ASC`.

4. Noms contenant « nga » (1,5 pt). Le joker % est placé des deux côtés de la syllabe, car elle peut apparaître n'importe où dans le nom :

```
SELECT * FROM ELEVE WHERE nom LIKE '%nga%';
```

Cette requête retrouve par exemple Nganou (début), Tchoupo-Ngassa (milieu) ou Manga (fin).

5. Mesures complémentaires (1 pt). Deux propositions parmi :
- installer un onduleur (UPS) pour protéger le serveur des coupures et surtensions électriques, fréquentes dans nos localités ;
 - mettre en place un second disque en miroir (RAID 1) pour que la panne d'un disque n'interrompe pas le service ;
 - automatiser la sauvegarde par une tâche planifiée (script `mysqldump` quotidien) plutôt qu'une copie manuelle ;
 - définir des comptes et privilèges MySQL distincts (`GRANT`) pour limiter les suppres-

sions accidentelles.

13 FICHE BILAN

Implémentation & Administration BDD

MCD → Schéma Relationnel

- Entités → Tables
- Associations → Clés étrangères
- Identifiants → Clés primaires

SGBD & Architecture

- MySQL, PostgreSQL
- Client-Serveur (PhpMyAdmin)
- Moteur InnoDB

DDL : Définition

- CREATE / DROP DATABASE
- CREATE / ALTER / DROP TABLE
- Contraintes : PK, FK, NOT NULL, UNIQUE, DEFAULT, CHECK

DML : Manipulation

- INSERT INTO ... VALUES
- UPDATE ... SET ... WHERE
- DELETE FROM ... WHERE
- SELECT ... FROM ... WHERE

Requêtes Avancées

- Jointures : INNER, LEFT, RIGHT
- Agrégation : COUNT, SUM, AVG, MIN, MAX
- GROUP BY / HAVING
- ORDER BY / LIMIT / Alias

Administration & Sécurité

- Sauvegarde : mysqldump
- Restauration : mysql < fichier.sql
- Utilisateurs / Privilèges (GRANT/REVOKE)

Opérateurs & Filtres

- Comparaison : =, <>, <, >, <=, >=
- Logiques : AND, OR, NOT
- Spéciaux : LIKE, BETWEEN, IN, IS NULL

Carte mentale : Leçon 14 — Implémentation et administration d'une base de données (SGBD, SQL)

FICHE BILAN

L'essentiel à connaître pour l'examen

13.1 Définitions Clés

DÉFINITION Modèle Conceptuel de Données (MCD)

Représentation graphique (Entités–Associations) indépendante du SGBD, décrivant la structure des données : entités (rectangles), associations (losanges), identifiants (soulignés), cardinalités (0,1; 1,1; 0,n; 1,n).

DÉFINITION Schéma Relationnel (Modèle Logique de Données – MLD)

Ensemble de relations (tables) issues de la transformation du MCD. Notation : `NomRelation(PK, attribut1, attribut2, ..., #FK_TableRef)`.

DÉFINITION SGBD (Système de Gestion de Base de Données)

Logiciel serveur (ex. MySQL/MariaDB, PostgreSQL) gérant le stockage, la sécurité, la concurrence et l'accès aux données via le langage SQL. Architecture Client–Serveur (port 3306 par défaut pour MySQL).

DÉFINITION Clé Primaire (Primary Key – PK)

Attribut ou groupe d'attributs identifiant uniquement chaque enregistrement (ligne). Contraintes : **NOT NULL** + **UNIQUE**. Une seule PK par table.

DÉFINITION Clé Étrangère (Foreign Key – FK)

Attribut d'une table référençant la PK d'une autre table (table parente). Garantit l'intégrité référentielle : toute valeur de FK doit exister comme PK dans la table parente (ou être **NULL** si autorisé).

13.2 Commandes SQL Essentielles (À connaître par cœur)

Catégorie	Commande / Syntaxe	Exemple / Note
DDL (Définition)	<pre>CREATE DATABASE nom; DROP DATABASE nom; CREATE TABLE t (c1 TYPE CONTRAINTES, ...); ALTER TABLE t ADD COLUMN c TYPE; ALTER TABLE t DROP COLUMN c; ALTER TABLE t MODIFY c NEW_TYPE; ALTER TABLE t ADD CONSTRAINT fk_name FOREIGN KEY (c) REFERENCES parent(pk);</pre>	<pre>CREATE DATABASE ecole_db CHARACTER SET utf8mb4;</pre> Suppression définitive (attention!) <pre>id INT AUTO_INCREMENT PRIMARY KEY</pre> Ajout colonne Suppression colonne Changement type (risque perte données) Ajout FK explicite
DML (Manipulation)	<pre>INSERT INTO t (c1, c2) VALUES (v1, v2); UPDATE t SET c1=v1, c2=v2 WHERE cond; DELETE FROM t WHERE cond;</pre>	Ordre colonnes = ordre valeurs. DEFAULT ou NULL si absent. Obligatoire : clause WHERE sinon MAJ toute la table! Obligatoire : clause WHERE sinon vidage table (TRUNCATE plus rapide).
DQL (Interrogation)	<pre>SELECT [DISTINCT] c1, c2 AS alias FROM t WHERE cond ORDER BY c1 [DESC] LIMIT n; SELECT c1, COUNT(*) AS nb FROM t GROUP BY c1 HAVING COUNT(*) > 1;</pre>	AS optionnel. LIMIT pour pagination. WHERE filtre lignes avant groupement, HAVING filtre groupes après.
Jointures	<pre>SELECT * FROM A INNER JOIN B ON A.fk = B.pk; SELECT * FROM A LEFT JOIN B ON A.fk = B.pk;</pre>	INNER : intersection (correspondances only). LEFT : tout A + B correspondantes (NULL si absent).
Agrégation	<pre>COUNT(*), COUNT(col), SUM(col), AVG(col), MIN(col), MAX(col)</pre>	COUNT(*) compte lignes (même NULL). COUNT(col) ignore NULL.
Admin	<pre>mysqldump -u user -p db_name > backup.sql mysql -u user -p db_name < backup.sql GRANT ALL PRIVILEGES ON db.* TO 'user'@'%' IDENTIFIED BY 'pwd';</pre>	Ligne de commande (shell). Structure + Données. Restoration. Création utilisateur + droits (MySQL 5.7-). MySQL 8.0+ : <pre>CREATE USER ...; GRANT ...;</pre>

13.3 Méthodes Express

MÉTHODE Transformer un MCD en Schéma Relationnel (Règles standards)

1. Entité forte → Table. Identifiant → Clé Primaire (PK). Attributs → Colonnes.
2. Entité faible (identifiant relatif) → Table. Clé primaire composée : (PK entité propriétaire, identifiant relatif).
3. Association 1 : n (ou 0 : n / 1 : 1) → Clé étrangère dans la table côté "n" (ou côté "1" pour 1 : 1), référençant la PK du côté "1".
4. Association n : m → Table de liaison (table de jointure). PK composée = (FK_TableA, FK_TableB). Deux FK vers les deux tables parentes.
5. Association avec attributs → Table (même si 1 : n ou 1 : 1). Attributs de l'association deviennent colonnes de cette table.
6. Héritage (IS-A) → Soit table unique (attributs nullable + type), soit table par classe concrète, soit table par classe (PK commune = FK).

MÉTHODE Écrire une Requête SELECT Complexe (Jointure + Agrégation)

1. Identifier les tables nécessaires (d'après les colonnes demandées dans SELECT/WHERE/ORDER BY).
2. Déterminer les jointures (INNER pour correspondances obligatoires, LEFT pour garder lignes table gauche).
3. Écrire la clause FROM + JOIN ... ON
4. Ajouter WHERE pour filtres sur lignes brutes (ex. date, statut).
5. Lister colonnes SELECT : colonnes de groupement + fonctions d'agrégation. Utiliser AS pour alias lisibles.
6. Ajouter GROUP BY sur toutes les colonnes non agrégées du SELECT.
7. Ajouter HAVING pour filtres sur résultats agrégés (ex. HAVING COUNT(*) > 5).
8. Ajouter ORDER BY (alias ou position ou expression) et LIMIT si besoin.

MÉTHODE Sauvegarder et Restaurer une Base MySQL (Ligne de commande / Terminal)

1. Ouvrir un terminal (invite de commandes Windows, Bash Linux, ou Git Bash).
2. Export (Dump) : `mysqldump -u root -p --databases ma_base > C:/sauvegardes/ma_base_$(date +%F).sql` (saisir mot de passe).
3. Vérifier le fichier .sql (texte lisible : CREATE TABLE, INSERT).
4. Import (Restauration) : Créer la BD vide si besoin (`CREATE DATABASE ma_base;`). Puis : `mysql -u root -p ma_base < C:/sauvegardes/ma_base_2025-01-15.sql`.
5. Bonnes pratiques : Automatiser via `cron` (Linux) / Planificateur de tâches (Windows). Stocker hors serveur (NAS, Cloud). Tester la restauration régulièrement.

À RETENIR Checklist avant le Bac : Leçon 14 — BDD & SQL

☐ Savoir transformer un MCD (entités, associations 1 : n, n : m, identifiants relatifs, héritage) en schéma relationnel textuel et créer les tables SQL correspondantes (PK, FK, types adaptés : INT, VARCHAR(n), DATE, DECIMAL(p,s), ENUM, BOOLEAN). ☐ Maîtriser la syntaxe exacte de CREATE TABLE avec contraintes : PRIMARY KEY, FOREIGN KEY (...) REFERENCES ..., NOT NULL, UNIQUE, DEFAULT 'valeur', CHECK (condition). ☐ Différencier WHERE (filtre lignes avant groupement) et HAVING (filtre groupes après GROUP BY). ☐ Écrire une jointure INNER JOIN (intersection) et LEFT JOIN (tout gauche + correspondance droite) avec condition ON correcte (FK = PK). ☐ Utiliser les fonctions d'agrégation : COUNT(*) vs COUNT(colonne), SUM, AVG, MIN, MAX; regrouper avec GROUP BY (toutes colonnes non agrégées du SELECT). ☐ Gérer les opérateurs : LIKE (% , _), BETWEEN ... AND ..., IN (... , ...), IS NULL / IS NOT NULL, AND / OR / NOT (priorité : NOT > AND > OR, utiliser parenthèses). ☐ Savoir ordonner (ORDER BY col ASC/DESC), limiter (LIMIT n OFFSET m), aliaser (AS alias ou col alias). ☐ Connaître les commandes d'administration : mysqldump (export structure + données), mysql < fichier.sql (import), gestion utilisateurs (CREATE USER, GRANT, REVOKE, FLUSH PRIVILEGES). ☐ Piège UPDATE/DELETE : Jamais sans clause WHERE (sauf intention explicite de tout modifier/supprimer). Tester d'abord avec un SELECT * FROM table WHERE ☐ Intégrité référentielle : On ne peut insérer une FK qui n'existe pas en PK parente; on ne peut supprimer une PK parente si des lignes filles existent (sauf ON DELETE CASCADE/SET NULL défini à la création FK). ☐ Types de données : Choisir DECIMAL(10,2) pour l'argent (pas FLOAT/DOUBLE – imprécision binaire), DATE pour date seule, DATETIME pour date+heure, VARCHAR pour texte variable, TEXT pour long texte. ☐ Encodage : Toujours CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci (ou utf8mb4_general_ci) à la création de la BD et des tables pour supporter accents, émojis,

langues africaines.



Signé • OnBuch+

Ludovic A.
CONFORME AU PROGRAMME OFFICIEL MINESEC

OnBuch+